

FORMULARIO DE PRODUCCIÓN DE CONTENIDOS

NOMBRE DE LA ASIGNATURA: Desarrollo en plataformas

NOMBRE COMPLETO DEL DOCENTE CONTENIDISTA: Damián Aníbal Nicolalde Rodríguez

1. INTRODUCCIÓN DE LA CLASE

La tecnología actual se construye sobre un ecosistema diverso de plataformas que evolucionan constantemente para responder a las demandas de escalabilidad, seguridad y accesibilidad. En esta primera clase exploraremos el panorama del desarrollo en plataformas, revisando cómo hemos pasado de aplicaciones monolíticas a arquitecturas distribuidas y servicios en la nube. Asimismo, estudiaremos los principales tipos de plataformas —web, escritorio e híbridas—, entendiendo sus ventajas, limitaciones y escenarios de uso. Esta visión inicial permitirá que, desde el primer momento, conectes la teoría con el futuro desarrollo de proyectos que demandan tomar decisiones sobre el entorno más adecuado para implementar una solución.

De igual manera, profundizaremos en la arquitectura cliente-servidor moderna, base de la mayoría de aplicaciones actuales, y en las herramientas de desarrollo contemporáneas, que permiten colaborar en entornos distribuidos y automatizar procesos de despliegue. Finalmente, revisaremos el HTML5 semántico y la accesibilidad web, componentes esenciales para construir aplicaciones inclusivas y alineadas con estándares internacionales. Con este recorrido, tendrás los fundamentos necesarios para afrontar los primeros retos de la asignatura, comprendiendo no solo cómo programar, sino también cómo diseñar soluciones eficientes, seguras y accesibles en distintos entornos de desarrollo.

Clase 1: Panorama del desarrollo en plataformas

RDA 1: Programar aplicaciones web dinámicas, interactivas y con diseño visualmente adaptable, que permita mejorar la experiencia del usuario.

Reto # 1

1.1 Panorama del desarrollo en plataformas

El concepto de *plataforma de desarrollo* se ha transformado radicalmente en las últimas décadas, respondiendo a la evolución tecnológica, a las necesidades empresariales y a la experiencia del usuario. En sus orígenes, el desarrollo de software se limitaba a aplicaciones monolíticas, instaladas de forma local en equipos de escritorio y con un alcance restringido al hardware específico donde operaban. Sin embargo, la llegada de internet, la masificación de dispositivos móviles y el crecimiento de la computación en la nube han provocado una reconfiguración del ecosistema. Hoy en día, los desarrolladores trabajan con arquitecturas distribuidas, servicios escalables y entornos de integración continua que permiten construir soluciones ágiles, modulares y con un alto grado de interoperabilidad.

Este panorama es especialmente relevante porque establece la base para comprender las diferencias y ventajas de los distintos tipos de plataformas que se verán en los apartados posteriores (web, escritorio e híbridas).

Evolución histórica del desarrollo en plataformas

En la década de los 80 y 90, la mayoría de los sistemas se construían como **aplicaciones monolíticas**: grandes bloques de código donde la interfaz gráfica, la lógica de negocio y la gestión de datos estaban fuertemente acopladas. Este enfoque resultaba funcional en entornos reducidos, pero presentaba problemas graves al crecer: la modificación de un módulo afectaba a toda la aplicación, dificultando la escalabilidad y el mantenimiento.

Con el auge de internet a finales de los 90, se consolidó la **arquitectura cliente-servidor**. Aquí, la lógica de negocio se trasladaba a servidores centralizados, mientras que los clientes solo necesitaban un navegador o aplicación ligera para interactuar. Este modelo permitió optimizar los recursos y abrir paso a aplicaciones accesibles desde cualquier lugar. Sin embargo, también introdujo nuevos desafíos: seguridad, latencia y compatibilidad entre dispositivos.

En la actualidad, el panorama está dominado por **arquitecturas distribuidas, microservicios y computación en la nube**. Empresas como Netflix, Amazon o Google han demostrado la importancia de separar la lógica en servicios independientes que se comunican entre sí mediante APIs. Esto facilita el escalado horizontal, la resiliencia ante fallos y la posibilidad de desplegar nuevas funcionalidades sin detener el sistema completo.

Factores que impulsan la evolución

Existen tres factores principales que explican por qué las plataformas de desarrollo han evolucionado:

1. **Escalabilidad**: los sistemas modernos requieren soportar millones de usuarios simultáneos, lo que obliga a diseñar arquitecturas flexibles.
2. **Experiencia del usuario (UX)**: se demanda que las aplicaciones sean rápidas, intuitivas y accesibles desde múltiples dispositivos.
3. **Integración con tecnologías emergentes**: inteligencia artificial, big data, blockchain y realidad aumentada son ejemplos de tendencias que requieren plataformas adaptables.

Estos factores, además, explican por qué el estudiante deberá aprender no solo a programar, sino también a diseñar soluciones que tengan en cuenta **rendimiento, seguridad y accesibilidad**.

Estado actual del desarrollo en plataformas

Hoy, el desarrollo de software se caracteriza por la diversidad de plataformas disponibles. Se construyen aplicaciones para web, escritorio, móviles y entornos híbridos, todas coexistiendo y respondiendo a necesidades específicas. Además, se ha generalizado el uso **de** entornos de desarrollo colaborativos (como GitHub o GitLab), que permiten a equipos distribuidos trabajar de manera simultánea, implementar pruebas automáticas y desplegar aplicaciones en minutos gracias a pipelines de CI/CD (*Continuous Integration/Continuous Deployment*).

Otro rasgo esencial es la **democratización del desarrollo**. Herramientas *low-code* y *no-code* facilitan que usuarios sin conocimientos avanzados construyan aplicaciones básicas. Sin embargo, para proyectos de nivel empresarial o científico sigue siendo indispensable dominar

los fundamentos de programación estructurada, orientada a objetos y arquitecturas modernas, que son la base de esta asignatura.

Ejemplos prácticos

Para ilustrar este panorama, consideremos dos ejemplos:

- **Netflix:** opera sobre una arquitectura de microservicios desplegada en la nube de Amazon (AWS). Cada microservicio maneja una funcionalidad (catálogo, recomendaciones, facturación). Esto asegura que, si un módulo falla, los demás sigan funcionando.
- **Google Docs:** funciona bajo una arquitectura cliente-servidor moderna, pero con sincronización en tiempo real y escalado global. Permite que miles de usuarios trabajen sobre el mismo documento con mínima latencia.

Estos ejemplos muestran que el desarrollo en plataformas no es estático, sino un proceso de adaptación constante a nuevos contextos y demandas.



Imagen 1. Línea de tiempo de la evolución del desarrollo en plataformas: monolítico (1990) → cliente-servidor (2000) → servicios distribuidos (2010) → microservicios en la nube (2015+). (Creación propia del autor, Damián Nicolalde, 2025).

1.2 Tipos de plataformas: web, escritorio, híbridas

En el panorama tecnológico actual, comprender los **tipos de plataformas** constituye un paso fundamental para diseñar soluciones de software adaptadas a contextos específicos. Las tres principales categorías —**plataformas web, de escritorio e híbridas**— representan diferentes enfoques de desarrollo, cada una con ventajas, limitaciones y casos de uso particulares. Este

análisis prepara al estudiante para tomar decisiones fundamentadas al momento de implementar estructuras y algoritmos en los retos de la asignatura, evaluando qué tipo de plataforma ofrece mayor eficiencia, usabilidad y escalabilidad según el problema planteado.

1.2.1 Plataformas web

Las plataformas web son aquellas cuyo acceso se realiza a través de un navegador y que dependen de una conexión a internet. La gran ventaja de este modelo es la **portabilidad**: una aplicación web puede ejecutarse en cualquier dispositivo que disponga de un navegador moderno, sin necesidad de instalaciones adicionales.

Desde un punto de vista técnico, las aplicaciones web modernas se basan en **arquitecturas de cliente ligero y servidor robusto**, donde la lógica principal está en el backend, mientras que el frontend gestiona la interfaz de usuario y parte de la interacción mediante JavaScript. Tecnologías como **React, Angular o Vue.js** permiten construir interfaces dinámicas, mientras que frameworks como **Django, Laravel o Express.js**, ASPNet Core, ASPNet Framework, Laravel, CodeIgniter se encargan de la lógica del servidor.

Ventajas principales:

- Accesibilidad global.
- Actualización centralizada (los cambios se aplican directamente en el servidor).
- Escalabilidad y facilidad de integración con APIs externas.

Limitaciones:

- Dependencia de la conexión a internet.
- Restricciones de rendimiento frente a aplicaciones nativas.
- Limitado acceso a hardware específico (GPU, drivers especializados).

Ejemplo: Google Drive, que ofrece almacenamiento y edición de documentos accesible desde cualquier dispositivo conectado.

1.2.2 Plataformas de escritorio

Las plataformas de escritorio son aplicaciones que se instalan directamente en el sistema operativo del usuario (Windows, macOS o Linux). Históricamente, han sido el estándar del software debido a su capacidad de aprovechar al máximo los recursos del hardware.

A nivel técnico, las aplicaciones de escritorio ofrecen **alto rendimiento y control completo** del sistema. Lenguajes como C++, C# o Java han sido pilares en este tipo de desarrollo, y en la actualidad frameworks como **.NET, JavaFX o Qt** siguen siendo relevantes.

Ventajas principales:

- Mayor rendimiento y acceso completo al hardware.
- Posibilidad de trabajar sin conexión a internet.
- Experiencia de usuario optimizada y fluida.

Limitaciones:

- Requieren procesos de instalación y actualización complejos.
- Menor portabilidad frente a soluciones web.
- Dependencia del sistema operativo y sus librerías.

Ejemplo: AutoCAD, herramienta de diseño asistido por computadora que exige procesamiento intensivo y acceso directo a la GPU.

1.2.3 Plataformas híbridas

Las plataformas híbridas buscan combinar lo mejor de los dos mundos: la portabilidad de la web con la potencia de las aplicaciones nativas. Se construyen sobre tecnologías web (HTML, CSS, JavaScript), pero se empaquetan para ejecutarse como aplicaciones de escritorio o móviles mediante frameworks como **Electron, Ionic, Flutter**, React Native.

En este modelo, el código base puede reutilizarse para múltiples entornos, lo que reduce tiempos y costos de desarrollo. Sin embargo, el rendimiento puede no ser tan alto como el de aplicaciones nativas, ya que dependen de capas de compatibilidad.

Ventajas principales:

- Reutilización de código en distintas plataformas.
- Reducción de tiempos de desarrollo y costos de mantenimiento.
- Acceso a hardware mediante APIs específicas (ej. cámaras, sensores).

Limitaciones:

- Rendimiento inferior en escenarios de alto consumo de recursos.
- Tamaño de instalación elevado (ejemplo: aplicaciones con Electron).
- Complejidad en la depuración por capas intermedias.

Ejemplo: Visual Studio Code, desarrollado en Electron, funciona de forma idéntica en Windows, Linux y macOS.

1.2.4 Comparación técnica

La elección de una plataforma no es trivial: depende de los objetivos, del tipo de usuario y de los recursos disponibles. A continuación, se presenta una tabla comparativa:

Tabla 1. Comparación de tipos de plataformas

Característica	Web	Escritorio	Híbrida
Acceso	Navegador	Instalación local	Navegador + empaquetado
Rendimiento	Medio	Alto	Medio-alto
Portabilidad	Alta	Baja	Alta
Dependencia de internet	Sí	No	Parcial
Ejemplo	Google Drive	AutoCAD	VS Code

Comprender el contexto de cada plataforma facilita tomar decisiones correctas en diseño e implementación, asegurando que las soluciones no solo funcionen en un entorno académico, sino que también sean viables en escenarios de la industria.

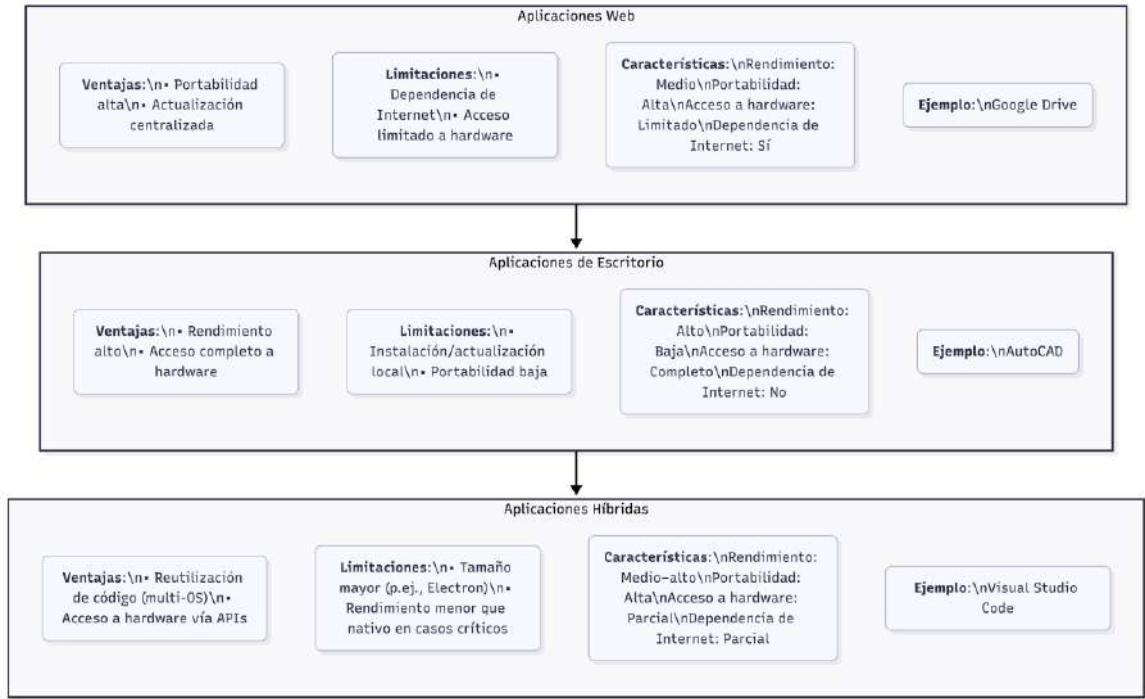


Imagen 2. Diagrama comparativo entre aplicaciones web, escritorio e híbridas (Creación propia del autor, Damián Nicolalde, 2025).

1.3 Arquitectura cliente-servidor moderna

La **arquitectura cliente-servidor** constituye la base de la mayoría de las aplicaciones digitales actuales. Aunque nació hace varias décadas, ha evolucionado hasta convertirse en un modelo altamente flexible que sostiene desde aplicaciones web simples hasta complejas plataformas globales con millones de usuarios simultáneos. Comprender su funcionamiento y sus variantes modernas es crucial, ya que este conocimiento permitirá a los estudiantes contextualizar la implementación de estructuras de datos y algoritmos en sistemas distribuidos.

1.3.1 Principios básicos del modelo cliente-servidor

En esencia, la arquitectura cliente-servidor se basa en la **separación de responsabilidades**:

- El **cliente** (navegador, aplicación móvil o de escritorio) se encarga de la interfaz de usuario y de enviar solicitudes.
- El **servidor** procesa dichas solicitudes, accede a los datos y devuelve respuestas.

Esta separación permite mayor modularidad y escalabilidad, dado que múltiples clientes pueden conectarse a un mismo servidor centralizado.

En su forma más simple, el cliente envía una request (solicitud) a través de un protocolo como HTTP, y el servidor responde con los datos o recursos solicitados. Sin embargo, el modelo moderno introduce capas adicionales como seguridad, balanceo de carga, microservicios y almacenamiento distribuido.

1.3.2 Evolución hacia arquitecturas modernas

El modelo cliente-servidor tradicional estaba limitado a **una conexión directa entre cliente y servidor**, lo que generaba cuellos de botella. Hoy, las arquitecturas modernas incluyen varios componentes intermedios que aumentan la robustez del sistema:

- **Balanceadores de carga**: distribuyen las solicitudes entre múltiples servidores.
- **Servidores de aplicaciones**: encargados de ejecutar la lógica de negocio.
- **Servidores de base de datos distribuidos**: replican y particionan datos para mejorar el rendimiento.
- **Cachés intermedias**: reducen la latencia mediante almacenamiento temporal de datos frecuentes.

Además, se han incorporado modelos de comunicación avanzados:

- **REST (Representational State Transfer)**: basado en operaciones HTTP estándar (GET, POST, PUT, DELETE).
- **GraphQL**: permite a los clientes solicitar solo los datos que necesitan, optimizando ancho de banda.
- **WebSockets**: habilitan comunicación bidireccional en tiempo real (ej. chats, videojuegos online).

1.3.3 Seguridad en el modelo cliente-servidor

La seguridad es un aspecto crítico en esta arquitectura. Algunos mecanismos esenciales son:

1. **HTTPS**: protege la comunicación mediante cifrado TLS/SSL.
2. **Autenticación y autorización**: gestión de identidad mediante protocolos como OAuth 2.0 u OpenID Connect.
3. **Tokens JWT (JSON Web Tokens)**: permiten mantener sesiones seguras sin necesidad de almacenar credenciales en el servidor.
4. **CORS (Cross-Origin Resource Sharing)**: controla qué dominios pueden consumir los recursos del servidor.

1.3.4 Ejemplo práctico de flujo cliente-servidor

Consideremos un sistema de gestión de turnos en un hospital:

1. El cliente (navegador del paciente) envía una solicitud POST al servidor para registrarse en la cola de atención.
2. El servidor valida los datos, los almacena en la base de datos y actualiza la cola.
3. El servidor devuelve una respuesta con el número de turno y tiempo estimado de espera.
4. Otro cliente (navegador del médico) consulta la cola en tiempo real mediante WebSockets para saber qué paciente sigue.

1.3.5 Ventajas y limitaciones

Ventajas:

- Escalabilidad: se pueden añadir más servidores para manejar mayor carga.
- Centralización: facilita la actualización y mantenimiento del sistema.
- Flexibilidad: admite distintos tipos de clientes (web, móvil, escritorio).

Limitaciones:

- Complejidad: las arquitecturas modernas requieren múltiples capas y herramientas.
- Latencia: cada petición puede generar tiempos de espera.
- Dependencia de red: sin conexión estable, la aplicación pierde funcionalidad.

1.3.6 Tabla comparativa de variantes

Tabla 2. Comparación de arquitecturas cliente-servidor modernas

Variante	Características principales	Ejemplo de uso
REST	Operaciones CRUD vía HTTP	APIs públicas (Twitter API)
GraphQL	Peticiones específicas de datos	GitHub API
WebSockets	Comunicación bidireccional en tiempo real	Chats, juegos en línea
Microservicios	División en servicios independientes	Netflix, Amazon

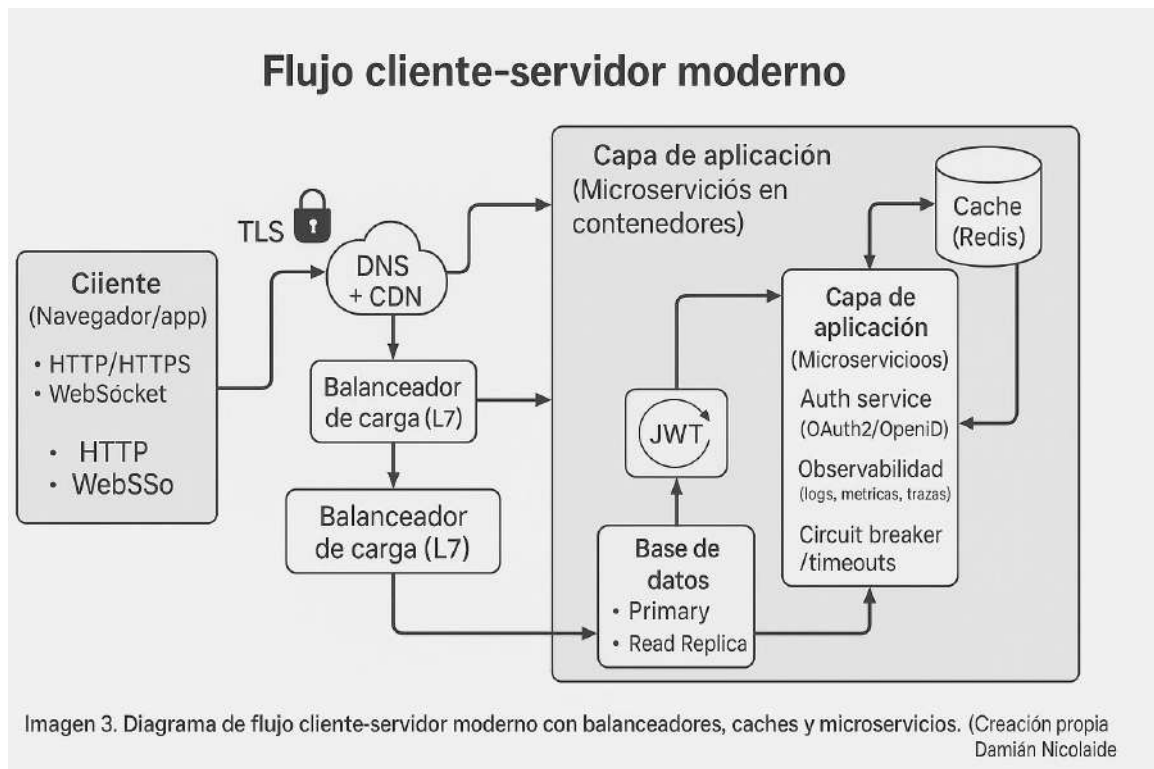


Imagen 3. Diagrama de flujo cliente-servidor moderno con balanceadores, cachés y microservicios. (Creación propia del autor, Damián Nicolalde, 2025).

1.4 Herramientas y entornos de desarrollo actuales

En el ecosistema tecnológico moderno, las **herramientas y entornos de desarrollo** son factores determinantes para la productividad, la calidad del software y la colaboración en equipos multidisciplinarios. Mientras que en décadas anteriores el desarrollo se centraba en un único compilador o editor de texto, hoy en día los profesionales cuentan con un ecosistema integral que combina editores de código inteligentes, control de versiones, entornos virtualizados y sistemas de despliegue continuo.

1.4.1 Entornos de desarrollo integrados (IDEs)

Los **IDEs (Integrated Development Environments)** son plataformas que centralizan las funciones necesarias para programar: edición de código, compilación, depuración y pruebas. En la actualidad, destacan:

- **Visual Studio Code (VS Code):** editor ligero, multiplataforma, con soporte para múltiples lenguajes mediante extensiones. Su integración con Git y Docker lo hace ideal para proyectos colaborativos.
- **IntelliJ IDEA y PyCharm:** especializados en lenguajes como Java y Python, ofrecen análisis estático de código, autocompletado avanzado y refactorización inteligente.
- **Eclipse y NetBeans:** ampliamente usados en entornos académicos y corporativos, sobre todo para proyectos de gran escala en Java.

Estos entornos permiten a los estudiantes trabajar con proyectos modulares, facilitando la comprensión de buenas prácticas y estándares de programación.

1.4.2 Sistemas de control de versiones

El desarrollo moderno es inseparable del uso de **control de versiones**, siendo Git el estándar de facto. Este sistema distribuye el historial de cambios de un proyecto entre todos los colaboradores, permitiendo:

- Registrar la evolución del código de manera segura.
- Colaborar en paralelo, creando ramas (*branches*) para trabajar en nuevas funcionalidades.
- Fusionar y resolver conflictos, asegurando la coherencia del proyecto.

Plataformas como **GitHub, GitLab y Bitbucket** han transformado a Git en un entorno social de desarrollo, al permitir no solo alojar repositorios, sino también automatizar pruebas, coordinar equipos y desplegar aplicaciones directamente en la nube.

Ejemplo práctico: en un proyecto académico, cada estudiante podría crear una rama para implementar una estructura de datos (pila, cola o lista) y luego integrarlas en el repositorio central.

1.4.3 Entornos virtualizados y contenedores

La necesidad de que las aplicaciones funcionen de forma idéntica en distintos entornos llevó al uso de virtualización ligera mediante contenedores.

- **Docker:** empaqueta una aplicación junto con sus dependencias en un contenedor, garantizando que se ejecute de la misma forma en cualquier sistema operativo.
- **Kubernetes:** orquesta miles de contenedores en entornos empresariales, gestionando escalabilidad y balanceo de carga.

En el contexto educativo, los contenedores permiten simular entornos cliente-servidor sin necesidad de hardware costoso.

1.4.4 Integración y despliegue continuo (CI/CD)

Las prácticas de Integración Continua (CI) y Despliegue Continuo (CD) permiten automatizar pruebas, validaciones y entregas de software. Herramientas como Jenkins, Travis CI o GitHub Actions ejecutan flujos de trabajo que incluyen:

1. Compilar el código.
2. Ejecutar pruebas automáticas.
3. Generar reportes de calidad.
4. Desplegar la aplicación en un servidor o nube.

Estas prácticas fomentan la disciplina en el estudiante, pues muestran que la programación no se trata solo de escribir código funcional, sino de garantizar que el sistema sea robusto, probado y mantenible.

1.4.5 Entornos locales vs. entornos en la nube

Otra diferencia clave en el desarrollo moderno es el lugar donde se ejecuta el código:

- **Entorno local:** el estudiante instala IDEs, compiladores y dependencias en su computadora personal. Ventaja: mayor control. Limitación: posibles errores de compatibilidad.
- **Entorno en la nube:** plataformas como **GitHub Codespaces** o **Replit** permiten programar desde un navegador, con entornos preconfigurados. Ventaja: accesibilidad y colaboración inmediata.

La tendencia actual apunta hacia entornos híbridos donde el código puede desarrollarse localmente y sincronizarse con la nube para pruebas y despliegues.

1.4.6 Tabla comparativa de herramientas

Tabla 3. Herramientas modernas de desarrollo

Categoría	Herramienta principal	Ventaja clave
IDEs	VS Code, IntelliJ	Productividad y extensibilidad
Control de versiones	Git/GitHub	Colaboración y registro de cambios
Contenedores	Docker, Kubernetes	Portabilidad y escalabilidad
CI/CD	Jenkins, GitHub Actions	Automatización de pruebas y despliegue
Cloud IDE	Codespaces, Replit	Acceso remoto y configuración rápida

De esta forma, los estudiantes no solo adquieren competencias técnicas, sino también habilidades de gestión de proyectos y trabajo colaborativo.

Ecosistema moderno de herramientas de desarrollo

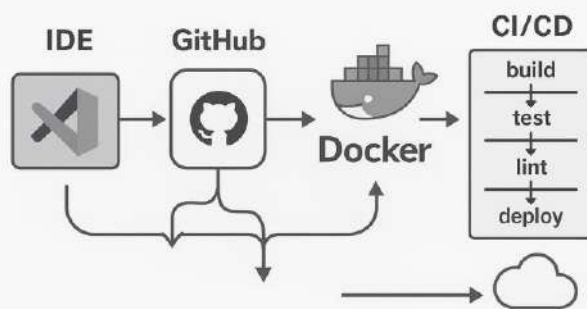


Imagen 4. Ecosistema moderno de herramientas de desarrollo (IDE, Github, Docker y CI/CD).

Imagen 4. Ecosistema moderno de herramientas de desarrollo (IDE, Github, Docker y CI/CD).
(Creación propia del autor, *Damián Nicolalde*, 2025).

1.5 HTML5 semántico y accesibilidad web

El **HTML5** no solo es la última evolución del lenguaje de marcado fundamental de la web, sino también un estándar que promueve la **semántica** y la **accesibilidad** como principios esenciales del desarrollo moderno. A diferencia de versiones anteriores, HTML5 introduce etiquetas que otorgan significado al contenido, permitiendo que tanto los navegadores como las tecnologías de asistencia (lectores de pantalla, teclados adaptados, software de navegación por voz) interpreten adecuadamente la estructura de un sitio. Este enfoque no solo mejora la experiencia del usuario, sino que también fortalece el posicionamiento en buscadores (SEO) y garantiza el cumplimiento de normativas internacionales como las **WCAG (Web Content Accessibility Guidelines)**.

1.5.1 La importancia de la semántica

La semántica en HTML consiste en utilizar etiquetas que describan el propósito del contenido en lugar de limitarse a su apariencia visual. Por ejemplo, antes se utilizaba `<div>` para todo tipo de secciones; ahora HTML5 introduce etiquetas como:

- `<header>`: cabecera de un sitio o sección.
- `<nav>`: menú de navegación.

- <section>: agrupación de contenido temáticamente relacionado.
- <article>: contenido independiente, como un post de blog.
- <aside>: información complementaria.
- <footer>: pie de página.

Este enfoque estructurado facilita que los motores de búsqueda y las herramientas de accesibilidad comprendan la jerarquía del documento, lo que repercute directamente en la usabilidad.

Ejemplo de código semántico:

```
<header>
<h1>Sistema de Gestión de Procesos</h1>
<nav>
<ul>
<li><a href="#inicio">Inicio</a></li>
<li><a href="#modulos">Módulos</a></li>
<li><a href="#contacto">Contacto</a></li>
</ul>
</nav>
</header>
<section>
<article>
<h2>Módulo de Turnos</h2>
<p>Este módulo gestiona las colas de atención en tiempo real.</p>
</article>
</section>
<footer>
<p>&copy; 2025 Universidad</p>
</footer>
```

Este código refleja una organización clara, comprensible tanto para usuarios como para sistemas de procesamiento automático.

1.5.2 Accesibilidad web

La accesibilidad busca garantizar que cualquier persona, independientemente de sus capacidades físicas, cognitivas o tecnológicas, pueda interactuar con una aplicación. Para ello, se siguen las **WCAG (Pautas de Accesibilidad para el Contenido Web)**, organizadas en tres niveles:

- **A:** accesibilidad básica (mínimo exigido).
- **AA:** nivel intermedio, recomendado en la mayoría de los proyectos.
- **AAA:** nivel avanzado, dirigido a aplicaciones críticas o gubernamentales.

Ejemplos de buenas prácticas en accesibilidad incluyen:

1. **Uso de etiquetas <label> en formularios**, vinculadas a sus campos mediante el atributo for.
2. **Texto alternativo (alt) en imágenes**, para que lectores de pantalla describan el contenido.

- 3. **Contraste adecuado** entre texto y fondo, garantizando legibilidad.
- 4. **Navegación mediante teclado**, permitiendo acceder a todos los elementos sin ratón.

1.5.3 Atributos ARIA (Accessible Rich Internet Applications)

HTML5 se complementa con atributos ARIA, que proporcionan descripciones adicionales para tecnologías de asistencia. Algunos ejemplos:

- `role="navigation"`: indica que un bloque corresponde a la navegación.
- `aria-label="Cerrar ventana"`: etiqueta descriptiva para un botón.
- `aria-live="polite"`: informa que un contenido se actualizará dinámicamente.

Estos atributos resultan especialmente útiles en aplicaciones modernas con contenido dinámico, como tableros en tiempo real o chats.

1.5.4 Herramientas de validación y auditoría

Para garantizar accesibilidad y semántica, se utilizan herramientas como:

- **Lighthouse (Google Chrome)**: evalúa rendimiento, SEO y accesibilidad.
- **Wave Accessibility Tool**: identifica errores de etiquetas, contrastes y jerarquías.
- **Axe DevTools**: extensión para desarrolladores que automatiza pruebas de accesibilidad.

El estudiante debe familiarizarse con estas herramientas, ya que serán necesarias para validar los sistemas implementados en los retos, asegurando que sus proyectos no solo funcionen, sino que sean inclusivos.

Tabla comparativa

Tabla 4. Beneficios del HTML5 semántico y accesibilidad

Aspecto	Beneficio principal	Ejemplo práctico
Semántica	Mejor organización y SEO	<code><article></code> en un blog académico
Accesibilidad WCAG	Inclusión de usuarios con discapacidad	Formularios con <code><label></code> claros
Atributos ARIA	Mejor interpretación por lectores de pantalla	<code>aria-label</code> en botones dinámicos

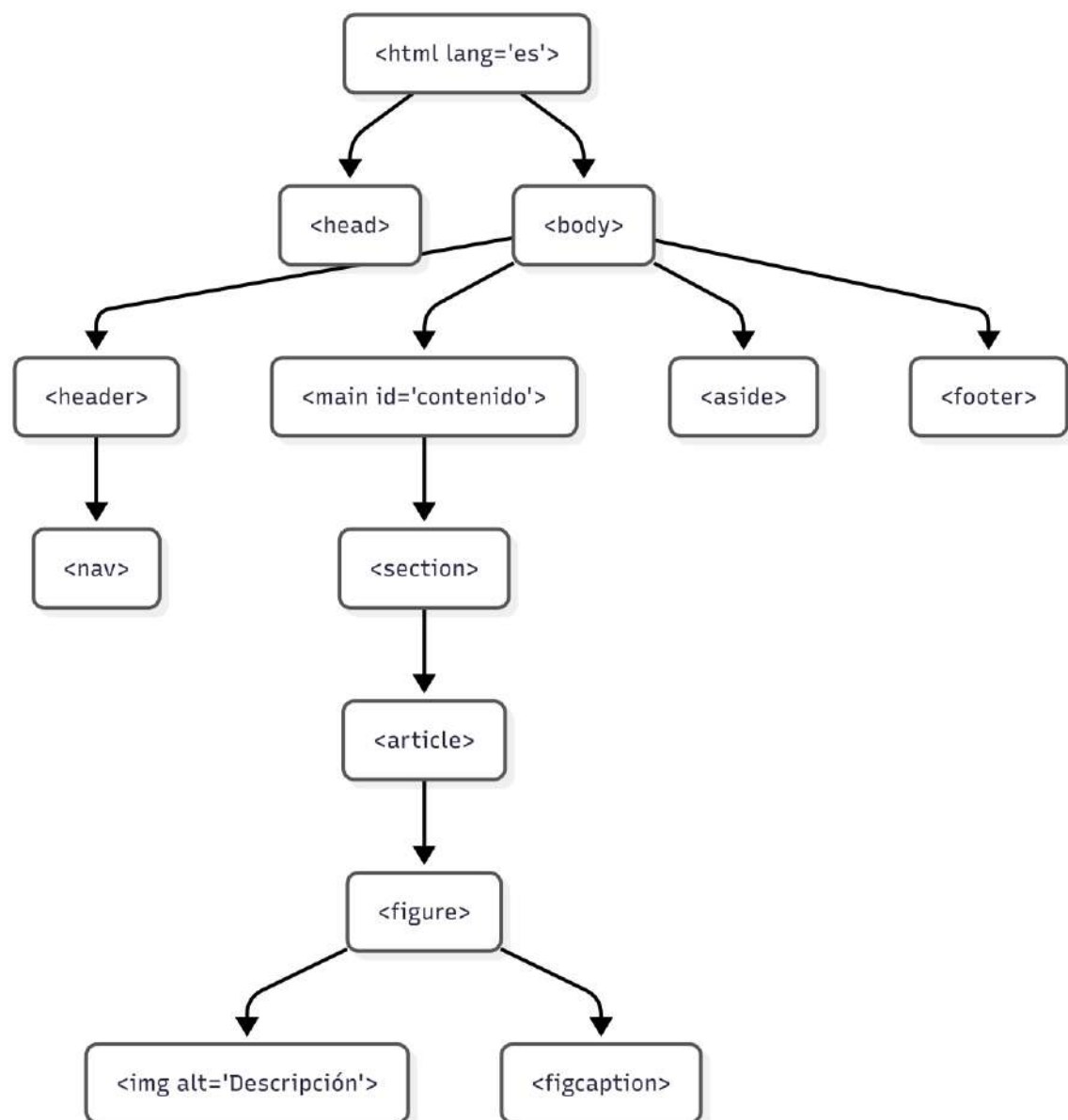


Imagen 5. Ejemplo visual de un documento HTML5 estructurado con etiquetas semánticas.
(Creación propia del autor, Damián Nicolalde, 2025).

Referencias citadas en la Clase 1.

- Fielding, R. T., & Taylor, R. N. (2000). *Principled design of the modern web architecture*. ACM Transactions on Internet Technology, 2(2), 115–150. <https://doi.org/10.1145/514183.514185>
- W3C. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. World Wide Web Consortium. <https://www.w3.org/TR/WCAG21/>
- Mozilla Developer Network (MDN). (2025). *HTML5 semantic elements*. Recuperado de <https://developer.mozilla.org/>
- Google Developers. (2025). *Lighthouse*. Recuperado de <https://developers.google.com/web/tools/lighthouse>

Enlaces externos

Título del enlace relacionado: Pautas de Accesibilidad para el Contenido Web (WCAG) 2.2 – W3C

Descripción del enlace relacionado: Recomendaciones normativas y criterios de éxito para construir contenido web accesible. Incluye los cambios respecto de WCAG 2.1 y materiales de apoyo (“Understanding WCAG”).

Enlace: <https://www.w3.org/TR/WCAG22/> W3C

Título del enlace relacionado: HTML semántico – MDN Web Docs

Descripción del enlace relacionado: Guía práctica para estructurar documentos con elementos semánticos (<main>, <section>, <article>, <header>, <nav>, <footer>), con ejemplos y buenas prácticas que mejoran accesibilidad y SEO.

Enlace: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content



Desarrollo en HTML, CSS y Javascript de
Apps Web, Android, IOS, FirefoxOS, ...



Objetivo del curso

Curso de **diseño y programación de aplicaciones Web** multi-dispositivo

(PC, tableta, teléfono, ...)

para la plataforma Web

(**HTML5, CSS3 y JavaScript5**)

y de como empaquetarlas

para su publicación en las **tiendas**

de Android, iOS,



Equipos y servicios a utilizar

◆ Un PC o portatil de trabajo (necesario)

- con S.O. Windows de Microsoft
- con S.O. UNIX (Ubuntu, ..) de software libre
- Ordenador MAC con sistema operativo con OS X

◆ Móvil o tableta (conveniente para probar, pero no necesario)

- Android, iPhone, iPad, FirefoxOS, .. pero no es necesario
 - ◆ Se puede utilizar un simulador de FirefoxOS para probar Apps
 - ◆ Solicitar dispositivos móviles en el LTIC

◆ Cuenta en Neocities (es gratis)

- para publicar Páginas y Apps Web en la nube
 - ◆ <https://neocities.org/> (crear una cuenta gratuita para publicar las páginas web)

Herramientas a utilizar

◆ Simulador/Editor interactivo

- Editor y visualizador de HTML5, CSS y JS con ejemplos cargados
 - ◆ <http://vishub.org/excursions/2209.full> (aquí están los ejemplos del curso y además se puede usar la plataforma para trabajar de manera online)

◆ Sublime Text 2, Visual Studio Code

- Editor de HTML, CCS y JavaScript con ayudas visuales
 - ◆ <http://www.sublimetext.com>

◆ Creadores (wysiwyg) de páginas Web (opcional)

- Dreamweaver: Creador de páginas Web muy potente e intuitivo.
 - ◆ <http://www.adobe.com/es/products/dreamweaver.html>
- Bluegriffon: editor wysiwyg gratuito y de código abierto
 - ◆ <http://www.bluegriffon.org>

◆ Navegador (Chrome, Firefox, ...) y su entorno de desarrollo

- <https://developer.chrome.com/devtools>
- <https://www.mozilla.org/es-ES/firefox/new/>

Simulador en ViSH

Guardar modificaciones en memoria local del navegador

Descargar ejemplo con modificaciones a un fichero

Subir código de un fichero al area de edición

Inicializar

Visualizar modificaciones

Seleccionar módulo

Seleccionar tema

Seleccionar ejemplo

Area de edición del código fuente de los ejemplo

Area de visualización de la aplicación

Ejemplo: Primera pagina con imagen

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Párrafo con imágenes</title>
6 </head>
7
8 <body>
9 <h1>Titular nivel 1 </h1>
10 <p>Párrafo de texto. <strong>negrita</strong> y <em>cursiva</em>
11 <ul>
12 <li>Elemento de la lista 1</li>
13 <li>Elemento de la lista 2</li>
14 <li>Elemento de la lista 3</li>
15 </ul>
16 
17 </body>
18 </html>
19
```

Resultado:

Titular nivel 1

Párrafo de texto, **negrita** y *cursiva*

- Elemento de la lista 1
- Elemento de la lista 2
- Elemento de la lista 3

<http://vishub.org/excursions/2209.full>



Final del tema
Muchas gracias!



Introducción a Internet y a la Web

Cientes, servidores y la nube



◆ Clientes:

- Dan acceso a información y servicios en Internet

◆ Servidores:

- Alojan la información y los servicios

◆ La nube: conjunto de terminales y servidores

- interconectados con aplicaciones y protocolos de Internet

◆ TCP/IP: protocolos de interconexión de redes de fibra, cable, WIFI, ...

- sobre los que se implementan las aplicaciones de Internet y sus protocolos

Cientes, navegadores y tiendas



◆ **Cientes** de acceso a Internet más importantes

- PCs, portátiles, tabletas, teléfonos inteligentes

◆ **Navegador (browser)** cliente Web de acceso a servidores

- Utilizando: **URL, HTTP, HTML, CSS y JS**
 - ◆ p.e. Chrome, Firefox, Internet Explorer, Opera, Safari, ...



◆ **Tiendas de aplicaciones**

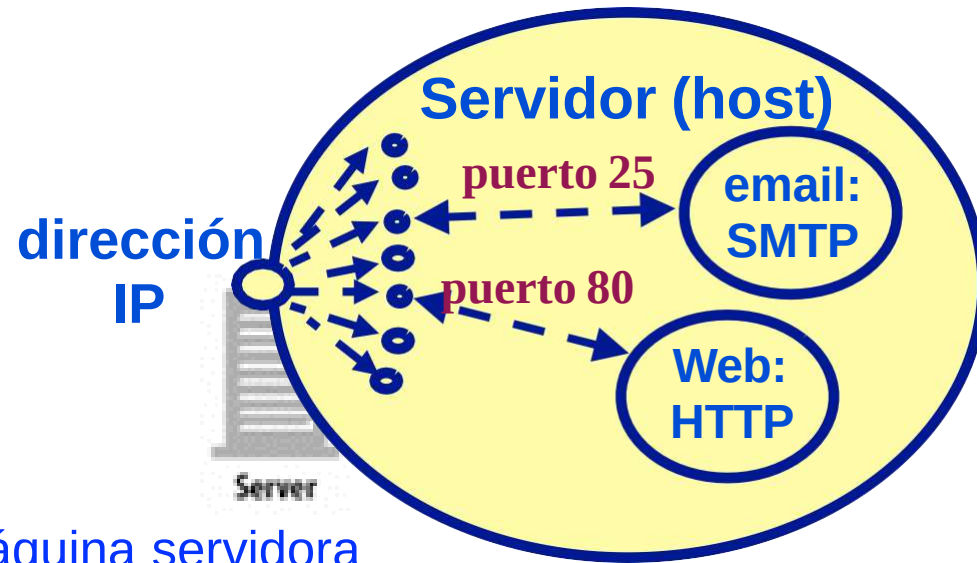
- Instalan aplicaciones en móviles y tabletas
 - ◆ Las aplicaciones usan las normas de la Web (URL, HTTP,)

Máquina servidora (host)



- ◆ Contiene información y servicios
- ◆ Una máquina servidora tiene una dirección “**conocida**” en Internet
 - Dirección simbólica (de dominio o **DNS**): **puce.edu.ec**, **google.com**, ...
 - ◆ Cada dirección de dominio tiene una dirección IP (binaria) asociada
- ◆ Hay 2 tipos de direcciones IP: IPv4 e IPv6
 - **IPv4**: versión 4 del protocolo IP con dirección de **32 bits** o **4 octetos**
 - ◆ ejemplo: **192.9.0.144**, **127.0.0.1** (localhost - mi máquina), ...
 - **IPv6**: versión 6 del protocolo IP (última) con dirección de **128 bits**
 - ◆ ejemplo: **2001:db8:85a3::8a2e:370:7334**,

Servidores y puertos



◆ Puerto

- Dirección de 16 bits dentro de la máquina servidora
 - ◆ Es donde se instala el **programa servidor**
- El **programa servidor** es lo que normalmente denominamos **servidor**
 - ◆ Cliente y servidor se comunican a través de un protocolo: HTTP, SMTP,
 - Utilizando el interfaz de sockets TCP/IP para comunicar entre ambos

◆ Los servicios tienen un protocolo y un puerto por defecto

- **Web:** protocolo HTTP (puerto 80), HTTPS (443)
- **Email:** protocolo SMTP (puerto 25), POP3 (110), IMAP143)
- **Shell segura:** protocolo SSH (puerto 22)

◆ Si un servidor no está en el puerto por defecto

- Su dirección debe incluir el puerto, p.e. puceing.edu.ec:8080,
192.9.0.144:8080

URL



◆ URL (Uniform Resource Locator)

- **Dirección de un recurso en un servidor en Internet**
- ◆ Internet soporta muchos tipos de servicios diferentes
 - Cada tipo de servicio utiliza un URL y protocolo diferentes
- ◆ Algunos ejemplos de tipos de URLs
 - **URL Web:** utiliza HTTP para acceder a recursos, incluye
 - ◆ Protocolo, servidor y recurso (camino): <http://google.com/picture.png>
 - **URL de correo (email):** identifica el buzón de usuario, incluye
 - ◆ protocolo, buzón de usuario y servidor: mailto:pepe_garcia@gmail.com

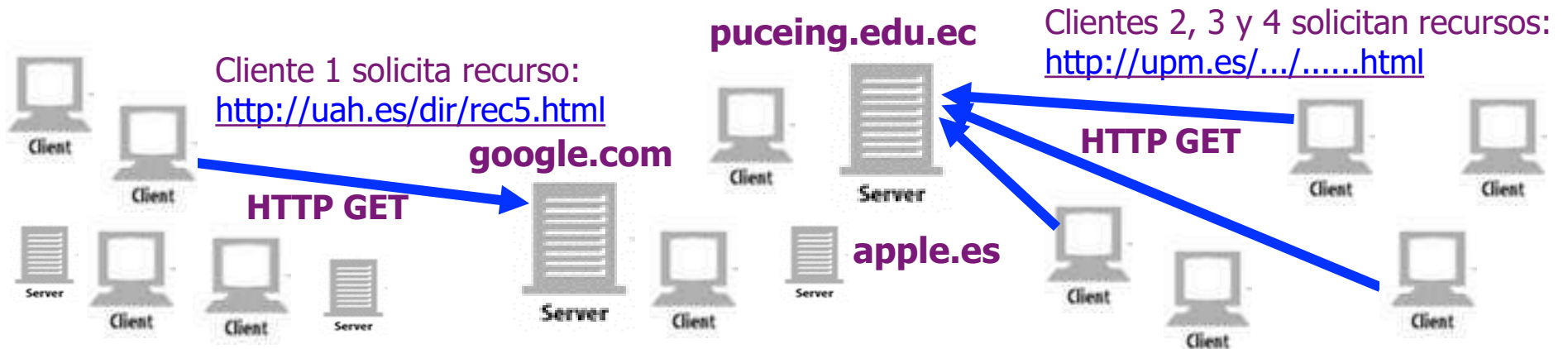
HTTP (HiperText Transfer Protocol)

◆ Protocolo del Web

- Procesa recursos identificados por un URL en un servidor remoto

◆ Métodos o comandos principales de HTTP

- **GET:** trae al cliente (lee) un recurso identificado por un **URL**
- **POST:** crea un recurso identificado por un URL
- **PUT:** actualiza un recurso identificado por un URL
- **DELETE:** borra un recurso identificado por un URL
- (hay mas comandos)



Aplicación Web

- ◆ Aplicaciones ejecutables en un navegador creadas con
 - **HTML, CSS y JavaScript**



- ◆ **HTML**

- Lenguaje de marcado de páginas Web
 - ◆ define la estructura del contenido de una página Web
- En WebApps define la interfaz de la aplicación con el usuario

- ◆ **CSS**

- Define el estilo visual de una página o aplicación Web (HTML)

- ◆ **JavaScript**

- Lenguaje de programación de aplicaciones de cliente

Aplicación Web: HTML, CSS y JavaScript

◆ HTML

- Lenguaje de marcado

◆ CSS

- Estilo la visualización

◆ JavaScript

- Lenguaje de programación

```
<!DOCTYPE html><html>
<head>
  <title>Ejemplo</title>
  <meta charset="UTF-8">

  <style type="text/css">
    body {color: blue;}
  </style>
</head>

<body>
  <h3>Fecha y hora</h3>

  <script type="text/javascript">
    document.write(new Date());
  </script>
</body>
</html>
```



WebApps o aplicaciones de cliente

◆ Aplicaciones que **residen en un servidor**

- pero se **ejecutan en un cliente**

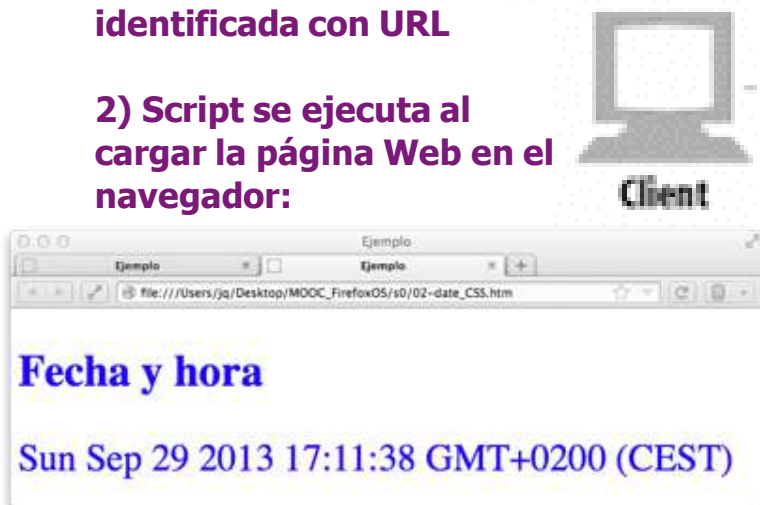
- ◆ Se identifican con un **URL**: <http://puceing.edu.ec/apps/webapp.html>
 - El cliente trae la aplicación del servidor con el **protocolo HTTP (GET)**

◆ Las apps se construyen con las tecnologías de la Web

- URLs, HTTP, HTML, CSS y JavaScript

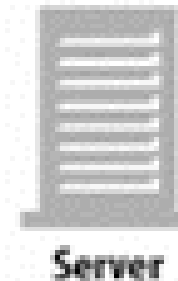
1) Cliente solicita WebApp identificada con URL

2) Script se ejecuta al cargar la página Web en el navegador:



Solicitud HTTP GET asociada a un URL

Respuesta HTTP: página Web con script



Servidor sirve fichero identificado por URL



HTML/CSS

Mi primera página

Qué es HTML

- Es un **lenguaje de marcas** para formatear y estructurar un documento, que puede leerse en cualquier navegador.

<html> </html>

- Estandarizado en la norma ISO de **SGML** (Standard Generalized Markup Language).
- El **W3C** desarrolla especificaciones técnicas y directrices, de forma que se pueda asegurar una alta calidad técnica y editorial.

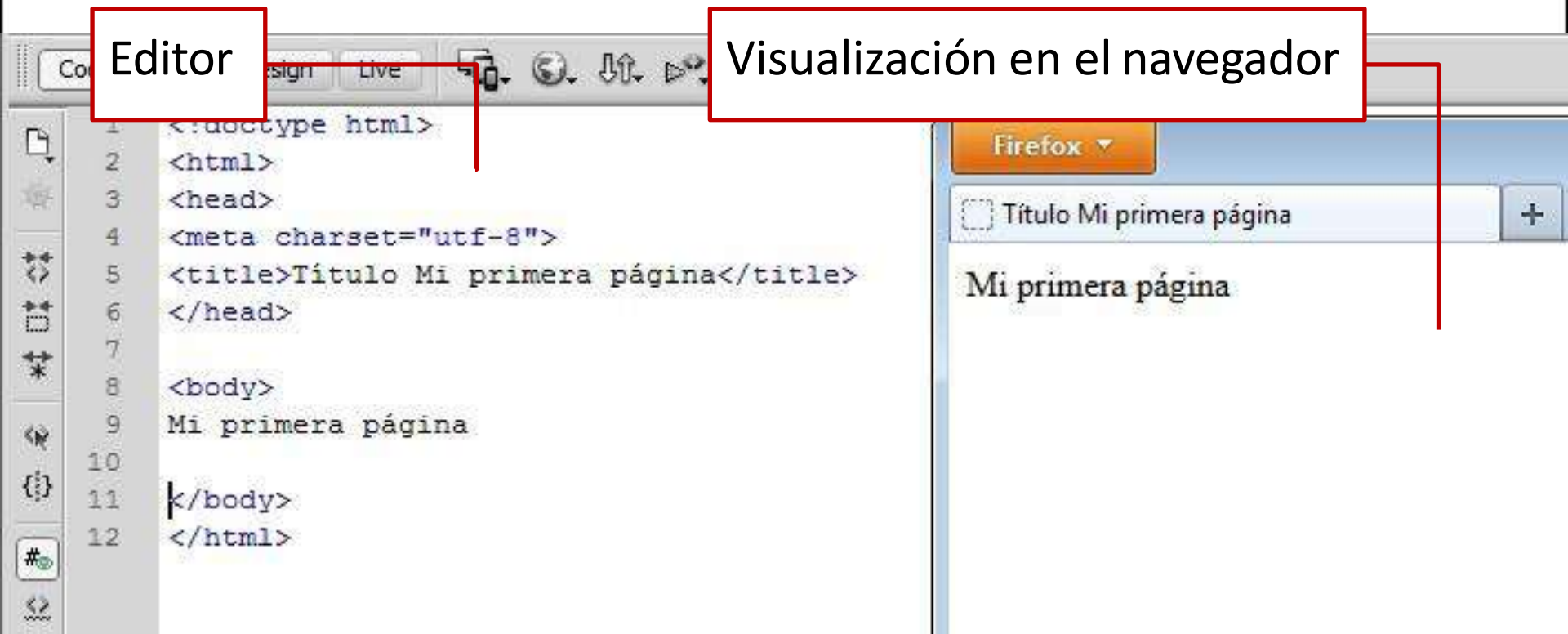
Estructura HTML

Un documento HTML tiene tres etiquetas que describen la estructura general de un documento y dan una información sencilla sobre él. `<html>`, `<head>` y `<body>`

Las etiquetas pueden escribirse tanto en mayúsculas como en minúsculas, pero se recomienda el uso de **minúsculas**: `<html>` o `<HTML>`, `<body>` o `<BODY>`

Editor

Visualización en el navegador



Lenguaje HTML

Las marcas de texto en html, se emplean para el **estructurado semántico del contenido**.

Los **textos** habitualmente están formados básicamente por títulos `<h1></h1>` párrafos `<p></p>` , resaltando en ellos algunas palabras en negrita `<strong></strong>` o en cursiva `<em></em>` .

Las **imágenes** se vinculan en una página HTML con la etiqueta **img**
``

con los atributos:

src = URL, **width** = ancho, **height** = alto, **alt** = texto alternativo de la imagen.

Las imágenes que se usan en la web son: GIF, JPG, PNG y las imágenes vectoriales en SVG

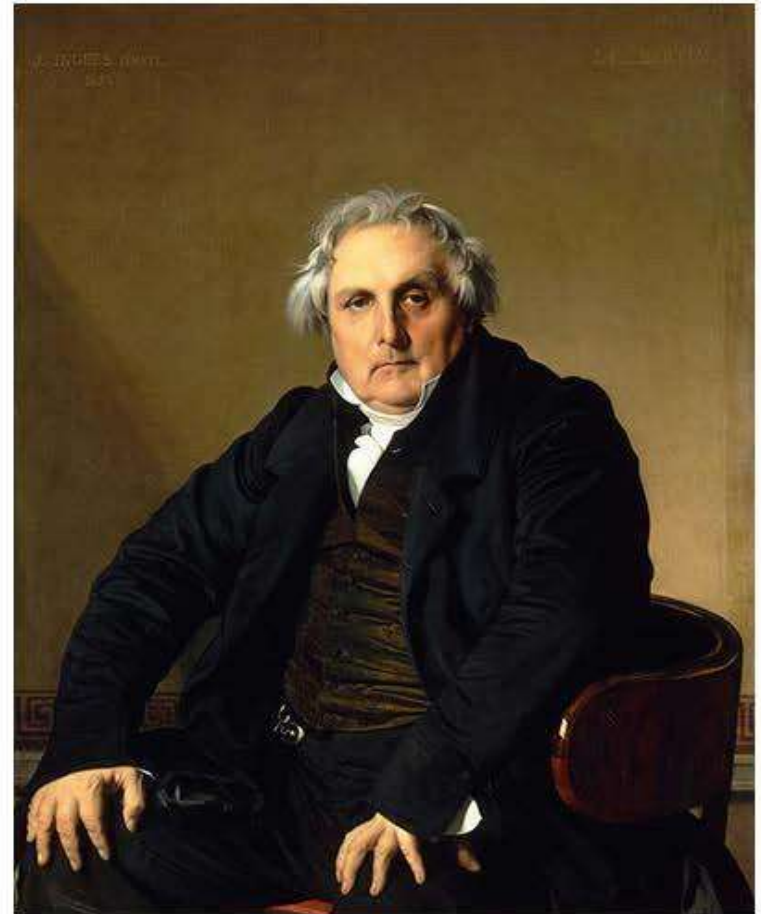
Párrafo e Imágenes HTML

Ejemplo

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Párrafo con imágenes</title>
6 </head>
7
8 <body>
9 <h1>Titular nivel 1 </h1>
10 <p>Párrafo de texto. <strong>negrita</strong> y <em>cursiva</em></p>
11 
13 </body>
14 </html>
```

Titular nivel 1

Párrafo de texto. **negrita** y *cursiva*



URL

URL (Uniform Resource Locator) es el localizador de un archivo en internet.

Una dirección absoluta en internet, que combina el nombre del servidor que proporciona la información, el directorio donde se encuentra, el nombre del fichero:

esquema: [//www.miDominio.com/ruta/miArchivo.html](http://www.miDominio.com/ruta/miArchivo.html)

un esquema define el tipo de servicio de internet:

http, es el protocolo usado para la transacción en la Web

`<a href="http://www.nytimes.com/"> The New York Times</a>`

https es la versión segura de http

`<a href="https://www.google.es/"> Google España</a>`

mailto, es el esquema que se emplea para enlazar a una dirección de correo.

`<a href="mailto:nombre@miDominio.com"> Contacto</a>`

ftp, es el esquema para la transferencia archivos.

`<a href="ftp://ftp.miDominio.com/ruta/miArchivo.zip">`

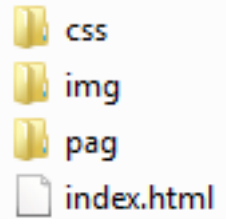
`Descarga por FTP</a>`

URL

Una URL **local** nos lleva a otra página o archivo del mismo sitio de Internet.

Ejemplo de un sitio web, de la raíz parten todas las subcarpetas.

css, img, pag



Cuando el archivos a enlazar está en la misma carpeta: la ruta es directamente el nombre del archivo de destino.

```
<a href="pag2.html">Ver siguiente</a>
```

El archivo de destino está en una carpeta de nivel inferior a su origen, en este caso deberemos escribir la ruta a partir de la carpeta donde está el archivo de origen:

```
<link href="css/estilo.css" rel="stylesheet" type="text/css"/>
```

```
<a href="pag/pag5.html">Ver ejemplo 5</a>
```

Si el archivo está en una carpeta de nivel superior, es necesario subir uno o varios niveles en la jerarquía de directorios, hasta llegar al nivel donde está el archivo. Subimos un nivel escribiendo (../); si subimos dos niveles (../..), hasta llegar al nivel donde esté el archivo.

```
<a href="../index.html">Página principal</a>
```

Para ir a una carpeta del mismo nivel, debemos subir al directorio raíz mediante ../ y desde ahí a la carpeta buscada.

```

```

e/s/d/
madrid

CSS

El **HTML**, controla el aspecto gráfico del documento mediante hojas de estilo CSS (*cascading style sheets*), el **CSS**, indica al navegador como se deben visualizar los elementos de un documento HTML.

Así se consigue separar el aspecto del contenido.

El estilo CSS, se puede añadir mediante un bloque marcado como **<style>** en la **cabecera (<head>)** y aplica el estilo a los elementos de la página.

Para aplicar un estilo de presentación, el **selector** de elemento puede ser una etiqueta HTML:

p, h1, ul, li

El estilo afecta a todo el contenido incluido dentro de esa etiqueta **<p> </p>**, **<h1></h1>**, ****, ****.

CSS

El estilo se define entre llaves "{" y "}"

H1	{	color	:	green	;	...	}
Selector	propiedad		valor				
	declaración				declaración; ...		
	bloque de declaración						
norma de estilo							

CodeSplitDesignLive

Icons

Title: Marcas de texto con CSS

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Marcas de texto con CSS</title>
6 <style type="text/css">
7 body { font-family: Arial, Helvetica, sans-serif; }
8 h1 { font-size: 1.4em; background-color: #CCC; }
9 p { background-color: #CCC; }
10 em { color: orange; background-color: #E8E8E8; }
11 strong { color: red; background-color: #E8E8E8; }
12 </style>
13 </head>
14 <body>
15 <h1>Titular nivel 1</h1>
16 <p>En este párrafo hay rexta marcado
17 como <em>importante</em> y otro texto
18 marcado como <strong>muy importante.</strong></p>
19 <p>Este es el segundo párrafo.</p>
20 </body>
21 </html>
```

Firefox

Marcas de texto con CSS

Titular nivel 1

En este párrafo hay rexta marcado como **importante** y otro texto marcado como **muy importante.**

Este es el segundo párrafo.

Editores HTML

Antes de comenzar a trabajar con un editor específico, es recomendable conocer el código.

Podemos usar programas que trabajen con **texto plano**, sin añadir sus propias marcas de edición.

Los documentos HTML deben tener la **extensión** html o htm.

Usar un editor **wysiwyg** como el Adobe *Dreamweaver* o el *BlueGriffon* (<http://www.bluegriffon.org/>) o un editor con ayudas visuales como el *sublime* (<http://www.sublimetext.com/>) o el *Brackets* (<http://brackets.io/>) o el visual Studio Code nos facilitará las cosas.

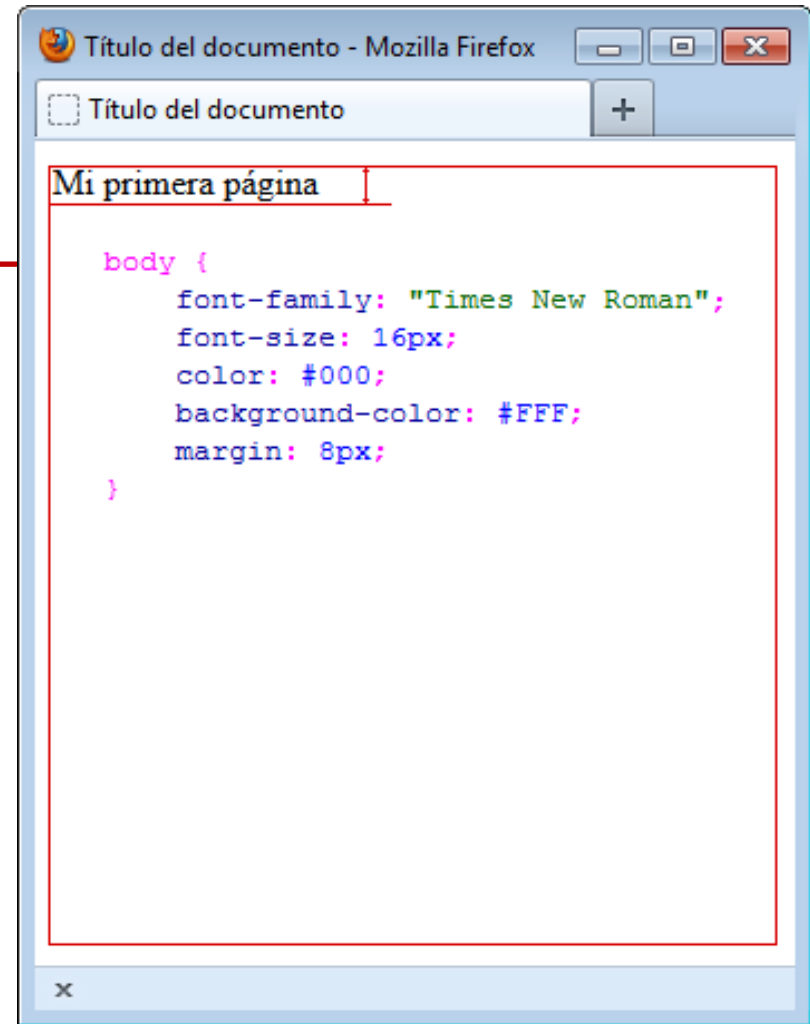
Ver comparativa: http://en.wikipedia.org/wiki/Comparison_of_HTML_editors

CSS

Cada etiqueta HTML tiene unos valores de CSS por defecto, que pueden variar según el navegador, que tendremos que modificar con nuestros estilos personales.

Visualización valores por defecto CSS en el navegador

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Título del documento</title>
6 </head>
7
8 <body>
9 Mi primera página
10 </body>
11 </html>
12
```



HTML

Ejemplo valores CSS por defecto del navegador, de: **h1**, **p**, **strong**, **em**.

```
Code Split Design Live
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Marcas de texto</title>
6 </head>
7
8 <body>
9 <h1>Titular nivel 1</h1>
10 <p>En este párrafo hay rexta marcado
11 como <em>importante</em> y otro texto
12 marcado como <strong>muy importante.</strong></p>
13 <p>Este es el segundo párrafo.</p>
14 </body>
15 </html>
16
```

```
h1 {
  display: block;
  font-size:
  2em;
  margin-before:
  0.67em; margin-after:
  0.67em; margin-start:
  0;
  margin-end: 0;
  font-weight: bold;
```

```
p {
  display: block;
  margin-before:
  1em; margin-after:
  1em; margin-start:
  0;
  margin-end: 0; }
```

```
strong {
  font-weight: bold; }
```

```
em {
  font-style: italic; }
```

Titular nivel 1

En este párrafo hay rexta marcado como *importante* y otro texto marcado como **muy importante**.

Este es el segundo párrafo.

CSS

Más información valores por defecto:

W3C:

<http://dev.w3.org/html5/markup/elements.html>

Internet Explorer:

<http://www.iecss.com/>

Firefox:

<http://mxr.mozilla.org/mozilla-central/source/layout/style/html.css>

Chrome, Safari:

<http://trac.webkit.org/browser/trunk/Source/WebCore/css/html.css>

Opera

<http://www.iecss.com/opera-10.51.css>

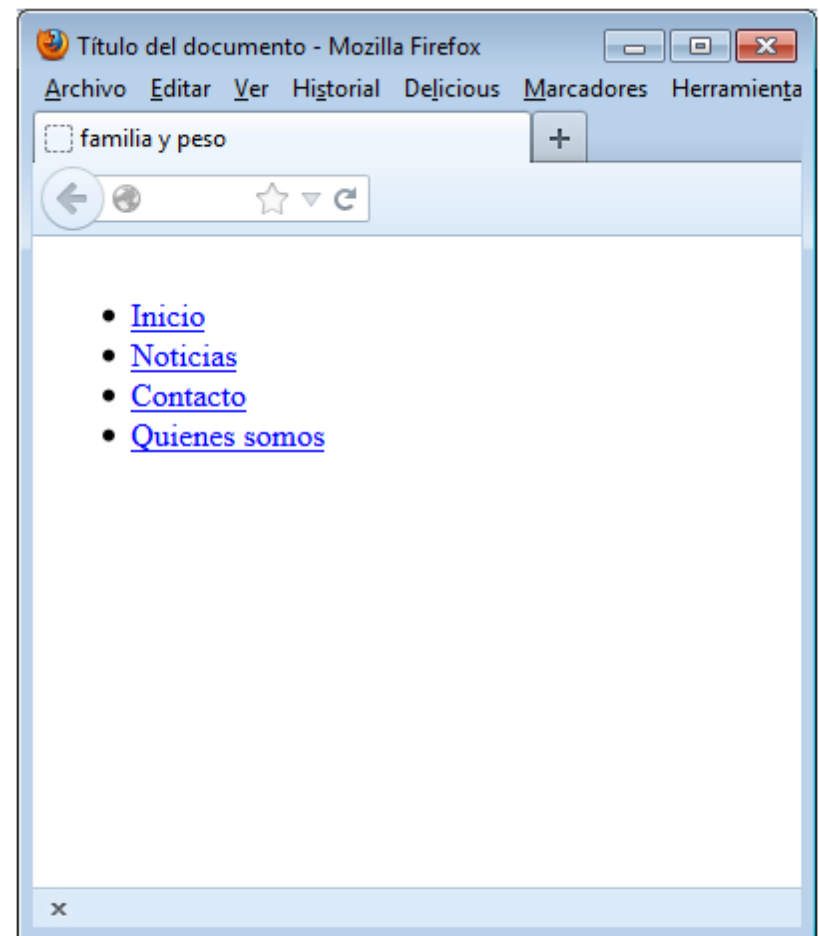
Navegación HTML

La etiqueta `<a>` se emplea para definir un enlace a otra página. Un **menú** es una lista de enlaces. Se recomienda usar las marcas de lista `<ul>` `<li>` y el enlace `<a>`.

```
<!doctype html>
<html>
<head>
<meta charset="utf-8">
<title>Menú</title>
</head>

<body>
<nav>
<ul>
<li><a href="index.html">Inicio</a></li>
<li><a href="news.html">Noticias</a></li>
<li><a href="contacto.html">Contacto</a></li>
<li><a href="quienes.html">Quienes somos</a></li>
</ul>
</nav>

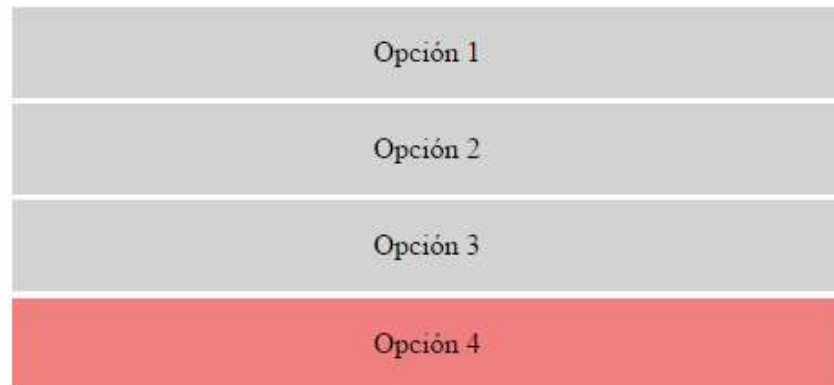
</body>
</html>
```



Navegación HTML

Un menú vertical y horizontal con FLEX, tendría este formato.

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>MENÚ HORIZONTAL FLEX</title>
6 <style type="text/css">
7 ul {
8     list-style-type:none;
9     margin:0;
10    padding:0;
11    display:flex;
12    flex-direction: row; flex-direction: column;
13 }
14 li { flex-grow: 1; }
15 a {
16     display:block;
17     padding:1em;
18     background-color:LightGray ;
19     text-align:center;
20     margin-right:0.2em; margin-bottom:0.2em;
21     text-decoration:none;
22     color:#000000;
23 }
24 a:hover {background-color:LightCoral; }
25 </style>
26 </head>
27 <body>
28 <nav>
29     <ul>
30         <li><a href="#">Opción 1</a></li>
31         <li><a href="#">Opción 2</a></li>
32         <li><a href="#">Opción 3</a></li>
33         <li><a href="#">Opción 4</a></li>
34     </ul>
35 </nav>
36 </body>
```



Recursos CSS

MDN Usando las cajas flexibles CSS

https://developer.mozilla.org/es/docs/Web/Guide/CSS/Cajas_flexibles

CSS-TRICKS A Complete Guide to Flexbox

<http://css-tricks.com/snippets/css/a-guide-to-flexbox/>

HTML/CSS

Mi primera página

HTML/CSS

Estructura HTML

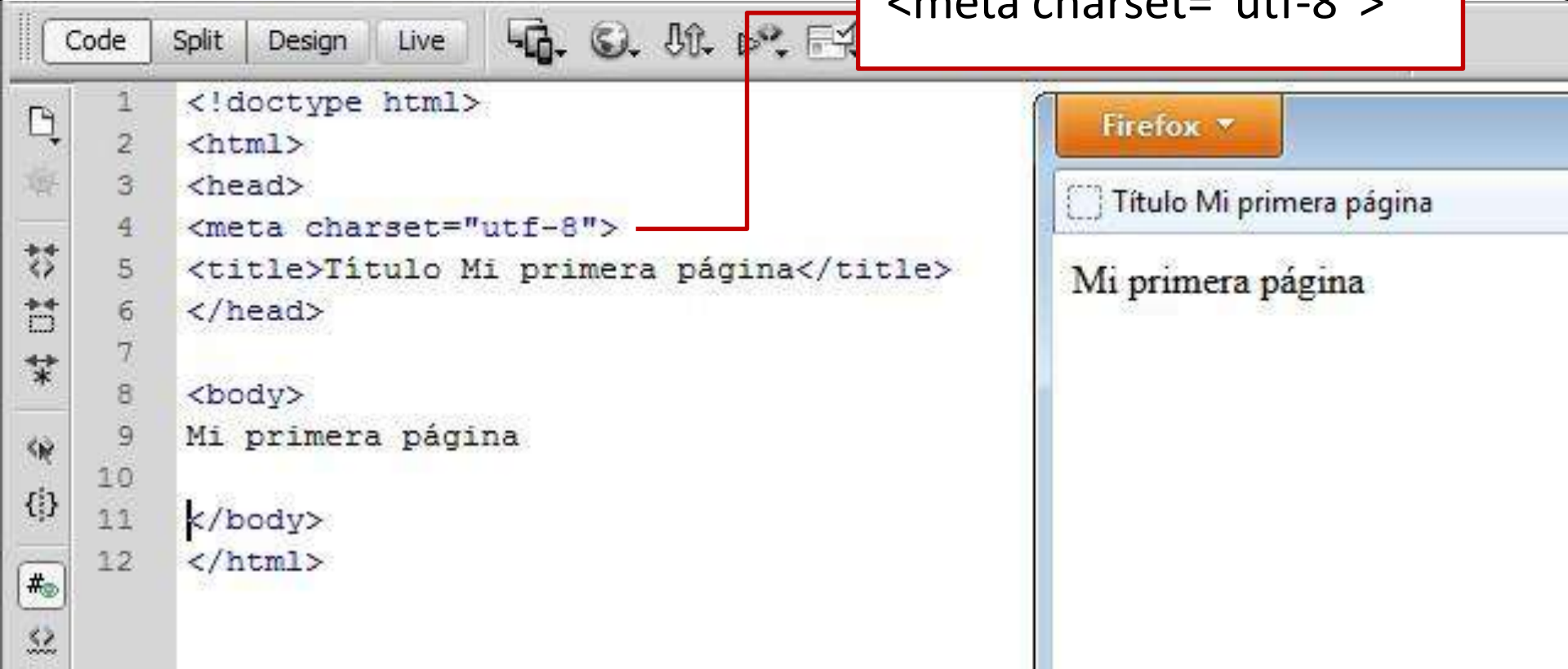
Estructura HTML

La etiqueta **meta** define varios tipos de metadatos.

El atributo **charset** especifica la codificación usada en nuestra página.

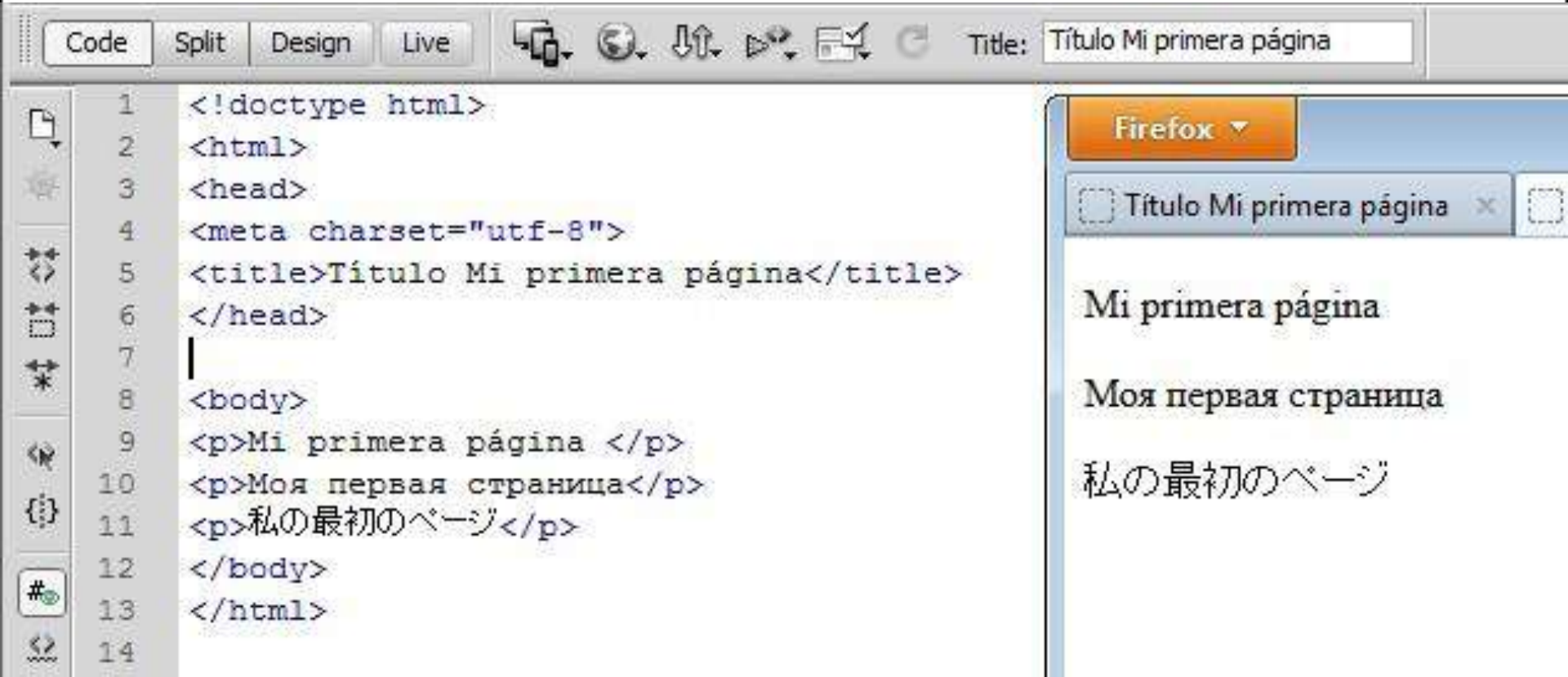
```
<meta charset="character_set">
```

```
<meta charset="utf-8">
```



Estructura HTML

Charset utf-8 es la **Unicode Transformation Format 8-bit** representa el código de caracteres UNICODE. Es compatible con ASCII. Permite visualizar los caracteres de todos los idioma.



Estructura HTML

Caracteres especiales: para incluir en el texto caracteres que son propias del lenguaje HTML es necesario usar su anotación codificada:

<http://dev.w3.org/html5/html-author/charref>

	Espacio en blanco (non-breaking space)	
<	Menor que	<
>	Mayor que	>
&	Ampersand	&
"	Comillas	"
'	apóstrofo	'

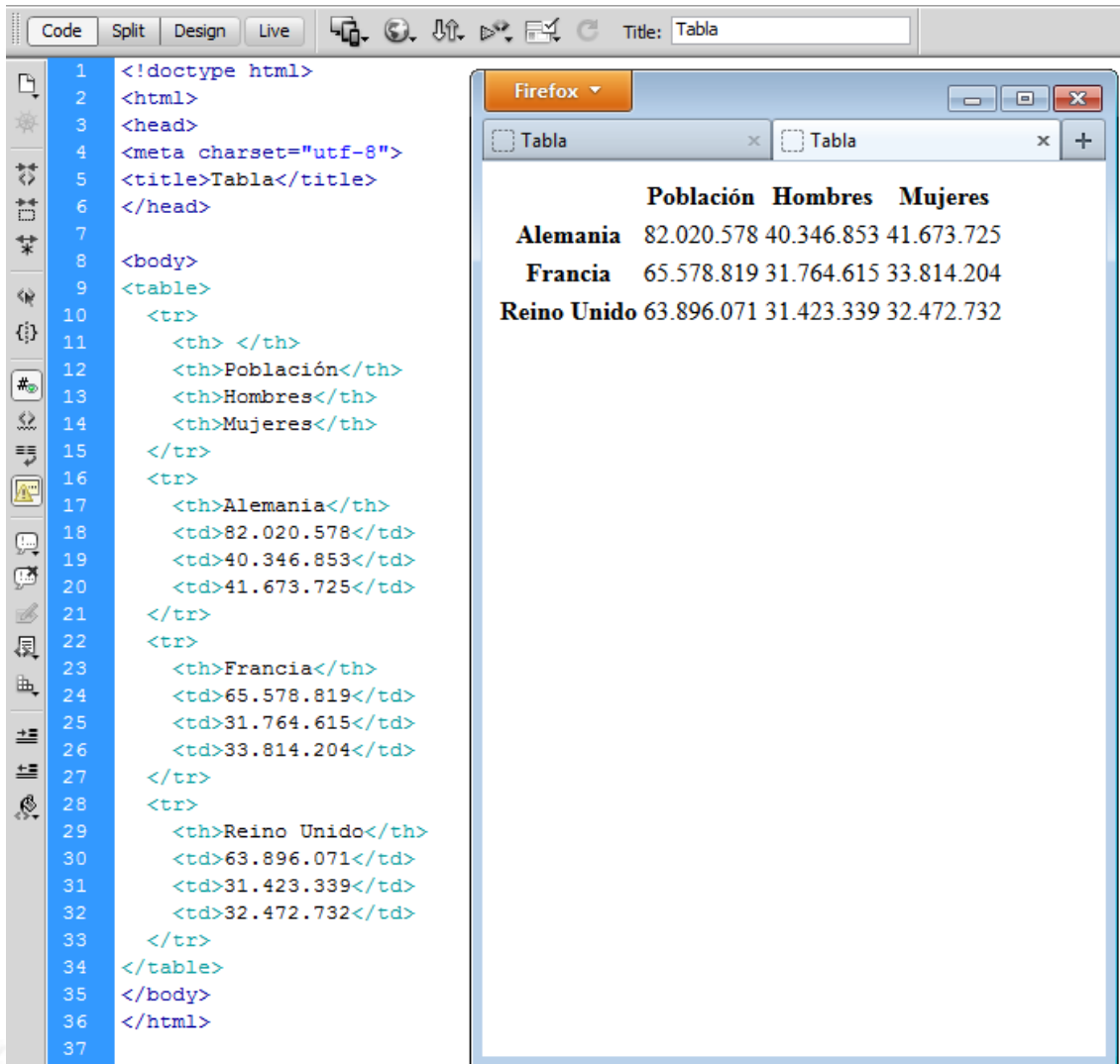
The screenshot shows a web development environment with a code editor and a browser preview. The code editor displays the following HTML code:

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>codificación de caracteres</title>
6 </head>
7
8 <body>
9 <p>Si necesitamos escribir caracteres que utiliza
10 el lenguaje HTML para definir sus etiquetas
11 (&lt; &gt; &amp; &quot; &nbsp; &apos;)
12 no se pueden utilizar libremente.
13
14 </p>
15 </body>
16 </html>
```

The browser preview (Firefox) shows the rendered output of the code. The title bar reads "Firefox". The address bar shows "codificación de caracteres". The main content area displays the text: "Si necesitamos escribir caracteres que utiliza el lenguaje HTML para definir sus etiquetas (< > & " ') no se pueden utilizar libremente."

Tabla

Las tablas se definen con las etiquetas `<table>`, `<tr>` para cada fila y `<td>` para cada columna `<th>` indica que una celda es la cabecera.



The image shows a code editor on the left and a Firefox browser window on the right. The code editor displays the following HTML code:

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Tabla</title>
6 </head>
7
8 <body>
9 <table>
10   <tr>
11     <th> </th>
12     <th>Población</th>
13     <th>Hombres</th>
14     <th>Mujeres</th>
15   </tr>
16   <tr>
17     <th>Alemania</th>
18     <td>82.020.578</td>
19     <td>40.346.853</td>
20     <td>41.673.725</td>
21   </tr>
22   <tr>
23     <th>Francia</th>
24     <td>65.578.819</td>
25     <td>31.764.615</td>
26     <td>33.814.204</td>
27   </tr>
28   <tr>
29     <th>Reino Unido</th>
30     <td>63.896.071</td>
31     <td>31.423.339</td>
32     <td>32.472.732</td>
33   </tr>
34 </table>
35 </body>
36 </html>
37
```

The Firefox browser window shows the rendered table:

	Población	Hombres	Mujeres
Alemania	82.020.578	40.346.853	41.673.725
Francia	65.578.819	31.764.615	33.814.204
Reino Unido	63.896.071	31.423.339	32.472.732

Tabla CSS

Código Dividir Diseño En vivo Título: Tabla

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Tabla</title>
6 <style type="text/css">
7 table {
8     padding: 0px;
9     margin: 0px;
10    border-spacing: 0px;
11    font-family: Arial, Helvetica, sans-serif;
12    font-size: 0.8em;
13    border-top-width: 1px;
14    border-top-style: solid;
15    border-top-color: #333;
16 }
17 th, td {
18     padding: 0.5em;
19     border-bottom-width: 1px;
20     border-bottom-style: solid;
21     border-bottom-color: #333;
22 }
23 th {
24     background-color: #E2E2E2;
25 }
26 td {
27     background-color: #F0F0F0;
28 }
```

Firefox

Tabla

	Población	Hombres	Mujeres
Alemania	82.020.578	40.346.853	41.673.725
Francia	65.578.819	31.764.615	33.814.204
Reino Unido	63.896.071	31.423.339	32.472.732

Recursos HTML

MDN: Lista de Elementos HTML5

https://developer.mozilla.org/es/docs/HTML/HTML5/HTML5_lista_elementos

W3C: HTML5 elements

<http://dev.w3.org/html5/markup/elements.html>

HTML5-Visual Cheat Sheet

<https://docs.google.com/viewer?a=v&pid=sites&srcid=bGFuZG1hcmtzY2hvb2wu b3JnfG1zLWNpcmFzLWNSYXNzLXdIYnNpdGV8Z3g6M2I3ZjY1NzJlMjlmNDA3Ng>

El estilo CSS se puede añadir escribiendo directamente las propiedades CSS en la **línea**, es el método más sencillo, se añade un atributo **style** en el elemento concreto dentro de la página. No se pueden reutilizar en otros elementos que comparten las mismas propiedades.

Se escriben las propiedades del estilo en las marcas del HTML como "*nombre: valor*" separadas por punto y coma.

```
<body style="color: red; ">
```

```
<p style="font-size: 16px; color: blue; font-family: Arial,  
Helvetica, sans-serif;">
```

```
Mi primera página</p>
```

CSS

Lo más aconsejable para mantener la separación entre contenido y presentación, es una hoja de estilo CSS separada del HTML, que se importa con un elemento **<link>** en la cabecera.



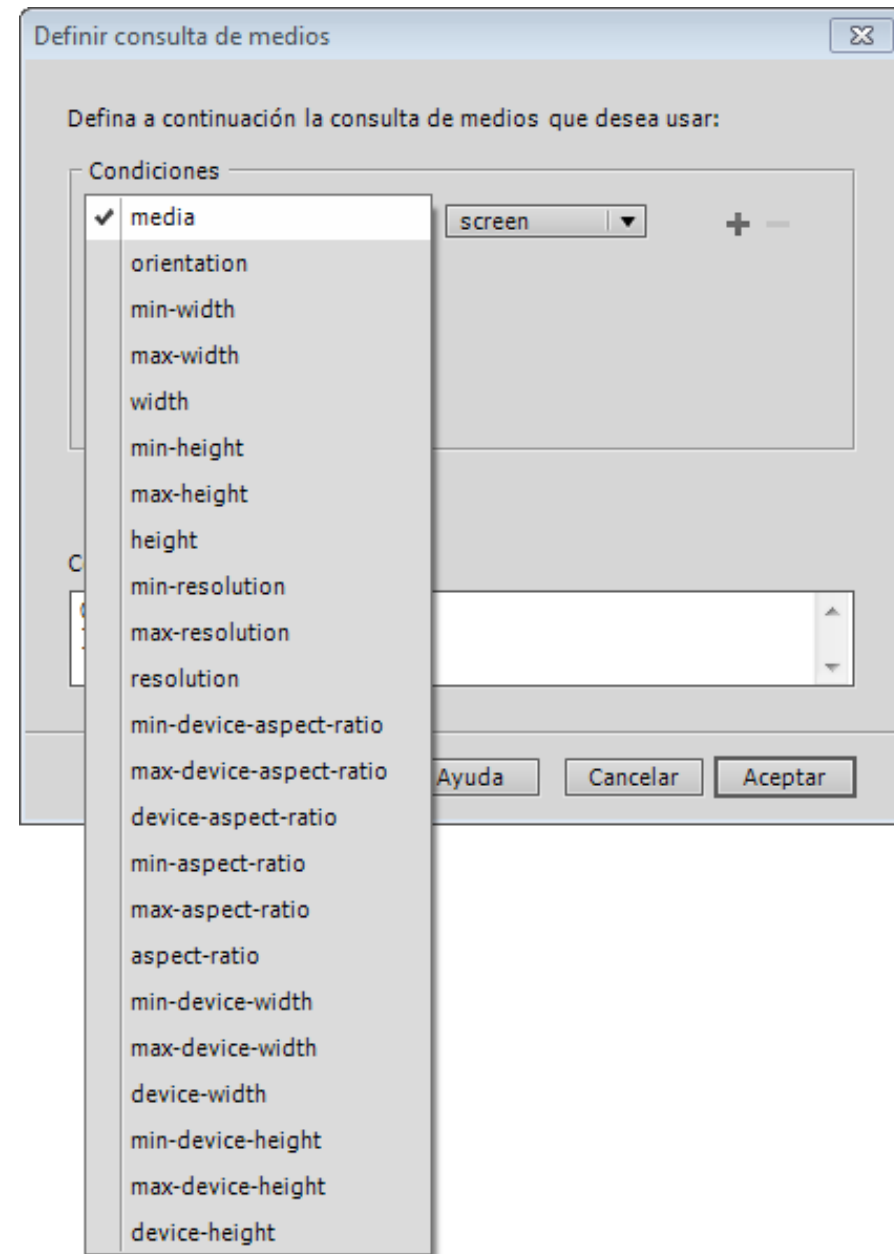
CSS

CSS Permite adaptar la presentación a PCs, móviles, tabletas o impresoras con el atributo **media** que activará el estilo específico de cada dispositivo.

```
<link rel="stylesheet"
type="text/css" media="screen"
href="sans-serif.css">
```

```
<link href="estilo.css"
rel="stylesheet" type="text/css"
media="(min-width:380px)">
```

<http://www.w3.org/TR/css3-mediaqueries/>
https://developer.mozilla.org/es/docs/CSS/Media_queries



Estructura HTML

Los elementos html pueden visualizarse por defecto como bloque o como en línea.

Los elementos bloques **block** ocupan todo el ancho de la página y fuerzan a una nueva línea antes y después.

`<article> <aside> <canvas> <div> <footer> <h1>, <h2>, <h3>, <h4>, <h5>, <h6> <header> <p> <pre> <section> <ul>...`

https://developer.mozilla.org/es/docs/Web/HTML/Block-level_elements

Los elementos en línea **inline** sólo ocupan el ancho necesario y no fuerzan nuevas líneas.

`<em> <a> <br> <img> <span> <button> <input> <select> <textarea>...`

https://developer.mozilla.org/es/docs/Web/HTML/Elementos_en_l%C3%ADnea

Estructura HTML

DIV

El elemento **<div>** es un elemento de bloque que se emplea para contener en su interior otros elementos **HTML**. No tiene ningún significado, es semánticamente neutro.

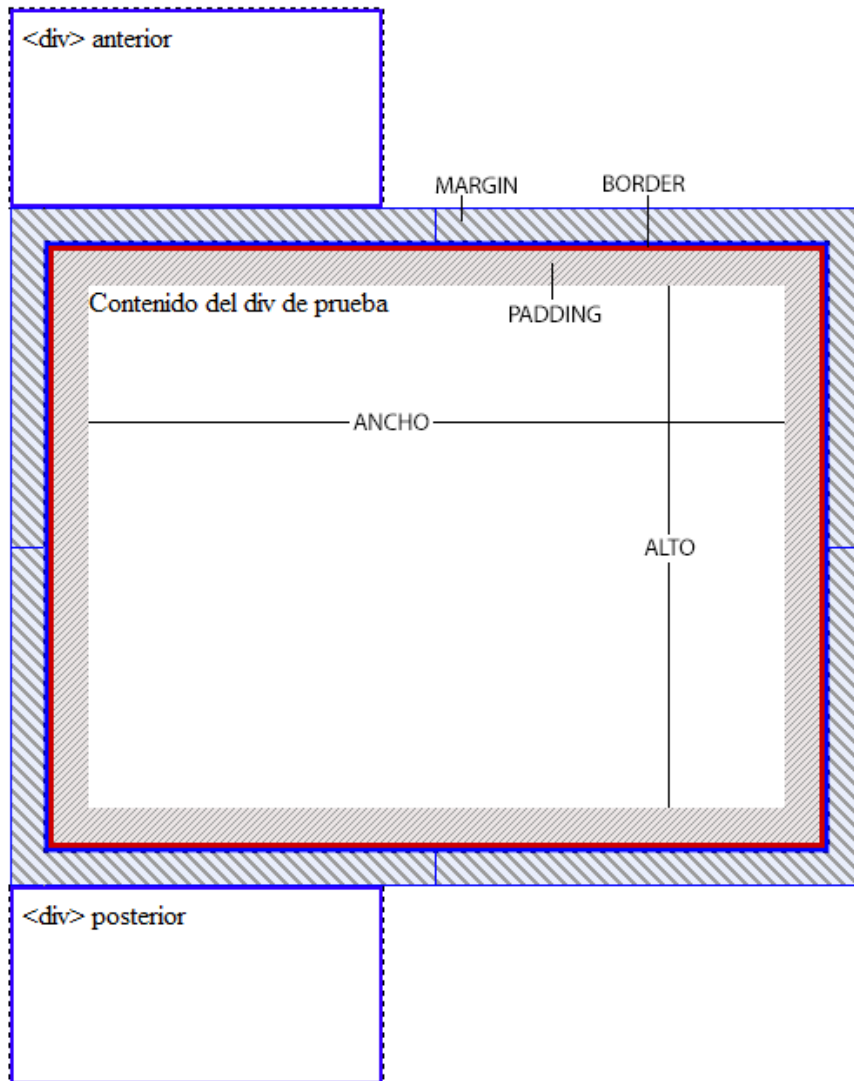
<div id="caja" class="card"> </div>

Atributo **ID**: identifica unívocamente un elemento HTML en una página. Un elemento HTML solo debe tener un atributo **ID** y este debe ser único en la página. El selector CSS **#caja** se refiere al elemento con atributo **id="caja"**, como en **#caja {color:blue}**

Atributo **CLASS**: define una clase de elementos HTML. Un elemento puede tener varios atributos **CLASS**. El selector CSS **.card** se refiere a todos los elemento de la clase **card** (que llevan el atributo **class="card"**) , como en **.card {color:red}**

div CSS

Padding, margin y border

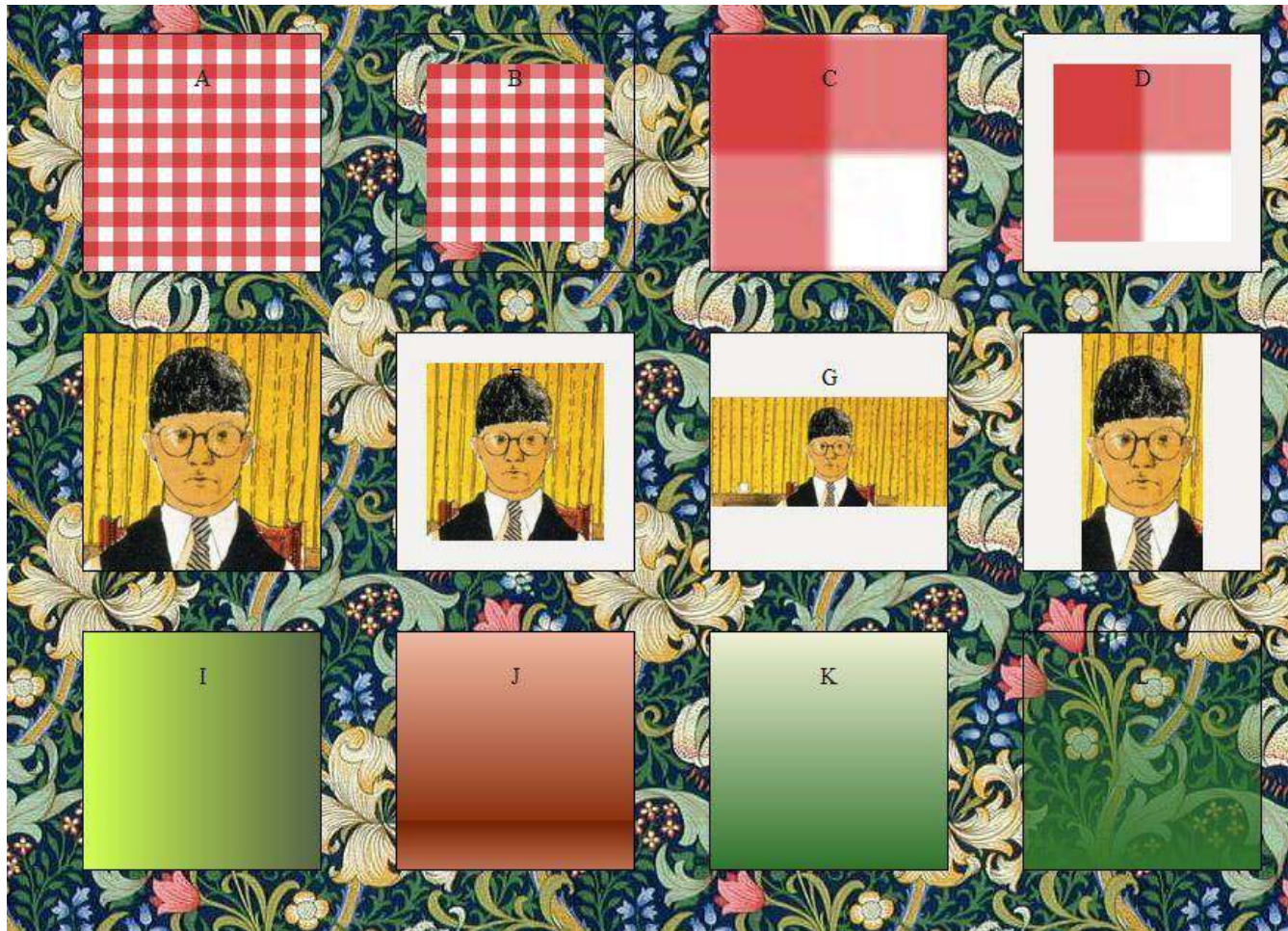


```
<!doctype html>
<html>
<head>
<meta charset="utf-8">
<title>div</title>
<style type="text/css">
.muestra {
    height: 300px;
    width: 400px;
    border: 5px solid #C00;
    margin: 20px;
    padding: 20px;
}
.anterior, .posterior {
    height: 100px;
    width: 200px;
    border: 2px solid #30F;
    padding: 5px;
}
</style>
</head>

<body>
<div class="anterior">&lt;div&gt; anterior</div>
<div class="muestra">Contenido del div de prueba</div>
<div class="posterior">&lt;div&gt; posterior</div>
</body>
</html>
```

Estructura CSS

Background



 `back_mantel.png`

 `dh.png`

 `dh_2.png`

 `dh_3.png`

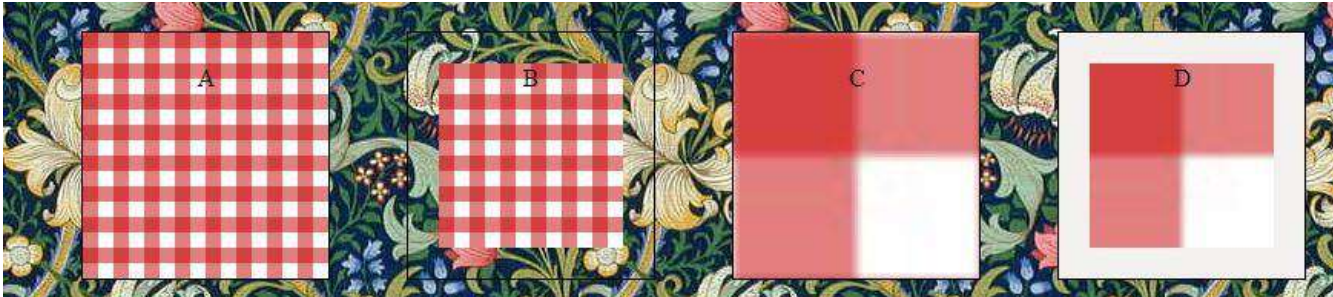
 `ac.jpg`

<http://www.css3files.com/background/>

e/s/d/
madrid

Estructura CSS

Background

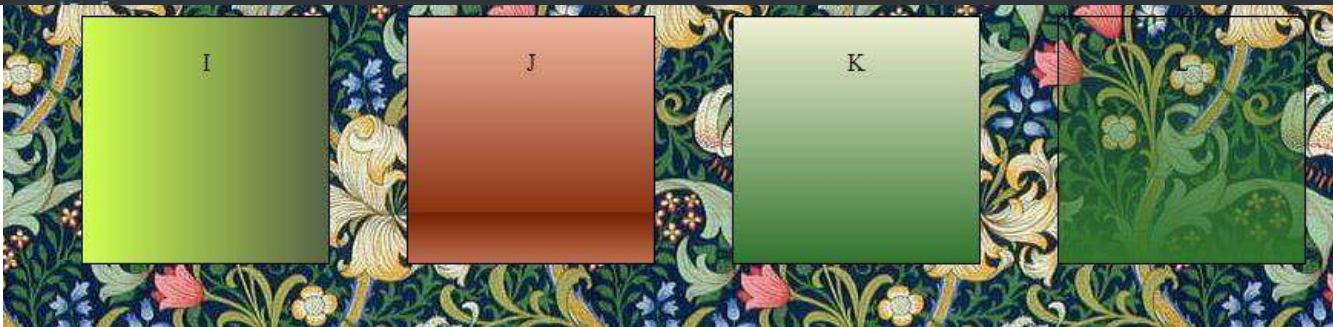


 `back_mantel.png`

 `dh.png`

```
19 body {  
20     background-image: url(images/ac.jpg);  
21 }  
22 .modA {  
23     background-image: url(images/back_mantel.jpg);  
24 }  
25 .modB {  
26     background-image: url(images/back_mantel.jpg);  
27     background-clip: content-box;  
28 }
```

```
29 .modC {  
30     background-image: url(images/back_mantel.jpg);  
31     background-size: 100% 100%;  
32 }  
33 .modD {  
34     background-image: url(images/back_mantel.jpg);  
35     background-repeat: no-repeat;  
36     background-size: 100% 100%;  
37     background-origin: content-box;  
38 }
```



 `dh_3.png`

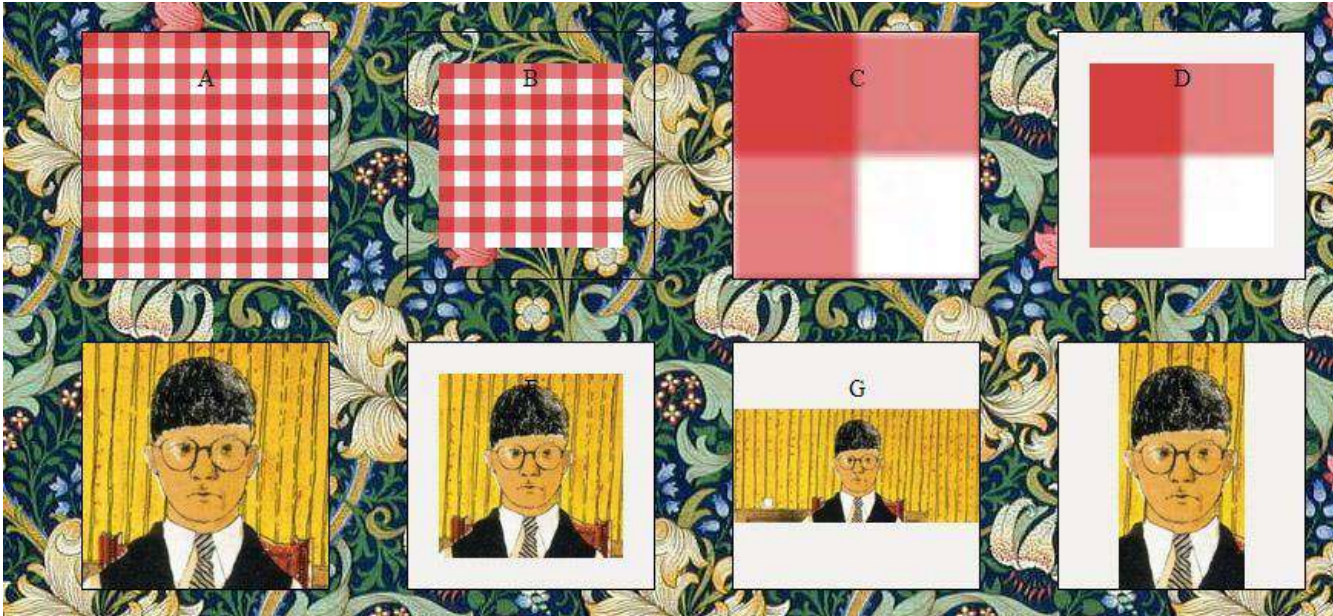
 `ac.jpg`

<http://www.css3files.com/background/>

e/s/d/
madrid

Estructura CSS

Background



 back_mantel.png

 dh.png

 dh_2.png



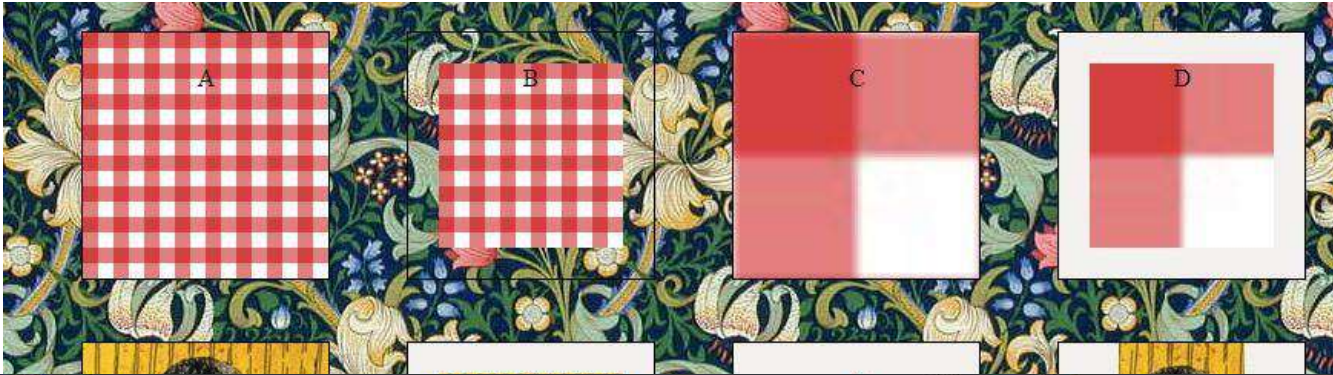
```
39 .modE {  
40     background-image: url(images/dh.jpg);  
41     background-repeat: no-repeat;  
42     background-size: cover;  
43 }  
44 .modF {  
45     background-image: url(images/dh.jpg);  
46     background-repeat: no-repeat;  
47     background-size: cover;  
48     background-origin: content-box;  
49 }
```

```
50 .modG {  
51     background-image: url(images/dh_2.jpg);  
52     background-repeat: no-repeat;  
53     background-size: contain;  
54     background-position: center;  
55 }  
56 .modH {  
57     background-image: url(images/dh_3.jpg);  
58     background-repeat: no-repeat;  
59     background-size: contain;  
60     background-position: center;  
61 }
```

<http://www.css3files.com/background/>

Estructura CSS

Background



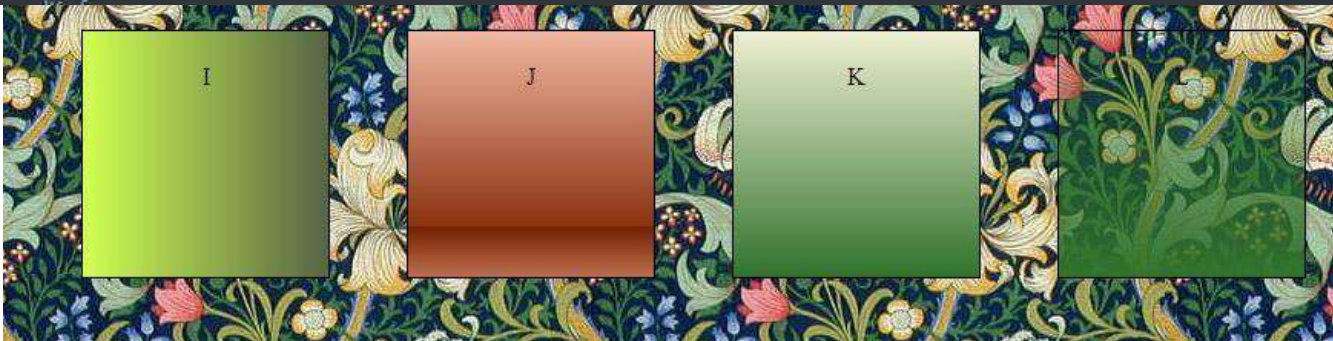
 `back_mantel.png`

 `dh.png`



```
62 .modI {  
63     background: linear-gradient(to right, #d2ff52 0%, #566647 100%);  
64 }  
65 .modJ {  
66     background: linear-gradient(to bottom, rgba(240,183,161,1)  
0%,rgba(140,51,16,1) 80%,rgba(117,34,1,1) 51%,rgba(191,110,78,1)  
100%);  
67 }
```

```
68 .modK {  
69     background: linear-gradient(to top, rgba(50,116,45,1) 0%,  
70     rgba(239,239,210,1) 100%);  
71 }  
72 .modL {  
73     background: linear-gradient(to top, rgba(50,116,45,1) 0%,
```



 `dh_3.png`

 `ac.jpg`

<http://www.css3files.com/background/>

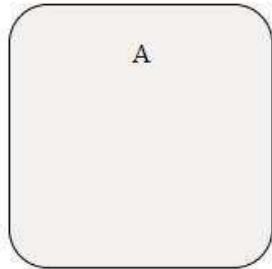
e/s/d/
madrid



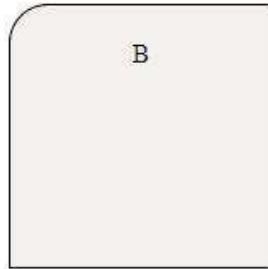
```
div {
  background: url(texto/f01.png) left bottom
    repeat-x,
    url(texto/f02.png) left bottom no-repeat,
    url(texto/f03.png) left top repeat-x;
  background-color: #a4d3ff;
}
```

Estructura CSS

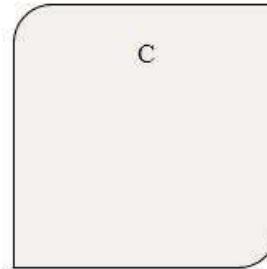
CSS3 permite añadir **bordes y bordes redondeados**



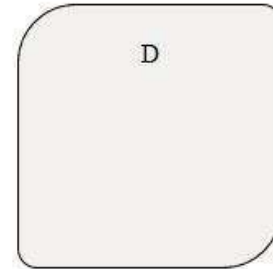
border-radius: 24px;



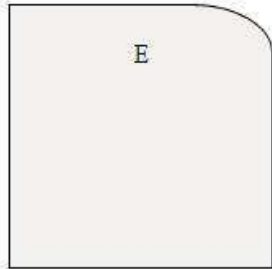
border-top-left-radius: 24px;



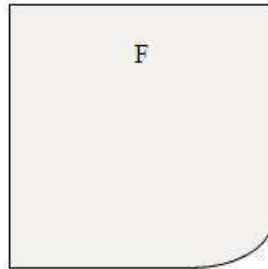
border-radius: 24px 0;



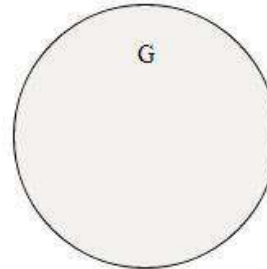
border-radius: 36px 12px;



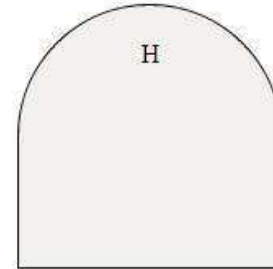
border-top-right-radius: 50px 30px;



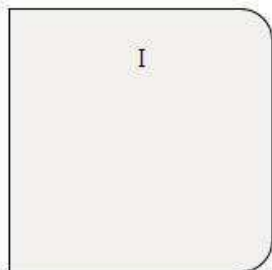
border-bottom-right-radius: 50px 30px;



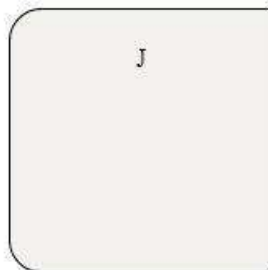
border-radius: 50%;



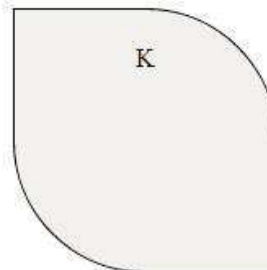
border-radius: 50% 50% 0 0;



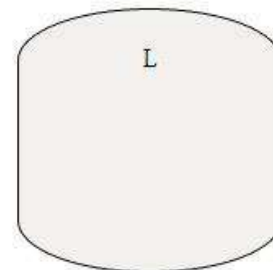
border-radius: 0 20px 20px 0;



border-radius: 20px 0 0 20px;



border-radius: 0 50% 0 50%;



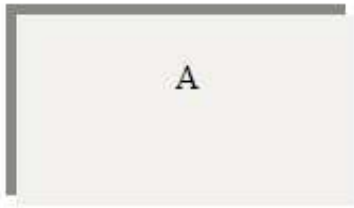
border-top-left-radius: 50% 20%;
border-top-right-radius: 50% 20%;
border-bottom-left-radius: 50% 20%;
border-bottom-right-radius: 50% 20%;

<http://www.cssmatic.com/es/border-radius>

Estructura CSS

CSS3 permite añadir **sombras**

`box-shadow: none|offset-x offset-y blur-radius spread-radius color |inset|initial|inherit;`



`box-shadow: -5px -5px #888;`



`box-shadow: -5px -5px 5px #888;`



`box-shadow: -5px -5px 0 5px #888;`



`box-shadow: -5px -5px 5px 5px #888;`



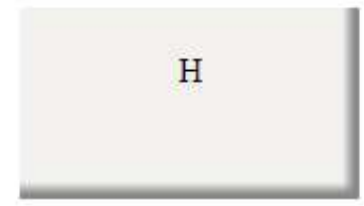
`box-shadow: 0 0 5px #888;`



`box-shadow: 0 0 5px 5px #888;`



`box-shadow: inset -5px -5px #888;`



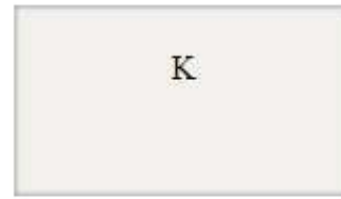
`box-shadow: inset -5px -5px 5px #888;`



`box-shadow:
inset -5px -5px 0 5px #888;`



`box-shadow:
inset -5px -5px 5px 5px #888;`



`box-shadow: inner 0 0 5px #888;`



`box-shadow: inset 0 0 5px 5px #888;`

<http://dev.w3.org/csswg/css-backgrounds/>

Estructura HTML

En **HTML5**, los bloques **div** son sustituidos por bloques semánticos que muestran significado por si mismos.

<header>

<nav>

<section>

<article>

Cabecera

THE TIMES

332,146
SUBSCRIBERS

JOIN NOW
FROM JUST
£2 A WEEK

Navegación

Business | Money | Sport | Life | Arts | Puzzles | Papers

Friday, August 2 | London | Max 26°C

Latest News

Ten charged with grooming offences against teenage girls

Contenido

Home Office investigated over stop-and-search checks

Sadie Gray
Last updated at 3:49PM, August 2 2013

The Home Office is to be investigated by the UK's equality watchdog after random spot-checks for illegal immigrants were reportedly aimed only at people from ethnic minorities. The stop-and-search style checks, performed yesterday outside London railway stations, contributed to a total of 139 arrests in a series of immigration raids around the country. The Equality and Human Rights Commission will investigate the spot-checks and a controversial advertisement campaign in which posters bearing the words "illegal immigrants – go home or face arrest" are displayed on the side of vans. A spokesman said this afternoon: "The commission is writing today to the Home Office about these reported operations, confirming that it will be



Ashes: Aussies get Root and Bresnan

Minimum 30 years jail for Daniel Pelka killer couple

China 'building aircraft carrier'

Satellite images of a shipyard near Shanghai have persuaded leading Western military experts that China may have begun building its first indigenous aircraft carrier.

Destacados

Fracking: drilling starts in Sussex as protests continue

Berlusconi: party relaunch despite looming house arrest

New RBS chief: on £1m-a-year - but he won't get a bonus

Zimbabwe: African observers give mixed verdict on election

Hotel deaths: cyanide suicide theory for Edinburgh couple

Terror alert: US to shut Mid East embassies



Follow reaction to The #Ashes with our live dashboard

Free to non-subscribers

THE TIMES

Stay connected by tablet, phone and web

The Digital Pack

Pie

13 | Version 4.6.0.0(86651)

My Account | RSS | Classified advertising | Display advertising | The Times Whisky Club | Encounters Dating | Sunday Times Wine Club | Privacy & Cookie Policy | Syndication | Site Map | FAQ | Terms & Conditions | Contact us | iPhone | Android smartphone | Blackberry | Windows phone | Android tablet | Kindle | Kindle Fire | Place an announcement in The Times | The Times Bookshop

<aside>

<footer>

e/s/d/
madrid

Estructura HTML

Ejemplo de estructura de una página realizada con los bloques semánticos **HTML5**

The image shows a code editor on the left and a browser window on the right, illustrating the visual output of the HTML5 semantic structure.

Code Editor (Left):

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <link href="html5_estilo.css" rel="stylesheet" type="text/css">
6 <title>estructura HTML5</title>
7 </head>
8 <body>
9 <div class="caja">
10 <header>
11 <h1>Cabecera: header</h1>
12 </header>
13 <nav>
14 <ul>
15 <li><a href="#">Menú 1</a></li>
16 <li><a href="#">Menú 2</a></li>
17 <li><a href="#">Menú 3</a></li>
18 </ul>
19 </nav>
20 <article>Contenido de la página:
21 <strong>article &gt; section</strong></article>
22 <aside>Área de contenido destacado
23 y relacionado: <strong>aside</strong></aside>
24 <footer>Pie: <strong>footer</strong></footer>
25 </div>
26 </body>
27 </html>
```

Browser Window (Right):

The browser window displays the rendered page structure:

- Cabecera: header** (H1)
- Menú 1 Menú 2 Menú 3** (Navigation menu)
- Contenido de la página: article > section** (Main content area)
- Área de contenido destacado y relacionado: aside** (Side content area)
- Pie: footer** (Footer)

A red box labeled **+ CSS** is positioned below the code editor, with a red line pointing to the browser window, indicating that the visual styling is provided by a CSS file.

Estructura HTML

La estructura **HTML**, debe realizarse en el sentido lógico de lectura del documento, cuando la página se vea sin aplicar los estilos no pierda su significado.

theguardian

News Sport Comment Culture Business Money Life & style Travel Environment Tech TV Video Dating Offers Jobs

News UK World Development US Politics Media Education Society Science Women Law Data Football Observer

Breaking news: Review: Motorola phone notable for customization - AP

Twitter issues new rules to crack down on abuse



After rape and bomb threats, site to updates rules, roll out 'report abuse' button more widely and use extra staff to police tweets

Lucy Manzan: net trolls and the currency of hate

Immigration spotchecks 'not racist'

Home Office contests claims that non-white people are being targeted by border agency at tube stations

- Farage: immigrant checks 'deeply disturbing'
- Alex Andreou: a new low for the Home Office

Telecoms firms pass details to GCHQ



Leading companies, including BT, Vodafone and Verizon, secretly collaborating with Britain's spy agency

1102 comments

- Peter Sommer: who's watching GCHQ?
- Analysis: GCHQ's room for manoeuvre
- Inside top secret world of UK's biggest spy agency

British nationals told to leave Yemen



Foreign Office say any Britons who stay in Yemen are unlikely to be able to be evacuated if unrest gets worse

US on alert over al-Qaida threat

Chaos over Tory welfare reform



DWP employees criticise

The Ashes England v Australia day three - live!



guardianjobs

Guardian Jobs
Reach quality candidates today
Click here »

Online dating



Find your soulmate with the Guardian's dating site
Join Guardian Soulmates

Subscribe

SUBSCRIBE

Learn more about subscribing to the Guardian in print

Beginner's blogging course



Interactive Celebrity selfies



Will Self
On happy pills



Sport

Gerrard to Suárez: stay at Liverpool



Football: Steven Gerrard says disgruntled team-mate Luis Suárez should stay at Anfield and earn a dream move

- Wenger: €100m Bale bid is a joke
- Newcastle agree deal to sign Gomes
- Sheffield Utd 2-1 Notts County

Championship preview: 2013-14



Jacob Steinberg casts his eye over the second-tier league's chances as the Football League kicks off again

Your search terms... The Guardian Search

theguardian

Twitter issues new rules to crack down on abuse



After rape and bomb threats, site to updates rules, roll out 'report abuse' button more widely and use extra staff to police tweets

- Lucy Manzan: net trolls and the currency of hate

Immigration spotchecks 'not racist'

Home Office contests claims that non-white people are being targeted by border agency at tube stations

- Farage: immigrant checks 'deeply disturbing'
- Alex Andreou: a new low for the Home Office

Telecoms firms pass details to GCHQ



Leading companies, including BT, Vodafone and Verizon, secretly collaborating with Britain's spy agency

1102 comments

- Peter Sommer: who's watching GCHQ?
- Analysis: GCHQ's room for manoeuvre
- Inside top secret world of UK's biggest spy agency

British nationals told to leave Yemen



Foreign Office say any Britons who stay in Yemen are unlikely to be able to be evacuated if unrest gets worse

e/s/d/
madrid

Recursos CSS

CSS3 Generator

<http://css3generator.com/>

CSS3 Gradient Generator

<http://gradients.glrzad.com/>

{CSS} Portal - on line generator

<http://www.cssportal.com/css3-rounded-corner/>

CSS3 MAKER:

<http://www.css3maker.com>

CSS3.com

<http://www.css3.com/>

Recurso 1

Título del recurso relacionado: Del monolítico al cliente-servidor distribuido: guía conceptual paso a paso

Descripción del recurso: Hoja de ruta narrada para entender cómo se pasa de una aplicación monolítica a una arquitectura cliente-servidor moderna con servicios especializados. Incluye hitos, decisiones y riesgos típicos.

Objetivo de la guía

Que el/la estudiante visualice la evolución arquitectónica y entienda qué problemas resuelve cada transición, sin implementar código.

Paso a paso

Paso 1. Punto de partida: monolito.

Se parte de una sola base de código con UI, lógica y datos acoplados. *Ventaja:* simplicidad inicial. *Dolor:* despliegues lentos, una falla afecta todo.

Paso 2. Separación lógica por capas.

Se introduce la división conceptual (presentación, negocio, datos). Aunque sigue en un único binario, aparece la idea de responsabilidades. *Aprendizaje:* la modularidad reduce el impacto del cambio.

Paso 3. Primera escisión: cliente vs servidor.

La UI pasa a ejecutarse en el cliente (navegador/escritorio) y la lógica a un servidor accesible por red (HTTP/HTTPS). *Beneficio:* centralizar reglas y actualizar sin reinstalar clientes.

Paso 4. Contratos estables (APIs).

Se formalizan endpoints REST/GraphQL como “contratos”. *Claves:* versionado de API, documentación; *riesgo:* romper compatibilidad.

Paso 5. Escalado horizontal con balanceo.

Se replica el servidor detrás de un *load balancer* (L7). *Idea crítica:* estado de sesión fuera del proceso (cookies firmadas/JWT, caché central).

Paso 6. Datos distribuidos y caché.

Se introducen *read replicas*, *sharding* y cachés (ej. Redis) para reducir latencia. *Trade-off:* coherencia vs rendimiento.

Paso 7. Observabilidad y resiliencia.

Se agregan *logs estructurados*, métricas (p99 de latencia), *tracing* distribuido y patrones de resiliencia (circuit breaker, *timeouts*, reintentos exponenciales).

Paso 8. Endurecimiento de seguridad.

TLS en tránsito, *secret management*, políticas CORS, autenticación (OAuth2/OIDC), autorización por *scopes/roles* y WAF para amenazas OWASP.

Resultado esperado (conceptual): Comprender *qué problema resuelve cada salto* y *qué nuevo riesgo introduce*, para decidir con criterio en los retos.

Sugerencia visual

Infografía “Evolución arquitectónica” (creación propia): línea de tiempo con iconografía de monolito → 3 capas → cliente-servidor → balanceo → caché/replicas → observabilidad/seguridad.

Recurso 2

Título del recurso relacionado: Del request al response: recorrido de una solicitud en una arquitectura cliente-servidor actual

Descripción del recurso: Mapa narrado de extremo a extremo: qué sucede desde que el cliente hace clic hasta que ve la respuesta. Incluye responsabilidades por capa y señales de calidad.

Objetivo de la guía

Hacer visible el “camino” de una petición para relacionar los conceptos de la Arquitectura cliente-servidor moderna con decisiones de diseño y pruebas.

Paso a paso

Paso 1. DNS y CDN.

El dominio se resuelve a una IP; una CDN puede servir assets estáticos cerca del usuario.

Indicador: cache hit ratio alto.

Paso 2. Handshake TLS (HTTPS).

Negociación de cifrado y certificados válidos. *Indicador:* ningún contenido mixto; *riesgo:* certificados expirados.

Paso 3. Balanceador de carga (L7).

Distribuye tráfico por *health checks* y políticas (round-robin/least-connections). *Indicador:* error rate homogéneo entre instancias.

Paso 4. Reverse proxy y caché de aplicación.

Normaliza cabeceras, comprime, aplica reglas; puede cachear respuestas idempotentes.

Beneficio: latencias más bajas.

Paso 5. Capa de aplicación.

Controladores validan entrada, coordinan dominio y orquestan servicios. *Buenas prácticas:* *timeouts*, *circuit breakers*, idempotencia en POST críticos.

Paso 6. Persistencia y consultas.

Estrategias de lectura/escritura (replicas, colas asíncronas). *Riesgo:* *N+1 queries*; *mitigación:* *joins* adecuados o agregaciones.

Paso 7. Seguridad de sesión y CORS.

Tokens (JWT) firmados/rotados; CORS restrictivo. *Indicador:* cero *preflight* fallidos y revocación de tokens efectiva.

Paso 8. Observabilidad.

Trazas con *correlation id*, métricas (RPS, p95/p99, saturación), alertas SLO. *Meta*: detectar regresiones antes que el usuario.

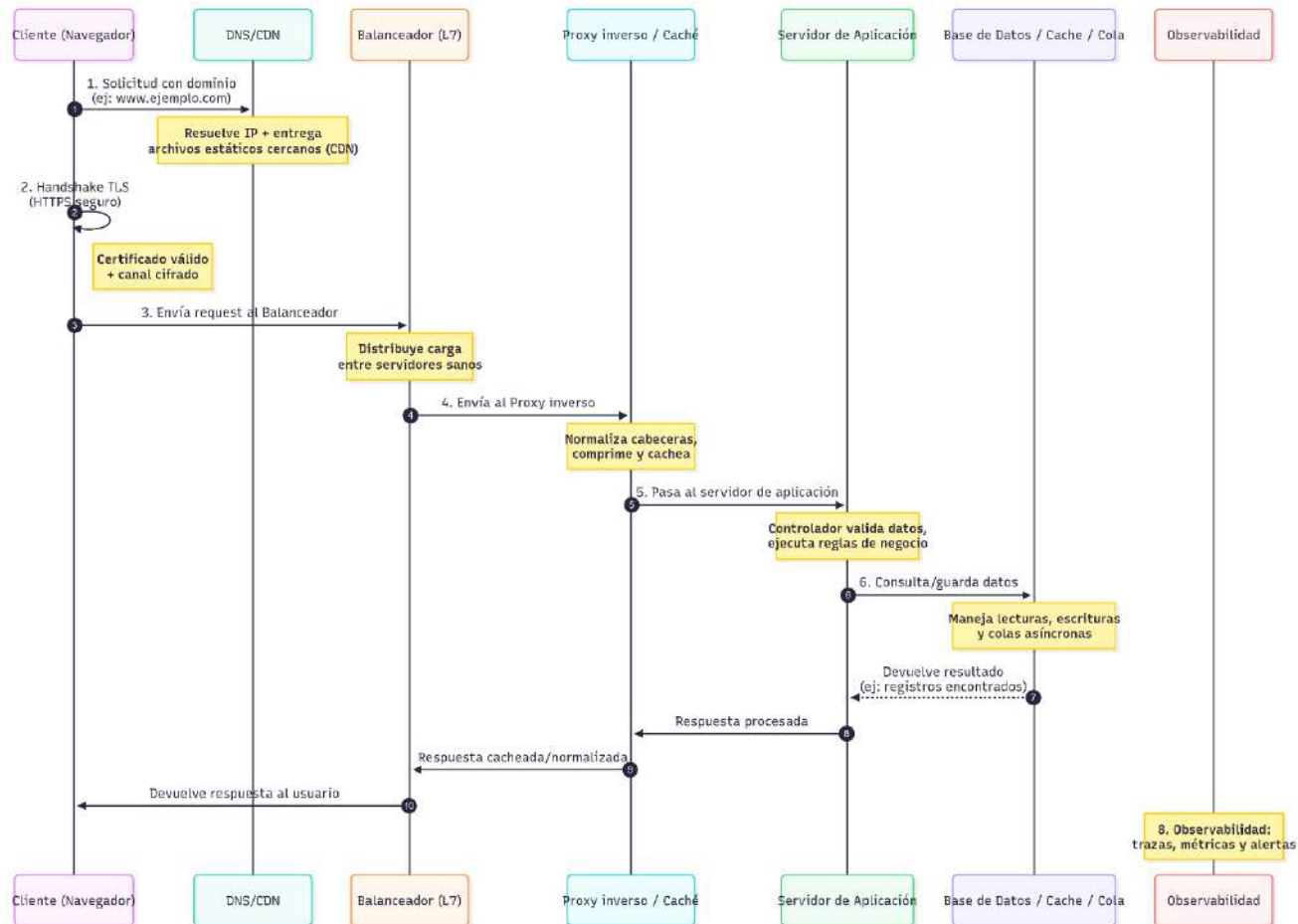
Tabla de referencia rápida

Tabla A. Capas, objetivos, fallos comunes y métricas clave

Capa	Objetivo principal	Fallos comunes	Métrica clave
DNS/CDN	Cercanía y cache de estáticos	TTL mal ajustado, <i>miss</i> constante	Cache hit ratio
TLS/HTTPS	Confidencialidad/Integridad	Certificado inválido/mixto	% tráfico en HTTPS
Balanceador (L7)	Distribución y <i>health checks</i>	<i>Hot spotting</i> , <i>sticky</i> excesivo	Error rate por instancia
Reverse proxy / Caché	Normalizar y acelerar	Reglas contradictorias	Latencia p95
App (controlador/dominio)	Ejecutar reglas de negocio	<i>Timeouts</i> ausentes, N+1 queries	Tasa de errores 5xx
Datos (DB/Cache/Queue)	Persistencia y performance	Bloqueos, contención, <i>thundering</i>	QPS, locks, cache hit
Seguridad (Auth/CORS)	Identidad y acceso	Tokens sin rotación, CORS laxo	Fallos de auth/autz
Observabilidad	Visibilidad y alerta temprana	Sin <i>correlation id</i>	MTTR/MTTD

Sugerencia visual

Diagrama de secuencia (creación propia): Cliente → CDN → LB → Proxy → App → DB/Cache → App → Cliente, con anotaciones de cabeceras y tiempos.



Recurso 3

Título del recurso relacionado: HTML5 semántico + WCAG 2.2 AA: checklist comentada y ejemplos canónicos

Descripción del recurso: Lista guiada para evaluar semántica y accesibilidad sin herramientas externas, con ejemplos típicos (navegación, tarjetas, formularios) y notas ARIA.

Objetivo de la guía

Ofrecer una ruta clara para reconocer buen marcado semántico y criterios AA de accesibilidad, con casos reales de interfaz.

Paso a paso (checklist comentada)

Paso 1. Estructura global correcta.

Un solo `<main>` por página; `<header>`/`<footer>` por sitio o sección.

Paso 2. Navegación identificable.

Usar `<nav>`; si hay varias, diferenciarlas con `aria-label` ("Navegación principal", "Breadcrumbs").

Paso 3. Jerarquía de encabezados.

Un `<h1>` por página; subsecciones con `<h2>`...`<h6>` en orden lógico (no para estilos).

Paso 4. Landmarks y roles.

Preferir elementos semánticos; si no hay, añadir role apropiado (por ejemplo, `role="complementary"` para un `<aside>` complejo).

Paso 5. Imágenes con propósito.

`alt` descriptivo para imágenes informativas; `alt=""` para decoración pura.

Paso 6. Formularios accesibles.

Cada `<input>` con `<label for>` o `aria-label`; *hint* con `aria-describedby`; mensajes de error vinculados al control.

Paso 7. Estado y foco visibles.

No eliminar *focus outline* sin reemplazo perceptible; *tab order* coherente.

Paso 8. Contraste de color.

Texto normal $\geq 4.5:1$; texto grande ($\geq 18\text{px}$ o 14px negrita) $\geq 3:1$. Evitar texto sobre imágenes sin *overlay*.

Paso 9. Componentes interactivos.

No usar <div> clicables: preferir <button>/<a> según corresponda. Añadir aria-pressed, aria-expanded si aplica.

Paso 10. Contenido dinámico.

Para actualizaciones, usar aria-live="polite" (no bloquear lectura) o assertive si es crítico. Evitar *announcements* redundantes.

Paso 11. Tablas de datos.

<th scope="col/row"> para cabeceras; caption para describir propósito; evitar tablas para layout.

Paso 12. Enlaces significativos.

El texto del enlace debe describir destino (“Ver detalles de matrícula”), no “clic aquí”.

Paso 13. Accesibilidad por teclado.

Todo alcanzable con Tab/Enter/Space. *Trap focus* manejado en modales con aria-modal="true".

Paso 14. Semántica en componentes de tarjeta.

Usar <article> para una tarjeta independiente; título en <h3> dentro de la tarjeta; acciones con <button>.

Paso 15. Validación básica.

Comprobar con atajos (Lighthouse/Axe) y luego inspección manual: *skip links*, foco lógico y lectura con lector de pantalla.

Tabla de mapeo útil

Tabla B. Patrón de UI → Elementos semánticos → ARIA sugerido → Falla común

Patrón UI	Elementos semánticos	ARIA sugerido	Falla común
Menú principal	<nav> + lista	aria-label="Principal"	<div> suelto con <i>onclick</i>
Tarjeta de noticia	<article> + <h3>	—	Encabezados saltados (h1→h4)

Modal	<dialog>/ contenedor	aria-modal, aria- labelledby	Foco sin retorno al cierre
Accordion	<button> + región	aria-expanded, aria- controls	Solo estilos; sin semántica ni foco
Formulario	<form> + <label>	aria-describedby	Inputs sin label ni mensajes de error

Resultado esperado (conceptual): Reconocer, sin “hacer”, cómo luce un documento correcto y dónde suelen aparecer los errores que rompen la accesibilidad.

Ejemplo canónico (ilustrativo, no actividad)

Fragmento de formulario correctamente asociado (con *hints* y error anclado).

```
<form aria-labelledby="tituloseccion">

  <h2 id="tituloseccion">Registro</h2>

  <label for="correo">Correo institucional</label>

  <input id="correo" type="email" aria-describedby="ayuda-correo error-correo">

  <p id="ayuda-correo">Usa tu cuenta @ejemplo.edu.ec</p>

  <p id="error-correo" role="alert">Formato inválido</p>

  <button type="submit">Enviar</button>

</form>
```



FORMULARIO DE PRODUCCIÓN DE CONTENIDOS

NOMBRE DE LA ASIGNATURA: Desarrollo en plataformas
NOMBRE COMPLETO DEL DOCENTE CONTENIDISTA: Damián Aníbal Nicolalde Rodríguez

DESARROLLO

1. INTRODUCCIÓN DE LA CLASE

Para la Clase 2 daremos un paso natural desde lo que construimos en la Semana 1: pasamos de una página base y accesible a entender cómo se organiza y se comunica un documento web cuando incluye formularios reales. Empezaremos por ver qué nos ofrece HTML5 para “hablar con el usuario” sin complicarnos: tipos de campo que ya validan lo básico, etiquetas que dan nombre accesible, ayudas y errores que se leen con lectores de pantalla, y una estructura semántica que convierte a la página en un mapa claro (landmarks, encabezados y piezas como figure/figcaption o time). La idea es que puedas mirar tu HTML y reconocer de inmediato dónde estás y qué es cada cosa, antes de pensar en estilos o JavaScript.

Luego conectaremos esa base con validación responsable y buenas prácticas WCAG. Verás cuándo dejar que el navegador haga su trabajo (nativo), cuándo sumar reglas de negocio con un poco de JS sin romper la accesibilidad (mensajes anunciados, foco al primer error), y por qué siempre debemos confirmar en el servidor. Cerraremos con el marco POUR (Perceptible, Operable, Comprensible, Robusto) para que tus formularios sean usables con teclado, legibles con zoom y claros para todos. Recuerda: hoy nos concentramos en el por qué y el cómo conceptual; en la práctica repetiremos estos patrones paso a paso para que queden listos para los retos.

Clase 2: HTML5 avanzado y formularios accesibles

RDA 1: Programar aplicaciones web dinámicas, interactivas y con diseño visualmente adaptable, que permita mejorar la experiencia del usuario.

Reto # 1



2.1. HTML5 avanzado y formularios accesibles (teoría)

Piensa en un formulario como una conversación bien guiada. HTML5 ya trae “buenas preguntas” (tipos de campo), “reglas claras” (atributos de validación) y “pistas” (etiquetas y ayudas). Nuestro trabajo es: 1) usar primero lo nativo, 2) nombrar todo con **label**, 3) enlazar ayudas/errores con **aria-describedby**, y 4) dejar que el navegador valide lo básico.

2.1.1. Mapa mental rápido

1. **Una tarea por formulario** (registro, solicitud, pago).
2. **Orden lógico**: obligatorios primero, opcionales después.
3. **Nombre accesible** con label for.
4. **Ayudas/errores** junto al campo y **vinculados**.
5. **Botón principal** único (type="submit").

2.1.2. Tipos de entrada: deja que el navegador ayude

Los tipos de `<input>` activan teclados móviles adecuados y validaciones de formato. Úsalos siempre que exista uno para tu dato.

Tabla 1. *Tipos de `<input>` útiles y su aplicación práctica*

Tipo	¿Para qué sirve?	Ventaja práctica de UX/A11y
email	Correos	Formato básico + teclado móvil adecuado
url	Enlaces	Valida esquema; reduce errores
tel	Teléfonos	Teclado numérico (móvil)
number	Cantidades exactas	min/max/step; evita letras
range	Preferencias con deslizador	Añade un valor visible con <code><output></code>
date/time	Fechas/horas	Selectores nativos; menos JS
file	Subir archivos	accept, multiple; valida también en servidor
color	Selector de color	Personalización nativa

Nota. Tabla de referencia. Elaboración propia (D. Nicolalde, 2025).

2.1.3. Estructura mínima accesible

```
<form aria-labelledby="t-registro">
  <!-- Título orienta a lectores de pantalla y a cualquier persona -->
  <h2 id="t-registro">Registro</h2>

  <!-- label for => "nombre accesible" del control -->
  <label for="nombre">Nombre completo</label>
  <input id="nombre" name="nombre" type="text"
    required minlength="2" autocomplete="name"
    aria-describedby="hint-nombre">
  <!-- Hint vinculado: se anuncia junto al campo -->
  <small id="hint-nombre">Como figura en tu documento (mín. 2 caracteres).</small>

  <label for="email">Correo institucional</label>
  <input id="email" name="email" type="email"
    required autocomplete="email"
    aria-describedby="err-email">
  <!-- Error: oculto al inicio; se mostrará si falla la validación -->
  <span id="err-email" role="alert" hidden>Usa el formato usuario@dominio.</span>

  <!-- Radios relacionados se agrupan con fieldset+legend -->
  <fieldset>
    <legend>Preferencia de contacto</legend>
    <label><input type="radio" name="pref" value="correo" checked> Correo</label>
    <label><input type="radio" name="pref" value="telefono"> Teléfono</label>
  </fieldset>

  <button type="submit">Crear cuenta</button>
</form>
```

CSS base (estados y foco):

```
/* Visualiza estados nativos sin JS */
input:required:invalid, select:required:invalid { border: 2px solid #c00; }
input:required:valid, select:required:valid { border: 2px solid #0a0; }

/* Foco visible para navegación con teclado */
:focus-visible { outline: 3px solid #000; outline-offset: 2px; }

/* Enlaces para saltar al contenido principal (accesibilidad global) */
.skip-link { position: absolute; left: -999px; top: -999px; }
.skip-link:focus { position: static; background: #eee; padding: .5rem; }
```



2.1.4. Restricciones nativas (sin JS)

Con atributos ya cubres mucha validación:

```
<!-- number con rango y paso: evita valores fuera de rejilla -->
<label for="creditos">Créditos</label>
<input id="creditos" name="creditos" type="number"
  min="1" max="40" step="1" required>

<!-- ID nacional hipotético (10 dígitos): pattern simple y teclado numérico -->
<label for="idnac">Identificación nacional</label>
<input id="idnac" name="idnac" type="text" required
  pattern="^\d{10}$" inputmode="numeric"
  aria-describedby="hint-id">
<small id="hint-id">10 dígitos, sin espacios ni guiones.</small>
```

Si más tarde necesitas controlar todos los mensajes con JS, puedes añadir novalidate en el <form>. De inicio, apóyate en lo nativo.

2.1.5. Mensajes claros y anunciados (accesibilidad)

Un buen mensaje explica la regla y cómo corregir. Si lo generas con JS, marca el control con aria-invalid="true" y anuncia el texto con role="alert".



```
<label for="correo">Correo</label>
<input id="correo" type="email" required aria-describedby="err-correo">
<span id="err-correo" role="alert" hidden>
  Escribe un correo con formato usuario@dominio.
</span>

<script>
  const correo = document.getElementById('correo');
  const err = document.getElementById('err-correo');

  // Validación ligera "en vivo": no bloquea mientras se teclea
  correo.addEventListener('input', () => {
    correo.setCustomValidity(""); // limpia estado previo
    err.hidden = true;           // oculta error por defecto

    if (!correo.checkValidity()) { // usa validación nativa (type="email")
      const msg = 'Escribe un correo con formato usuario@dominio.';
      correo.setCustomValidity(msg); // define texto del error
      correo.setAttribute('aria-invalid','true'); // marca estado
      err.textContent = msg; err.hidden = false; // se anunciará por role="alert"
    } else {
      correo.removeAttribute('aria-invalid');
    }
  });
</script>
```

2.1.6. Autocompletado: accesibilidad también es teclear menos

Usa tokens estándar (given-name, family-name, email, tel, address-line1, postal-code, country) para reducir errores y esfuerzo, sobre todo en móvil y para personas con ayudas técnicas.

```
<label for="nombres">Nombres</label>
<input id="nombres" name="nombres" autocomplete="given-name" required>

<label for="apellidos">Apellidos</label>
<input id="apellidos" name="apellidos" autocomplete="family-name" required>

<label for="telefono">Teléfono</label>
<input id="telefono" name="telefono" type="tel" autocomplete="tel" inputmode="tel">
```

2.1.7. Formularios por pasos

Cuando hay muchos campos, dividir en pasos baja la carga cognitiva. Cada paso debe tener **título** (<h2>), **progreso** (<progress value="1" max="3">) y **gestión del foco** (al avanzar, el foco va al título del nuevo paso). No borres datos al volver.

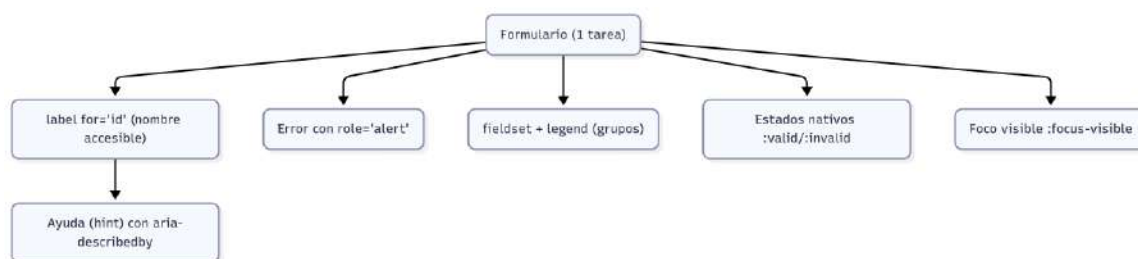


Figura 1. Anatomía de un formulario accesible

Nota. Diagrama conceptual. Elaboración propia (D. Nicolalde, 2025).

2.1.8. ¿Qué valida el navegador y qué agrego yo?

Tabla 2. Estados de validación nativa (ValidityState) y qué comunicar

Flag	Causa típica	Mensaje claro para el usuario
valueMissing	Falta valor (required)	“Completa este campo.”
typeMismatch	email/url mal formateado	“Usa el formato usuario@dominio.”
patternMismatch	No coincide con pattern	“Sigue el formato indicado.”
tooShort/tooLong	Longitud fuera de rango	“Ajusta la cantidad de caracteres.”
rangeUnder/Over	Fuera de min/max	“Elige un valor permitido.”
stepMismatch	No cae en la rejilla de step	“Ajusta al paso (p. ej., múltiplos de 1).”
customError	Error definido por JS	Explica la regla de negocio y solución

Nota. Resumen para guiar mensajes. Elaboración propia (D. Nicolalde, 2025).

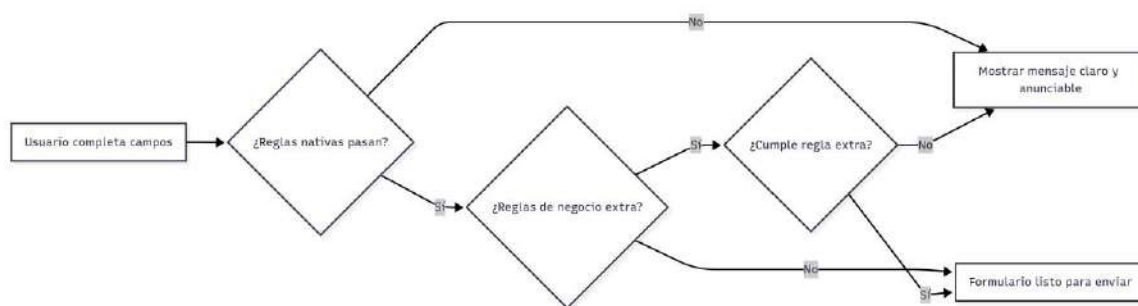


Figura 2. Flujo mental de validación en cliente (nativo + reglas propias)

Nota. Esquema secuencial para decidir qué capa valida y cuándo. Elaboración propia (D. Nicolalde, 2025).

Enlace 1

- **Título del enlace relacionado:** Guía práctica de validación de formularios
- **Descripción:** Introducción clara a validación nativa y personalizada, con ejemplos de `checkValidity()`, `setCustomValidity()` y patrones comunes.



- **Enlace:** https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation

2.1.9. Internacionalización y seguridad (mínimos)

- **Fechas/números:** la UI puede mostrarse local, pero guarda en ISO/UTC.
- **Mensajes:** en el idioma del documento (<html lang="es">), sin tecnicismos.
- **Seguridad:** el cliente mejora la experiencia; el servidor decide. Revalida y sanea, usa **HTTPS**, protege **CSRF** si hay sesión, y verifica **tipo real** y tamaño en archivos (no confíes solo en accept).

2.2. Etiquetas estructurales y semánticas

Piensa que una página responde dos preguntas todo el tiempo: **(a) dónde estoy** (estructura) y **(b) qué es esto** (semántica). Si usamos las etiquetas correctas, la página “habla” por sí misma: los lectores de pantalla pueden saltar por regiones, Google entiende el contenido, y tú (o tu equipo) pueden mantener el código sin adivinar. La regla madre es: **HTML nombra y ordena; CSS acomoda; JS anima.**

2.2.1. Landmarks: las “zonas” navegables

Los *landmarks* son regiones estándar que permiten saltos rápidos (teclado y lectores de pantalla). Los principales:

- header (cabecera global o de sección),
- nav (navegación),
- main (contenido principal, único por documento),
- section (bloque temático con título),
- article (unidad independiente: noticia, ficha, post),
- aside (contenido complementario),
- footer (pie global o de sección).

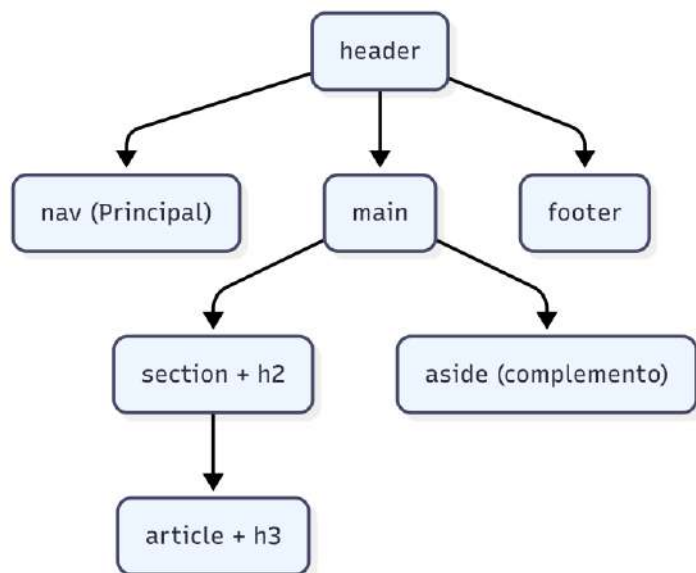


Figura 3. *Mapa de landmarks típicos en una página sencilla*

Nota. Diagrama conceptual de regiones semánticas. Elaboración propia (D. Nicolalde, 2025).



Fragmento base con landmarks:

```
<body>
  <!-- Enlace para saltar directamente al contenido principal (a11y) -->
  <a class="skip-link" href="#contenido">Saltar al contenido</a>

  <header>
    <h1>Plataforma Académica</h1>
    <!-- Menú principal dentro de nav; aria-label distingue si hay varias nav -->
    <nav aria-label="Principal">
      <ul>
        <li><a href="#" aria-current="page">Inicio</a></li>
        <li><a href="#novedades">Novedades</a></li>
        <li><a href="#ayuda">Ayuda</a></li>
      </ul>
    </nav>
  </header>

  <!-- Un único main por documento -->
  <main id="contenido">
    <!-- section: agrupación temática con título propio -->
    <section aria-labelledby="t-nov">
      <h2 id="t-nov">Novedades</h2>

      <!-- article: pieza que tiene sentido por sí sola -->
      <article aria-labelledby="a1">
        <h3 id="a1">Lanzamiento del Panel</h3>
        <p>Publicado el <time datetime="2025-09-22">22/09/2025</time>.</p>
      </article>

      <!-- aside: información complementaria (no esencial) -->
      <aside aria-label="Contactos rápidos">
        Soporte: <a href="mailto:soporte@ejemplo.edu.ec">soporte@ejemplo.edu.ec</a>
      </aside>
    </section>
  </main>

  <footer>© 2025 Plataforma Académica</footer>
</body>
```

2.2.2. Encabezados: el “índice” del documento

Los encabezados (h1–h6) indican jerarquía, no tamaño visual. Reglas prácticas:

1. Un solo h1 (título principal de la página).
2. Secciones con h2; subsecciones con h3, y así sucesivamente.

3. No saltes niveles por estética (el tamaño lo da CSS).
4. Toda section o article debería tener su encabezado.

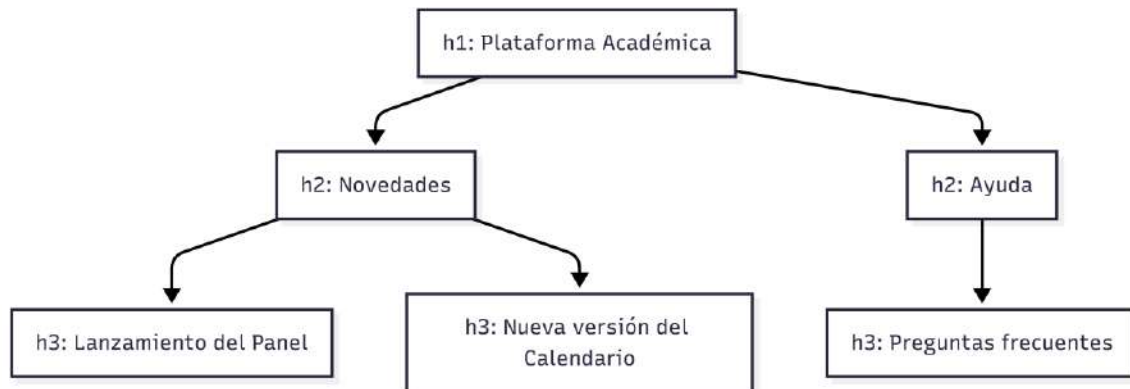


Figura 4. *Jerarquía de encabezados bien formada*

Nota. La jerarquía debe seguir el orden lógico (no saltar de h1 a h4). Elaboración propia (D. Nicolalde, 2025).

2.2.3. Semántica fina: “qué es” cada pieza

Además de las zonas, nombra las piezas internas:

- **Imágenes con contexto:** figure + figcaption. El alt describe la información visual; el pie añade contexto.
- **Fechas:** time datetime="YYYY-MM-DD" (texto humano + valor legible por máquinas).
- **Abreviaturas:** abbr title="..." (p. ej., WCAG).
- **Código/teclado/salida:** code, kbd, samp, y pre para formato preservado.
- **Énfasis/Importancia:** em y strong (significado, no solo estilo).
- **Resaltado contextual:** mark (p. ej., término buscado).
- **Definiciones/Pares clave–valor:** dl + dt + dd.
- **Sección colapsable:** details + summary (acordeón nativo y accesible).



Fragmentos útiles:

```
<!-- Imagen informativa con pie -->
<figure>
  
  <figcaption>Vista del panel tras iniciar sesión (métricas principales).</figcaption>
</figure>

<!-- Fecha legible por humanos + máquina -->
<p>Actualizado el <time datetime="2025-09-22">22 de septiembre de 2025</time>.</p>

<!-- Abreviatura -->
<p>Cumple con <abbr title="Pautas de Accesibilidad al Contenido Web">WCAG</abbr> AA.</p>

<!-- Acordeón nativo para FAQ -->
<details>
  <summary>¿Cómo recupero mi contraseña?</summary>
  <p>Usa la opción “Olvidé mi contraseña” y revisa tu correo institucional.</p>
</details>
```

2.2.4. Listas, definiciones y tablas (bien usadas)

- **Listas:** ul (sin orden), ol (ordenadas). Cada elemento es un li.
- **Definiciones:** dl/dt/dd son excelentes para glosarios o pares clave–valor.
- **Tablas** (solo datos tabulares, no para maquetar):
 - caption (qué estoy viendo),
 - th scope="col|row" para cabeceras,
 - thead, tbody, tfoot si aplica.

Tabla 3. *Mapeo UI → etiqueta semántica recomendada*

Patrón UI	Etiqueta(s) sugerida(s)	Nota de accesibilidad
Cabecera global	header	Puede contener nav/buscador
Menú principal / migas	nav (+ aria-label si hay varias)	Salto rápidos por landmarks
Contenido principal	main (único)	Primer destino tras “Saltar al contenido”
Tarjeta de noticia/ficha	article + h3 + figure/figcaption	Unidad independiente
Fecha de publicación	time datetime="YYYY-MM-DD"	Legible por máquinas
FAQ	details + summary	Nativo y accesible
Panel lateral	aside	Complementario, no esencial
Tabla de datos	table + caption + th scope="..."	No usar para layout

Nota. Tabla de referencia. Elaboración propia (D. Nicolalde, 2025).



Ejemplo de tabla correcta:

```
<table>
  <caption>Estados del sistema por entorno</caption>
  <thead>
    <tr>
      <th scope="col">Entorno</th>
      <th scope="col">Estado</th>
      <th scope="col">Actualizado</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <!-- Cabecera de fila con scope="row" -->
      <th scope="row">Desarrollo</th>
      <td>Operativo</td>
      <td><time datetime="2025-09-22">22/09/2025</time></td>
    </tr>
  </tbody>
</table>
```

2.2.5. ¿section, article o div?

Test rápido:

- ¿Tiene sentido por sí solo (se podría compartir o syndicar)? → article.
- ¿Agrupa contenido relacionado que merece título propio? → section.
- ¿Solo necesitas un contenedor para CSS/JS sin significado? → div.

Tabla 4. Elección entre section, article y div

Pregunta guía	Elemento	Ejemplo típico
¿Vive por sí mismo (unidad independiente)?	article	Noticia, ficha de producto
¿Agrupa temas con un título claro?	section	Bloque “Novedades”, “Ayuda”
¿Solo sirve para estilos/JS?	div	Contenedor de grid/flex sin semántica

Nota. Criterios prácticos de decisión. Elaboración propia (D. Nicolalde, 2025).

2.2.6. ARIA: cuándo sí y cuándo no

ARIA no reemplaza HTML. Primero elige el elemento correcto; luego, si no existe equivalente nativo, usa ARIA para completar:

- Varias nav → diferéncialas con aria-label="Principal", aria-label="Breadcrumbs".



- Acordeón custom (si no usas details): role="button", aria-expanded, aria-controls, y teclas Enter/Espacio.
- Evita roles redundantes: no pongas role="navigation" en nav, ni role="main" en main.

Fragmento (botón colapsable custom, solo como idea):

```
<button aria-expanded="false" aria-controls="bloque1" id="toggle1">
  Ver más
</button>
<div id="bloque1" hidden>
  <p>Contenido adicional...</p>
</div>

<script>
  const btn = document.getElementById('toggle1');
  const panel = document.getElementById('bloque1');
  btn.addEventListener('click', () => {
    const expanded = btn.getAttribute('aria-expanded') === 'true';
    btn.setAttribute('aria-expanded', String(!expanded));
    panel.hidden = expanded; // alterna visibilidad
  });
</script>
```

2.2.7. Anti-patrones comunes (y su arreglo)

- **“Div soup”** (todo con div) → Usa landmarks y semántica fina.
- **Encabezados por tamaño** (usar h4 “porque es más chico”) → Usa el nivel correcto y estiliza con CSS.
- **section sin encabezado** → Añade h2/h3 dentro o usa div.
- **Figuras sin figcaption** cuando la imagen requiere contexto → Añádelo.
- **Tablas para layout** → Maqueta con Flexbox/Grid (Semana 3).
- **Roles ARIA arbitrarios** que contradicen HTML → Valida y elimina lo redundante.

2.3. Validación nativa y personalizada

Cuando validas un formulario, no “castigas” al usuario: lo acompañas. HTML5 ya trae una validación nativa muy útil (tipos, rangos, obligatoriedad) y tú puedes añadir reglas propias con JavaScript sin perder accesibilidad. La idea es simple:

1. **Deja al navegador** lo común (formato de email, rango numérico).
2. **Agrega** con JS lo que es de negocio (ID, fechas relacionadas, disponibilidad).
3. **Revalida en servidor** siempre (seguridad e integridad).

2.3.1. Tres capas que cooperan

Tabla 5. *Capas de validación y responsabilidades principales*



Capa	¿Qué hace mejor?	Ejemplos típicos	Accesibilidad implicada
Nativa (HTML/CSS)	Formatos y restricciones genéricas	type="email", required, min/max, pattern	Estados :invalid/:valid, mensajes del navegador
Personalizada (JS)	Reglas de negocio y cruzadas	"Fin ≥ Inicio", checksum de ID, fuerza de contraseña	aria-invalid, aria-describedby, role="alert"
Servidor	Veracidad y seguridad	Revalidar, normalizar, sanitizar, CSRF, MIME real en file	Mensajes coherentes con la UI; no revelar detalles sensibles

Nota. Tabla de referencia. Elaboración propia (D. Nicolalde, 2025).

2.3.2. Activar la validación nativa (casi sin código)

Empieza por lo más simple: tipos correctos y atributos de restricción. El navegador hace el resto.

```
<form aria-labelledby="t-registro">
  <h2 id="t-registro">Registro</h2>

  <!-- label for = nombre accesible del control -->
  <label for="correo">Correo institucional</label>
  <input id="correo" name="correo" type="email" required
    autocomplete="email" aria-describedby="err-correo">

  <!-- Mensaje de error visible/legible (inicialmente oculto) -->
  <span id="err-correo" role="alert" hidden>Usa el formato usuario@dominio.</span>

  <!-- number con rango y paso -->
  <label for="creditos">Créditos a tomar</label>
  <input id="creditos" name="creditos" type="number"
    min="1" max="40" step="1" required>

  <button type="submit">Crear cuenta</button>
</form>
/* Visualiza estados nativos sin JS */
input:required:invalid { border: 2px solid #c00; } /* inválido */
input:required:valid { border: 2px solid #0a0; } /* válido */

/* Foco visible para teclado */
:focus-visible { outline: 3px solid #000; outline-offset: 2px; }
```

2.3.3. Mensajes claros y anunciados (accesibilidad)

Un buen mensaje dice qué regla se incumple y cómo corregir. Si personalizas mensajes, anúncialos con ARIA.

```
<label for="idnac">Identificación nacional (10 dígitos)</label>
<input id="idnac" name="idnac" type="text" required pattern="^\d{10}$"
  inputmode="numeric" aria-describedby="hint-id err-id">
<small id="hint-id">Escribe 10 dígitos, sin espacios ni guiones.</small>
<span id="err-id" class="error" role="alert" hidden>Revisa el formato (10 dígitos).</span>

<script>
  const idnac = document.getElementById('idnac');
  const err = document.getElementById('err-id');

  // Validación ligera mientras escribe (no bloquea): usa lo nativo
  idnac.addEventListener('input', () => {
    idnac.setCustomValidity("");
    idnac.removeAttribute('aria-invalid');
    err.hidden = true;

    if (!idnac.checkValidity()) {
      const msg = 'Revisa el formato: exactamente 10 dígitos.';
      idnac.setCustomValidity(msg); // mensaje personalizado
      idnac.setAttribute('aria-invalid', 'true');
      err.textContent = msg; err.hidden = false; // se anuncia por role="alert"
    }
  });
</script>
```

2.3.4. ¿Cuándo validar? Ritmo y tiempos

No interrumpas a cada tecla. Usa una estrategia combinada:

- **onblur** (al salir del campo): formato/obligatoriedad.
- **oninput** con *debounce* (200–400 ms): contadores o “fuerza” de contraseña.
- **onsubmit**: última barrera; enfoca el primer error.

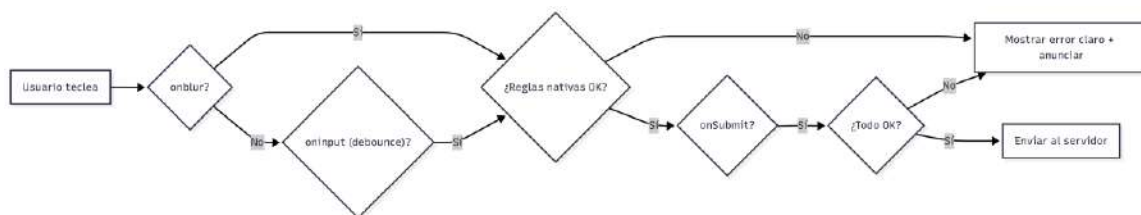


Imagen 5. Flujo de decisión de validación (cliente)

Nota. La validación se anuncia sin bloquear la escritura. Elaboración propia (D. Nicolalde, 2025).

2.3.5. Reglas cruzadas: campos que dependen entre sí

Ejemplo clásico: “Fin ≥ Inicio”. Valida cuando cambie cualquiera.



```
<label for="inicio">Fecha inicio</label>
<input id="inicio" type="date" required>

<label for="fin">Fecha fin</label>
<input id="fin" type="date" required aria-describedby="err-fechas">
<span id="err-fechas" role="alert" hidden>La fecha de fin debe ser posterior a inicio.</span>

<script>
  const ini = document.getElementById('inicio');
  const fin = document.getElementById('fin');
  const err = document.getElementById('err-fechas');

  function validarRango() {
    // Limpia estado anterior
    fin.setCustomValidity(''); fin.removeAttribute('aria-invalid'); err.hidden = true;

    // Si ambos valores existen, compara
    if (ini.value && fin.value && fin.value < ini.value) {
      const msg = 'La fecha de fin debe ser posterior a la de inicio.';
      fin.setCustomValidity(msg);
      fin.setAttribute('aria-invalid', 'true');
      err.textContent = msg; err.hidden = false;
    }
  }
  ini.addEventListener('change', validarRango);
  fin.addEventListener('change', validarRango);
</script>
```

2.3.6. Validación asíncrona (servidor): “¿Este correo existe?”

Indica estado, evita “carreras” y no satures al servidor.



```
<label for="correo2">Correo</label>
<input id="correo2" type="email" required aria-describedby="st-correo2 err-correo2">
<!-- role="status" anuncia cambios no urgentes -->
<small id="st-correo2" role="status" aria-live="polite"></small>
<span id="err-correo2" role="alert" hidden></span>

<script>
  const correo2 = document.getElementById('correo2');
  const st = document.getElementById('st-correo2');
  const er = document.getElementById('err-correo2');

  let timer; // para debounce
  let lastQuery = 0; // para evitar respuestas "viejas"

  correo2.addEventListener('input', () => {
    // Limpieza de mensajes y estado
    correo2.setCustomValidity(""); correo2.removeAttribute('aria-invalid');
    er.hidden = true; st.textContent = "";

    // Si el formato ya es inválido, no consultes servidor
    if (!correo2.checkValidity()) return;

    // Debounce: espera 300ms desde la última tecla
    clearTimeout(timer);
    timer = setTimeout(async () => {
      const qid = ++lastQuery; // marca esta consulta
      st.textContent = 'Comprobando disponibilidad...';

      try {
        // Simulación de endpoint (reemplaza por tu API)
        // const resp = await fetch('/api/email/check?e=' + encodeURIComponent(correo2.value));
        // const { exists } = await resp.json();
        const exists = false; // ← simulado

        if (qid !== lastQuery) return; // ignora si llegó tarde
        st.textContent = exists ? 'Este correo ya tiene cuenta.' : 'Disponible.';
        if (exists) {
          const msg = 'Este correo ya está registrado. ¿Quieres iniciar sesión?';
          correo2.setCustomValidity(msg);
          correo2.setAttribute('aria-invalid', 'true');
          er.textContent = msg; er.hidden = false;
        }
      } catch (e) {
        if (qid !== lastQuery) return;
        st.textContent = 'No se pudo comprobar ahora. Intenta luego.';
      }
    }, 300);
  });
```

</script>

2.3.7. Disparadores y ritmo (resumen visual)

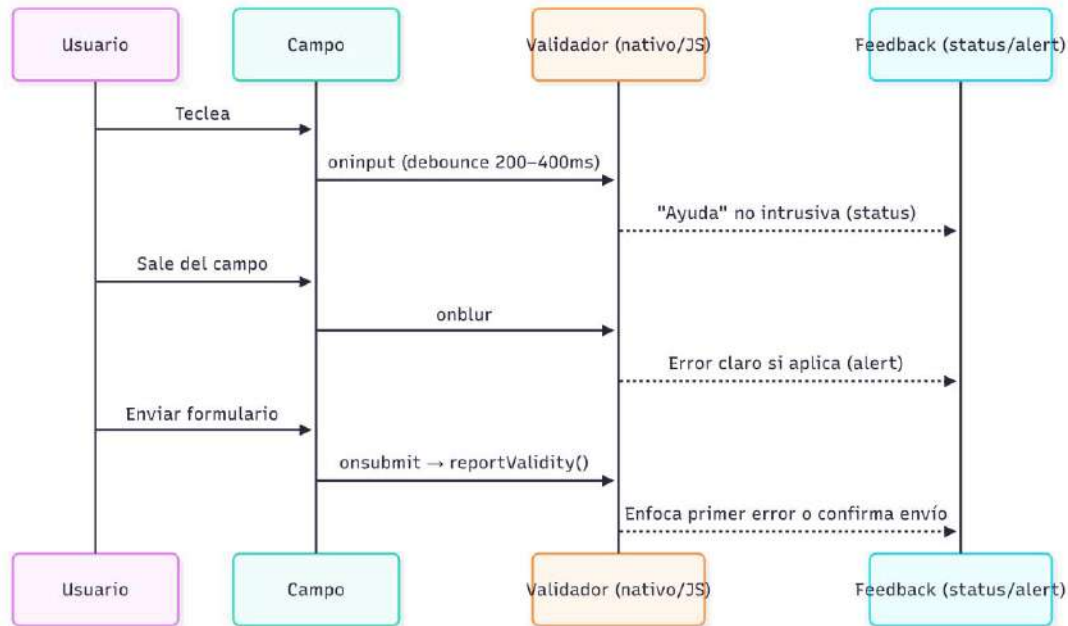


Imagen 6. Línea temporal de disparadores de validación y feedback

Nota. Primero ayudas “suaves”, luego errores “claros”, sin bloquear. Elaboración propia (D. Nicolalde, 2025).

2.3.8. Internacionalización y máscaras

- **Fechas/números:** la UI puede ser local, pero envía/guarda en ISO/UTC.
- **Mensajes:** en el idioma del documento (`lang="es"`), con verbos simples y acción sugerida.
- **Máscaras:** si usas, que no impidan seleccionar/pegar ni rompan lectores; a veces `pattern + hint` es suficiente.

2.3.9. Anti-patrones comunes

- **Placeholders como etiquetas** → Usa `label for`.
- **Solo color para errores** → Añade texto y `role="alert"`.
- **Quitar outline** → Personaliza el foco, no lo elimines.
- **disabled para “solo lectura”** → Usa `readonly`; `disabled` saca del foco y del envío.
- **Bloquear la escritura** con validación “en vivo” agresiva → Usa *debounce* o *onblur*.
- **Modales para cada error** → Mensajes junto al campo + (opcional) resumen arriba con anclas.

2.4. Buenas prácticas de accesibilidad (WCAG)

Cuando hablamos de accesibilidad no estamos “añadiendo algo al final”, estamos diseñando para todas las personas desde el principio. Las WCAG organizan sus pautas en cuatro principios que recordamos como **POUR: Perceptible, Operable, Comprensible y Robusto**. En clase trabajaremos con el objetivo **AA** como mínimo razonable.

2.4.1. Mapa rápido (POUR → acción → prueba)

Tabla 6. *POUR: mapa de acciones concretas y cómo probarlas*

Principio	Acción práctica (AA)	Cómo probar rápido
Perceptible	alt útil en imágenes informativas; figure+figcaption si hace falta contexto	Navega con SR (NVDA/VoiceOver); inspecciona alt
Perceptible	Contraste $\geq 4.5:1$ (texto normal) y $\geq 3:1$ (texto grande)	Lighthouse/Axe; comprobador de contraste
Perceptible	Reflow con zoom 200–400% sin scroll horizontal general	Ctrl/Cmd +; revisa columnas y tarjetas
Operable	Todo con teclado: Tab/Shift+Tab/Enter/Espacio	Recorre la página sin mouse
Operable	Foco visible y orden lógico	:focus-visible; prueba Tab en toda la UI
Operable	Modales accesibles (foco atrapado legit., aria-modal, cerrar con Esc)	Abrir/cerrar con teclado y lector
Comprensible	Etiquetas reales + aria-describedby para ayudas/errores	SR anuncia propósito + error
Comprensible	Jerarquía de encabezados (h1→h2→h3...) y navegación coherente	Vista de encabezados; inspección semántica
Robusto	Semántica nativa primero; ARIA solo si falta equivalente	Validador HTML; roles correctos en DevTools

Nota. Tabla de referencia. Elaboración propia del autor, Damián Nicolalde (2025).

2.4.2. Perceptible: que la información “llegue”

Imágenes. Si la imagen comunica algo, pon alt descriptivo; si es decorativa, alt="". Si requiere contexto, usa figure + figcaption.

```
<!-- Imagen informativa: alt describe la información visual -->
<figure>
  
  <figcaption>Arquitectura lógica del sistema presentada en clase.</figcaption>
</figure>
```

Contraste y reflow. No depende solo del color. Verifica contraste y que con zoom 200–400% la página no se rompa.

```
/* Respetar a usuarios que reducen animaciones por mareo o concentración */
@media (prefers-reduced-motion: reduce){
  * { animation-duration: .01ms !important; animation-iteration-count: 1 !important; }
}
```

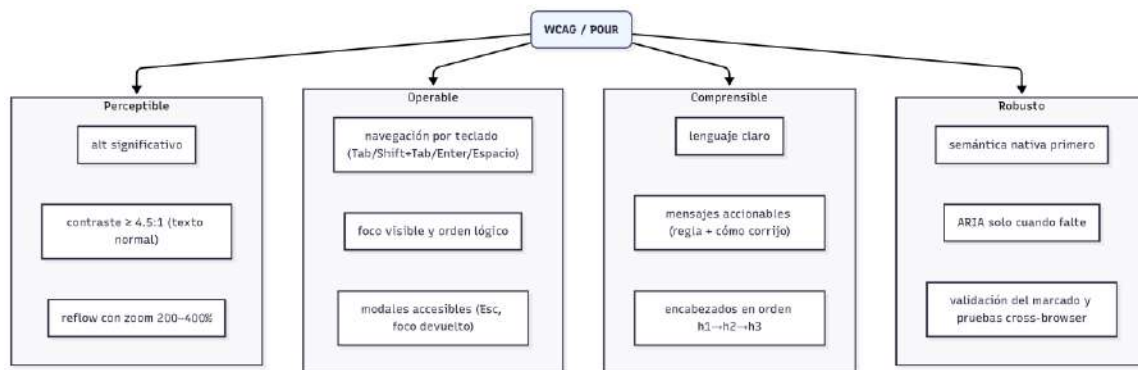


Imagen 7. Esquema mental de los cuatro principios WCAG (POUR)

Nota. Diagrama conceptual. Elaboración propia del autor, Damián Nicolalde (2025).

2.4.3. Operable: teclado first

El sitio debe poder usarse completamente con teclado. Asegura foco visible y orden lógico.



```
<!-- Enlace para saltar al contenido principal: mejora enorme con teclado/SR -->
<a class="skip-link" href="#contenido">Saltar al contenido</a>
```

```
<!-- ... -->
```

```
<main id="contenido">
  <!-- contenido principal -->
</main>
```

```
/* Foco visible: nunca lo elimines; personalízalo si quieres */
:focus-visible { outline: 3px solid #000; outline-offset: 2px; }
```

```
/* Skip link: oculto visualmente, visible al enfocarse */
.skip-link { position: absolute; left: -999px; top: -999px; }
.skip-link:focus { position: static; background: #eee; padding: .5rem; }
```

Modales accesibles (idea mínima). Deben: 1) atrapar foco, 2) anunciarse, 3) cerrarse con Esc, 4) devolver el foco al disparador.

```
<button id="abrir">Abrir modal</button>

<!-- role="dialog" y aria-modal anuncian que es un modal real -->
<div id="modal" role="dialog" aria-modal="true" aria-labelledby="t-modal" hidden>
  <div class="caja">
    <h2 id="t-modal">Confirmar acción</h2>
    <p>¿Deseas continuar?</p>
    <button id="cerrar">Cancelar</button>
    <button id="ok">Sí, continuar</button>
  </div>
</div>

<script>
const abrir = document.getElementById('abrir');
const modal = document.getElementById('modal');
const cerrar = document.getElementById('cerrar');
const ok = document.getElementById('ok');
let ultimoFoco;

function abrirModal(){
  ultimoFoco = document.activeElement;  // guarda quién tenía el foco
  modal.hidden = false;
  modal.querySelector('.caja').focus();  // mueve el foco al modal
  document.addEventListener('keydown', trap);
}

function cerrarModal(){
  modal.hidden = true;
  document.removeEventListener('keydown', trap);
  if (ultimoFoco) ultimoFoco.focus();  // devuelve el foco al disparador
}

function trap(e){
  if (e.key === 'Escape') return cerrarModal();
  // (Aquí iría el ciclo de foco si la caja tuviera muchos elementos)
}

abrir.addEventListener('click', abrirModal);
cerrar.addEventListener('click', cerrarModal);
ok.addEventListener('click', cerrarModal);
</script>
```

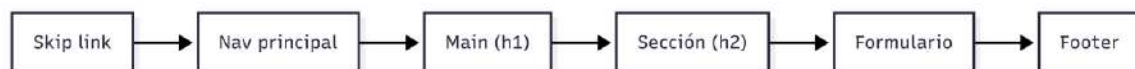


Imagen 8. Recorrido de foco recomendado en una página estándar



Nota. Flujo de foco simplificado para navegación por teclado. Elaboración propia del autor, Damián Nicolalde (2025).

2.4.4. Comprensible: que se entienda lo que pides

Usa lenguaje claro y etiquetas reales. Las ayudas y errores deben estar vinculados y anunciarse.

```
<label for="email">Correo institucional</label>
<input id="email" type="email" required aria-describedby="hint-email err-email">
<small id="hint-email">Formato: usuario@dominio.</small>
<span id="err-email" role="alert" hidden>Revisa el formato del correo.</span>
```

Mantén jerarquía de encabezados (h1→h2→h3...) y patrones de navegación consistentes entre páginas (mismo lugar para menú, buscador, etc.).

Tabla 7. Mensajes accionables: patrón “regla + cómo corrijo”

Caso	Evitar	Preferir
Email inválido	“Inválido”	“Usa el formato usuario@dominio.”
Longitud	“Error en texto”	“Mínimo 8 caracteres; añade 2 más.”
Rango	“Número fuera de rango”	“Elige entre 1 y 40.”
Campo requerido	“Campo vacío”	“Completa este campo.”

Nota. Ejemplos de redacción. Elaboración propia del autor, Damián Nicolalde (2025).

2.4.5. Robusto: que funcione “a prueba de futuro”

Semántica nativa primero: header, nav, main, section, article, aside, footer, figure/figcaption, time, dl/dt/dd, details/summary.

ARIA solo cuando falte semántica nativa o para desambiguar (ej.: varias nav con aria-label).

Valida HTML/CSS, prueba en más de un navegador y, si puedes, con un lector (NVDA en Windows, VoiceOver en macOS). Evita roles redundantes (role="navigation" en nav) o incorrectos (pueden confundir a las ayudas técnicas).

Enlace 2

- **Título del enlace relacionado:** WCAG 2.2 – Lista de pautas (visión general)
- **Descripción:** Panorama de las cuatro categorías (POUR) y objetivos AA más relevantes para formularios y estructura semántica.
- **Enlace:** <https://www.w3.org/WAI/standards-guidelines/wcag/>

2.4.6. Anti-patrones frecuentes

- **Quitar el foco por estética** → Personalízalo, no lo elimines.
- **Color como única señal** → Acompáñalo con texto/ícono.



- **Placeholders como etiquetas** → Usa label for.
- **Tablas para maquetar** → Usa Flexbox/Grid (Semana 3).
- **Componentes custom sin roles/teclas** → Usa nativos o implementa roles y teclado completos.
- **Modales que no devuelven el foco** → Devuélvelo al disparador.

Referencias citadas en la Clase 2.

- World Wide Web Consortium. (2018). *HTML 5.2: W3C Recommendation*. W3C.
- World Wide Web Consortium. (2018–2023). *WAI-ARIA Authoring Practices*. W3C.
- World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C.
- Mozilla Developer Network. (s. f.). *Web forms — Working with user data*. MDN Web Docs.
- Mozilla Developer Network. (s. f.). *HTML: Living Standard — Form validation*. MDN Web Docs.
- WHATWG. (s. f.). *HTML Living Standard* (secciones: forms, semantics).

Definición de términos

- **Landmark**: Región semántica de alto nivel (p. ej., header, nav, main, aside, footer) que facilita la navegación por teclado y lectores de pantalla, permitiendo “saltos” rápidos entre zonas de la página. (Término resaltado en azul dentro del texto principal; en Formulario 2 se pide subrayar/resaltar y definir 2 términos).
- **aria-describedby**: Atributo ARIA que **vincula** un control con uno o varios elementos que contienen ayudas y/o errores. Cuando existe, los lectores de pantalla leen **el control + sus ayudas/errores** como un solo bloque informativo, mejorando la comprensión. (Término resaltado; definición al final).

HTML/CSS

Mi primera página

Qué es HTML

- Es un **lenguaje de marcas** para formatear y estructurar un documento, que puede leerse en cualquier navegador.

<html> </html>

- Estandarizado en la norma ISO de **SGML** (Standard Generalized Markup Language).
- El **W3C** desarrolla especificaciones técnicas y directrices, de forma que se pueda asegurar una alta calidad técnica y editorial.

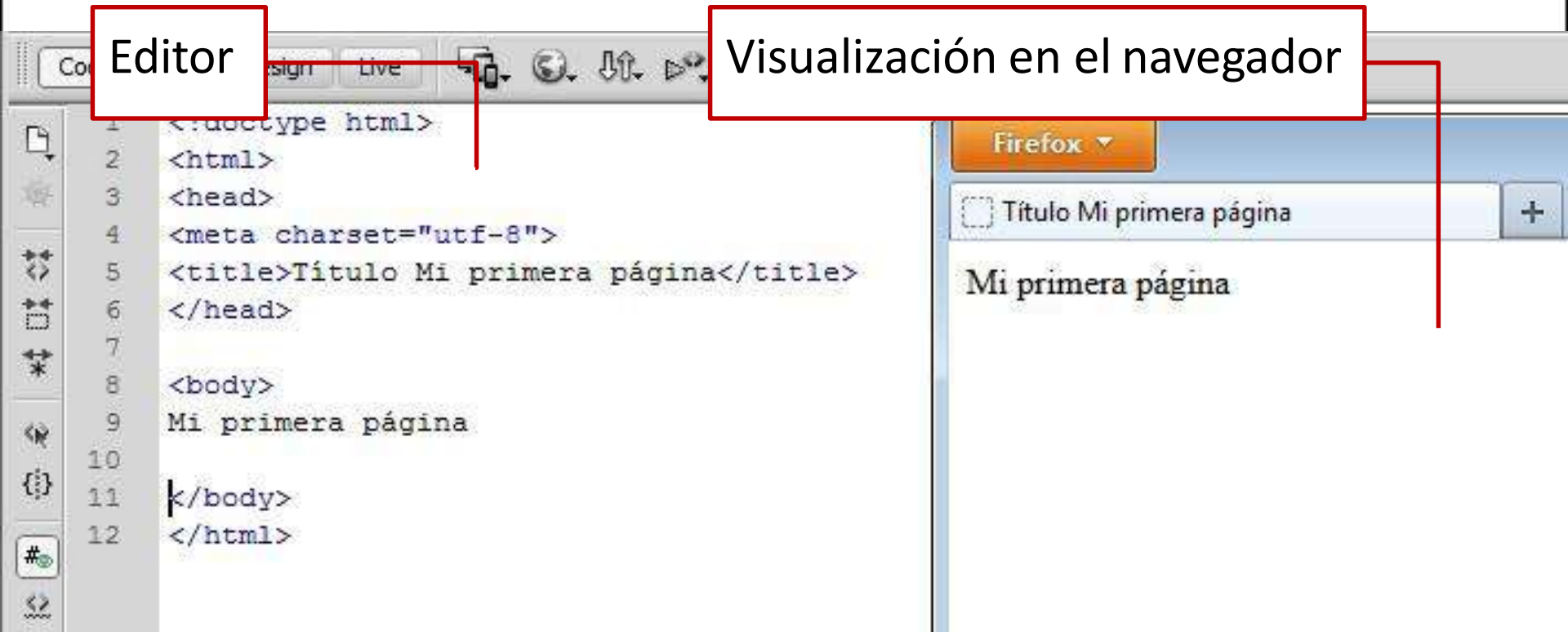
Estructura HTML

Un documento HTML tiene tres etiquetas que describen la estructura general de un documento y dan una información sencilla sobre él. `<html>`, `<head>` y `<body>`

Las etiquetas pueden escribirse tanto en mayúsculas como en minúsculas, pero se recomienda el uso de **minúsculas**: `<html>` o `<HTML>`, `<body>` o `<BODY>`

Editor

Visualización en el navegador



Lenguaje HTML

Las marcas de texto en html, se emplean para el **estructurado semántico del contenido**.

Los **textos** habitualmente están formados básicamente por títulos `<h1></h1>` párrafos `<p></p>` , resaltando en ellos algunas palabras en negrita `<strong></strong>` o en cursiva `<em></em>` .

Las **imágenes** se vinculan en una página HTML con la etiqueta **img**
``

con los atributos:

src = URL, **width** = ancho, **height** = alto, **alt** = texto alternativo de la imagen.

Las imágenes que se usan en la web son: GIF, JPG, PNG y las imágenes vectoriales en SVG

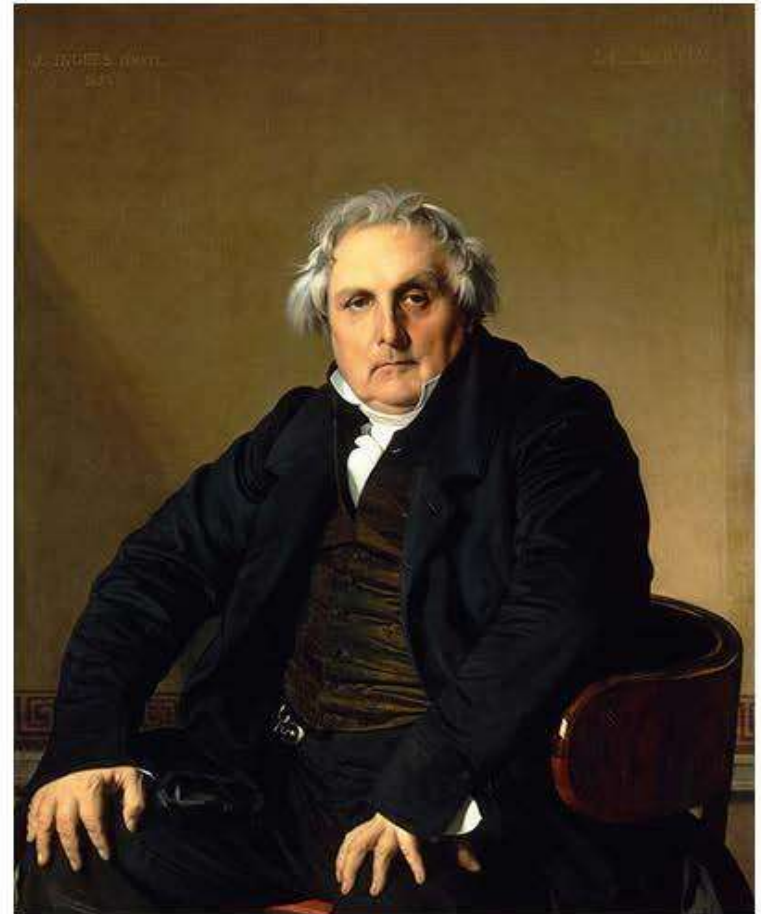
Párrafo e Imágenes HTML

Ejemplo

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Párrafo con imágenes</title>
6 </head>
7
8 <body>
9 <h1>Titular nivel 1 </h1>
10 <p>Párrafo de texto. <strong>negrita</strong> y <em>cursiva</em></p>
11 
13 </body>
14 </html>
```

Titular nivel 1

Párrafo de texto. **negrita** y *cursiva*



URL

URL (Uniform Resource Locator) es el localizador de un archivo en internet.

Una dirección absoluta en internet, que combina el nombre del servidor que proporciona la información, el directorio donde se encuentra, el nombre del fichero:

esquema: [//www.miDominio.com/ruta/miArchivo.html](http://www.miDominio.com/ruta/miArchivo.html)

un esquema define el tipo de servicio de internet:

http, es el protocolo usado para la transacción en la Web

`<a href="http://www.nytimes.com/"> The New York Times</a>`

https es la versión segura de http

`<a href="https://www.google.es/"> Google España</a>`

mailto, es el esquema que se emplea para enlazar a una dirección de correo.

`<a href="mailto:nombre@miDominio.com"> Contacto</a>`

ftp, es el esquema para la transferencia archivos.

`<a href="ftp://ftp.miDominio.com/ruta/miArchivo.zip">`

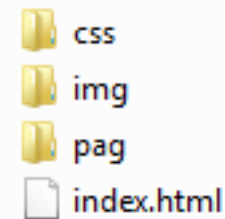
`Descarga por FTP</a>`

URL

Una URL **local** nos lleva a otra página o archivo del mismo sitio de Internet.

Ejemplo de un sitio web, de la raíz parten todas las subcarpetas.

css, img, pag



Cuando el archivos a enlazar está en la misma carpeta: la ruta es directamente el nombre del archivo de destino.

```
<a href="pag2.html">Ver siguiente</a>
```

El archivo de destino está en una carpeta de nivel inferior a su origen, en este caso deberemos escribir la ruta a partir de la carpeta donde está el archivo de origen:

```
<link href="css/estilo.css" rel="stylesheet" type="text/css"/>
```

```
<a href="pag/pag5.html">Ver ejemplo 5</a>
```

Si el archivo está en una carpeta de nivel superior, es necesario subir uno o varios niveles en la jerarquía de directorios, hasta llegar al nivel donde está el archivo. Subimos un nivel escribiendo (../); si subimos dos niveles (../..), hasta llegar al nivel donde esté el archivo.

```
<a href="../index.html">Página principal</a>
```

Para ir a una carpeta del mismo nivel, debemos subir al directorio raíz mediante ../ y desde ahí a la carpeta buscada.

```

```

e/s/d/
madrid

CSS

El **HTML**, controla el aspecto gráfico del documento mediante hojas de estilo CSS (*cascading style sheets*), el **CSS**, indica al navegador como se deben visualizar los elementos de un documento HTML.

Así se consigue separar el aspecto del contenido.

El estilo CSS, se puede añadir mediante un bloque marcado como **<style>** en la **cabecera (<head>)** y aplica el estilo a los elementos de la página.

Para aplicar un estilo de presentación, el **selector** de elemento puede ser una etiqueta HTML:

p, h1, ul, li

El estilo afecta a todo el contenido incluido dentro de esa etiqueta **<p> </p>**, **<h1></h1>**, ****, ****.

CSS

El estilo se define entre llaves "{" y "}"

H1	{	color	:	green	;	...	}
Selector	propiedad		valor				
	declaración				declaración; ...		
	bloque de declaración						
norma de estilo							

CodeSplitDesignLive

Title: Marcas de texto con CSS

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Marcas de texto con CSS</title>
6 <style type="text/css">
7 body { font-family: Arial, Helvetica, sans-serif; }
8 h1 { font-size: 1.4em; background-color: #CCC; }
9 p { background-color: #CCC; }
10 em { color: orange; background-color: #E8E8E8; }
11 strong { color: red; background-color: #E8E8E8; }
12 </style>
13 </head>
14 <body>
15 <h1>Titular nivel 1</h1>
16 <p>En este párrafo hay rexta marcado
17 como <em>importante</em> y otro texto
18 marcado como <strong>muy importante.</strong></p>
19 <p>Este es el segundo párrafo.</p>
20 </body>
21 </html>
```

Firefox

Marcas de texto con CSS

Titular nivel 1

En este párrafo hay rexta marcado como *importante* y otro texto marcado como **muy importante.**

Este es el segundo párrafo.

Editores HTML

Antes de comenzar a trabajar con un editor específico, es recomendable conocer el código.

Podemos usar programas que trabajen con **texto plano**, sin añadir sus propias marcas de edición.

Los documentos HTML deben tener la **extensión** html o htm.

Usar un editor **wysiwyg** como el Adobe *Dreamweaver* o el *BlueGriffon* (<http://www.bluegriffon.org/>) o un editor con ayudas visuales como el *sublime* (<http://www.sublimetext.com/>) o el *Brackets* (<http://brackets.io/>) o el visual Studio Code nos facilitará las cosas.

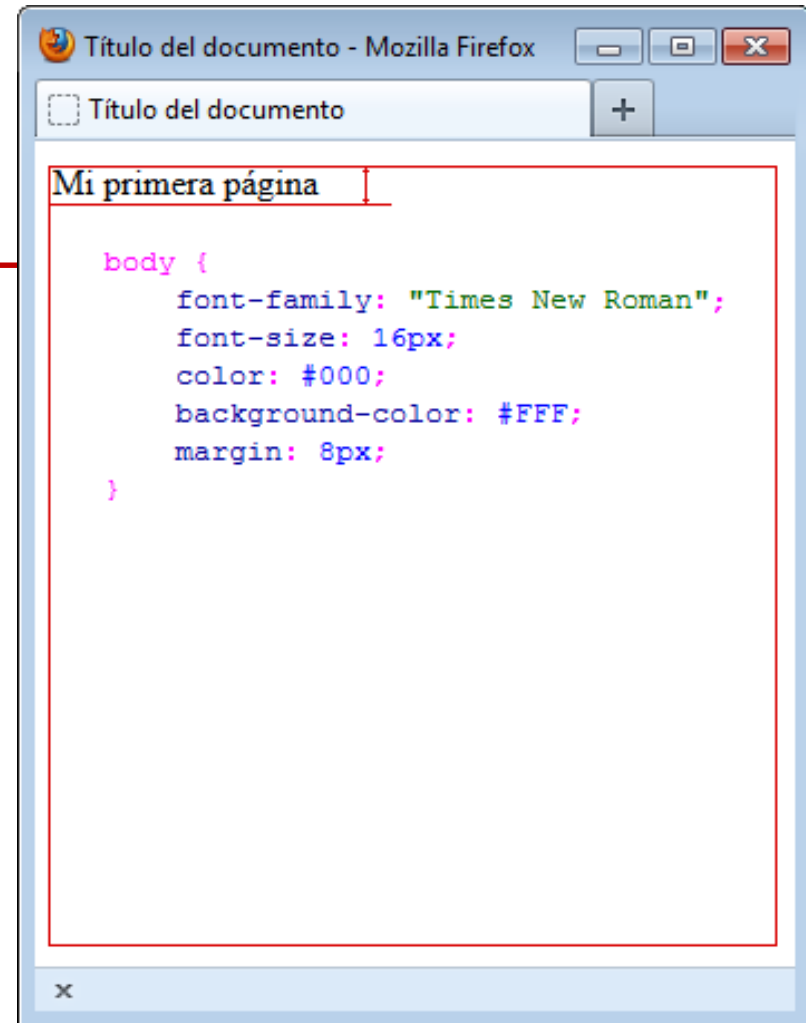
Ver comparativa: http://en.wikipedia.org/wiki/Comparison_of_HTML_editors

CSS

Cada etiqueta HTML tiene unos valores de CSS por defecto, que pueden variar según el navegador, que tendremos que modificar con nuestros estilos personales.

Visualización valores por defecto CSS en el navegador

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Título del documento</title>
6 </head>
7
8 <body>
9 Mi primera página
10 </body>
11 </html>
12
```



HTML

Ejemplo valores CSS por defecto del navegador, de: **h1**, **p**, **strong**, **em**.

```
Code Split Design Live
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Marcas de texto</title>
6 </head>
7
8 <body>
9 <h1>Titular nivel 1</h1>
10 <p>En este párrafo hay rexta marcado
11 como <em>importante</em> y otro texto
12 marcado como <strong>muy importante.</strong></p>
13 <p>Este es el segundo párrafo.</p>
14 </body>
15 </html>
16
```

```
h1 {
  display: block;
  font-size:
  2em;
  margin-before:
  0.67em; margin-after:
  0.67em; margin-start:
  0;
  margin-end: 0;
  font-weight: bold;
```

```
p {
  display: block;
  margin-before:
  1em; margin-after:
  1em; margin-start:
  0;
  margin-end: 0; }
```

```
strong {
  font-weight: bold; }
```

```
em {
  font-style: italic; }
```

Titular nivel 1

En este párrafo hay rexta marcado como *importante* y otro texto marcado como **muy importante**.

Este es el segundo párrafo.

CSS

Más información valores por defecto:

W3C:

<http://dev.w3.org/html5/markup/elements.html>

Internet Explorer:

<http://www.iecss.com/>

Firefox:

<http://mxr.mozilla.org/mozilla-central/source/layout/style/html.css>

Chrome, Safari:

<http://trac.webkit.org/browser/trunk/Source/WebCore/css/html.css>

Opera

<http://www.iecss.com/opera-10.51.css>

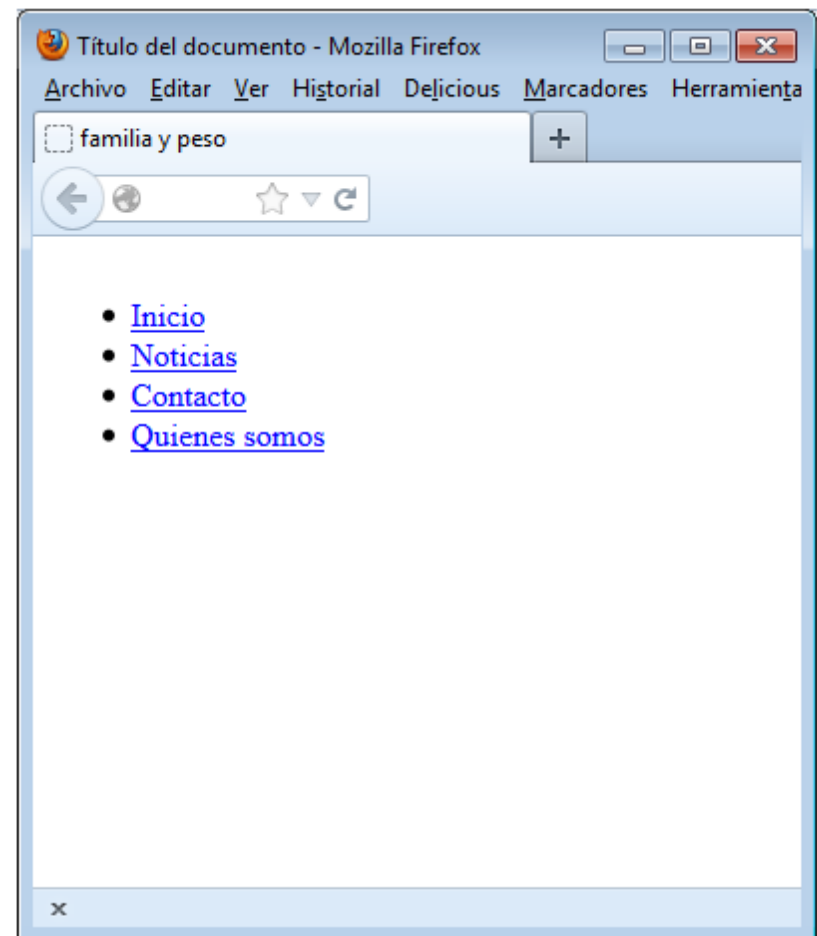
Navegación HTML

La etiqueta `<a>` se emplea para definir un enlace a otra página. Un **menú** es una lista de enlaces. Se recomienda usar las marcas de lista `<ul>` `<li>` y el enlace `<a>`.

```
<!doctype html>
<html>
<head>
<meta charset="utf-8">
<title>Menú</title>
</head>

<body>
<nav>
<ul>
<li><a href="index.html">Inicio</a></li>
<li><a href="news.html">Noticias</a></li>
<li><a href="contacto.html">Contacto</a></li>
<li><a href="quienes.html">Quienes somos</a></li>
</ul>
</nav>

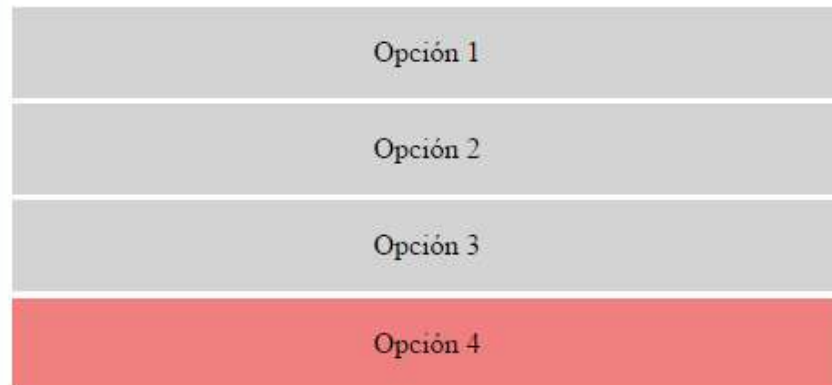
</body>
</html>
```



Navegación HTML

Un menú vertical y horizontal con FLEX, tendría este formato.

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>MENÚ HORIZONTAL FLEX</title>
6 <style type="text/css">
7 ul {
8     list-style-type:none;
9     margin:0;
10    padding:0;
11    display:flex;
12    flex-direction: row; flex-direction: column;
13 }
14 li { flex-grow: 1; }
15 a {
16     display:block;
17     padding:1em;
18     background-color:LightGray ;
19     text-align:center;
20     margin-right:0.2em; margin-bottom:0.2em;
21     text-decoration:none;
22     color:#000000;
23 }
24 a:hover {background-color:LightCoral; }
25 </style>
26 </head>
27 <body>
28 <nav>
29     <ul>
30         <li><a href="#">Opción 1</a></li>
31         <li><a href="#">Opción 2</a></li>
32         <li><a href="#">Opción 3</a></li>
33         <li><a href="#">Opción 4</a></li>
34     </ul>
35 </nav>
36 </body>
```



Recursos CSS

MDN Usando las cajas flexibles CSS

https://developer.mozilla.org/es/docs/Web/Guide/CSS/Cajas_flexibles

CSS-TRICKS A Complete Guide to Flexbox

<http://css-tricks.com/snippets/css/a-guide-to-flexbox/>

HTML/CSS

Mi primera página

HTML/CSS

Estructura HTML

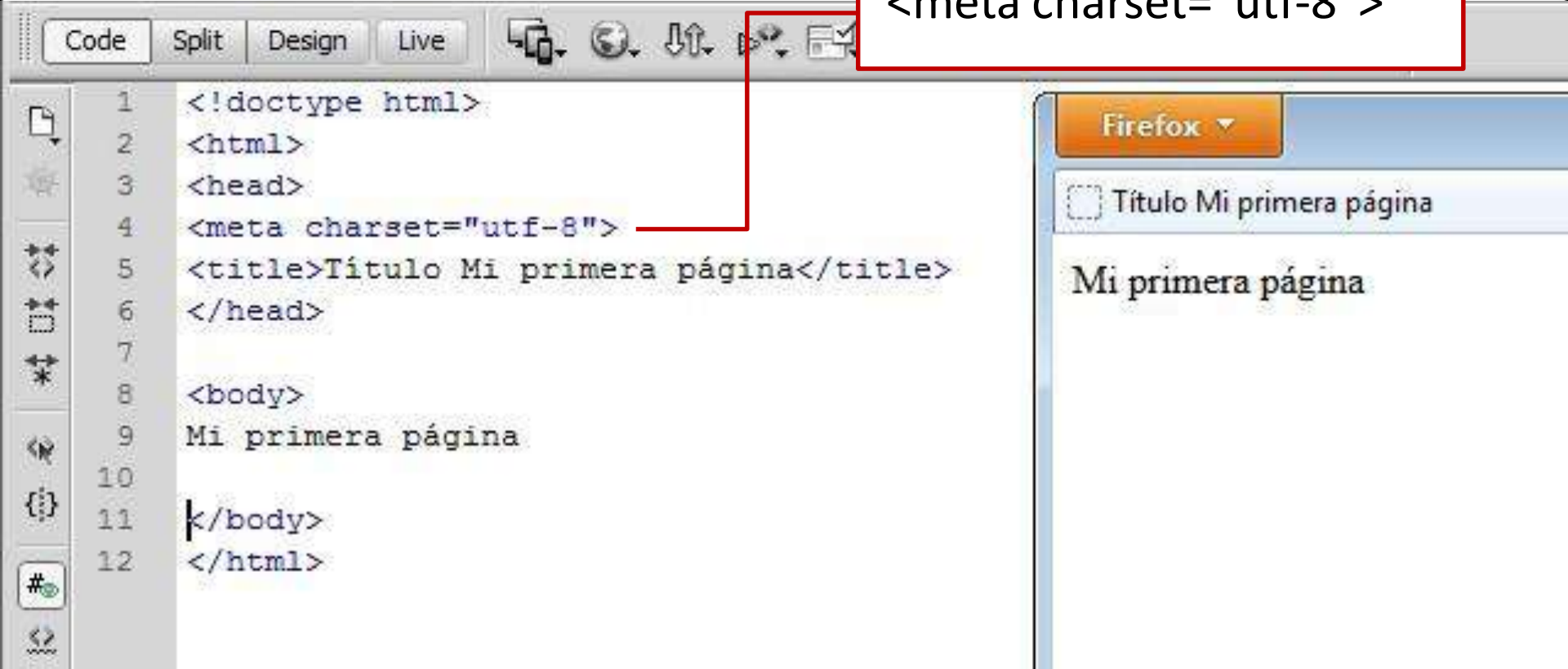
Estructura HTML

La etiqueta **meta** define varios tipos de metadatos.

El atributo **charset** especifica la codificación usada en nuestra página.

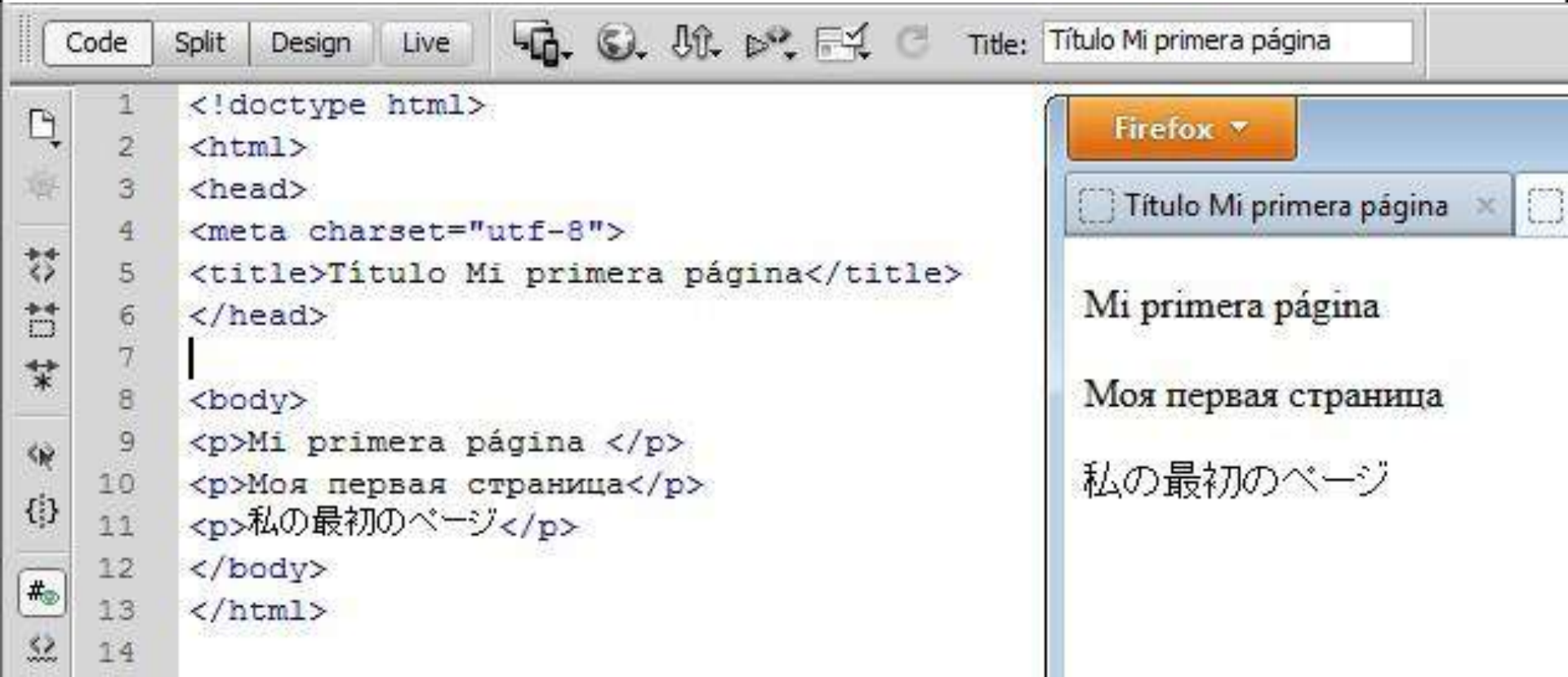
```
<meta charset="character_set">
```

```
<meta charset="utf-8">
```



Estructura HTML

Charset utf-8 es la **Unicode Transformation Format 8-bit** representa el código de caracteres UNICODE. Es compatible con ASCII. Permite visualizar los caracteres de todos los idioma.



Estructura HTML

Caracteres especiales: para incluir en el texto caracteres que son propias del lenguaje HTML es necesario usar su anotación codificada:

<http://dev.w3.org/html5/html-author/charref>

	Espacio en blanco (non-breaking space)	
<	Menor que	<
>	Mayor que	>
&	Ampersand	&
"	Comillas	"
'	apóstrofo	'

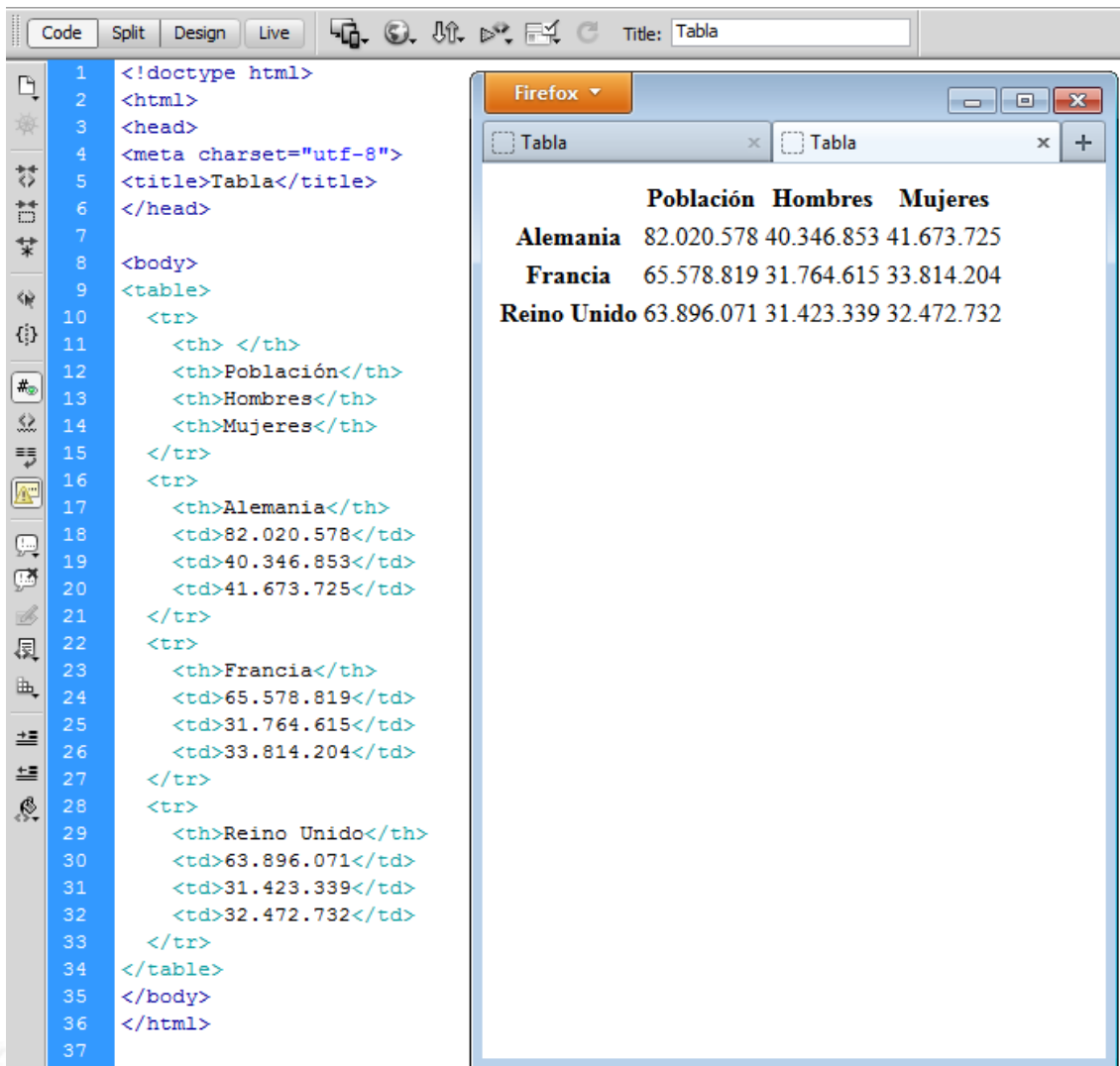
The screenshot shows a web editor interface with a code editor on the left and a Firefox preview window on the right. The code editor displays the following HTML code:

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>codificación de caracteres</title>
6 </head>
7
8 <body>
9 <p>Si necesitamos escribir caracteres que utiliza
10 el lenguaje HTML para definir sus etiquetas
11 (&lt; &gt; &amp; &quot; &nbsp; &apos;)
12 no se pueden utilizar libremente.
13
14 </p>
15 </body>
16 </html>
```

The Firefox preview window shows the rendered page with the title "codificación de caracteres" and the text: "Si necesitamos escribir caracteres que utiliza el lenguaje HTML para definir sus etiquetas (< > & " ') no se pueden utilizar libremente."

Tabla

Las tablas se definen con las etiquetas `<table>`, `<tr>` para cada fila y `<td>` para cada columna `<th>` indica que una celda es la cabecera.



The image shows a code editor on the left and a browser preview on the right. The code editor displays the following HTML code:

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Tabla</title>
6 </head>
7
8 <body>
9 <table>
10   <tr>
11     <th> </th>
12     <th>Población</th>
13     <th>Hombres</th>
14     <th>Mujeres</th>
15   </tr>
16   <tr>
17     <th>Alemania</th>
18     <td>82.020.578</td>
19     <td>40.346.853</td>
20     <td>41.673.725</td>
21   </tr>
22   <tr>
23     <th>Francia</th>
24     <td>65.578.819</td>
25     <td>31.764.615</td>
26     <td>33.814.204</td>
27   </tr>
28   <tr>
29     <th>Reino Unido</th>
30     <td>63.896.071</td>
31     <td>31.423.339</td>
32     <td>32.472.732</td>
33   </tr>
34 </table>
35 </body>
36 </html>
37
```

The browser preview (Firefox) shows the rendered table:

	Población	Hombres	Mujeres
Alemania	82.020.578	40.346.853	41.673.725
Francia	65.578.819	31.764.615	33.814.204
Reino Unido	63.896.071	31.423.339	32.472.732

Tabla CSS

Código Dividir Diseño En vivo Título: Tabla

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Tabla</title>
6 <style type="text/css">
7 table {
8     padding: 0px;
9     margin: 0px;
10    border-spacing: 0px;
11    font-family: Arial, Helvetica, sans-serif;
12    font-size: 0.8em;
13    border-top-width: 1px;
14    border-top-style: solid;
15    border-top-color: #333;
16 }
17 th, td {
18     padding: 0.5em;
19     border-bottom-width: 1px;
20     border-bottom-style: solid;
21     border-bottom-color: #333;
22 }
23 th {
24     background-color: #E2E2E2;
25 }
26 td {
27     background-color: #F0F0F0;
28 }
```

Firefox

Tabla

	Población	Hombres	Mujeres
Alemania	82.020.578	40.346.853	41.673.725
Francia	65.578.819	31.764.615	33.814.204
Reino Unido	63.896.071	31.423.339	32.472.732

Recursos HTML

MDN: Lista de Elementos HTML5

https://developer.mozilla.org/es/docs/HTML/HTML5/HTML5_lista_elementos

W3C: HTML5 elements

<http://dev.w3.org/html5/markup/elements.html>

HTML5-Visual Cheat Sheet

<https://docs.google.com/viewer?a=v&pid=sites&srcid=bGFuZG1hcmtzY2hvb2wuY2JnfG1zLWNpcmFzLWNSYXNzLXdIYnNpdGV8Z3g6M2I3ZjY1NzJlMjlmNDA3Ng>

El estilo CSS se puede añadir escribiendo directamente las propiedades CSS en la **línea**, es el método más sencillo, se añade un atributo **style** en el elemento concreto dentro de la página. No se pueden reutilizar en otros elementos que comparten las mismas propiedades.

Se escriben las propiedades del estilo en las marcas del HTML como "*nombre: valor*" separadas por punto y coma.

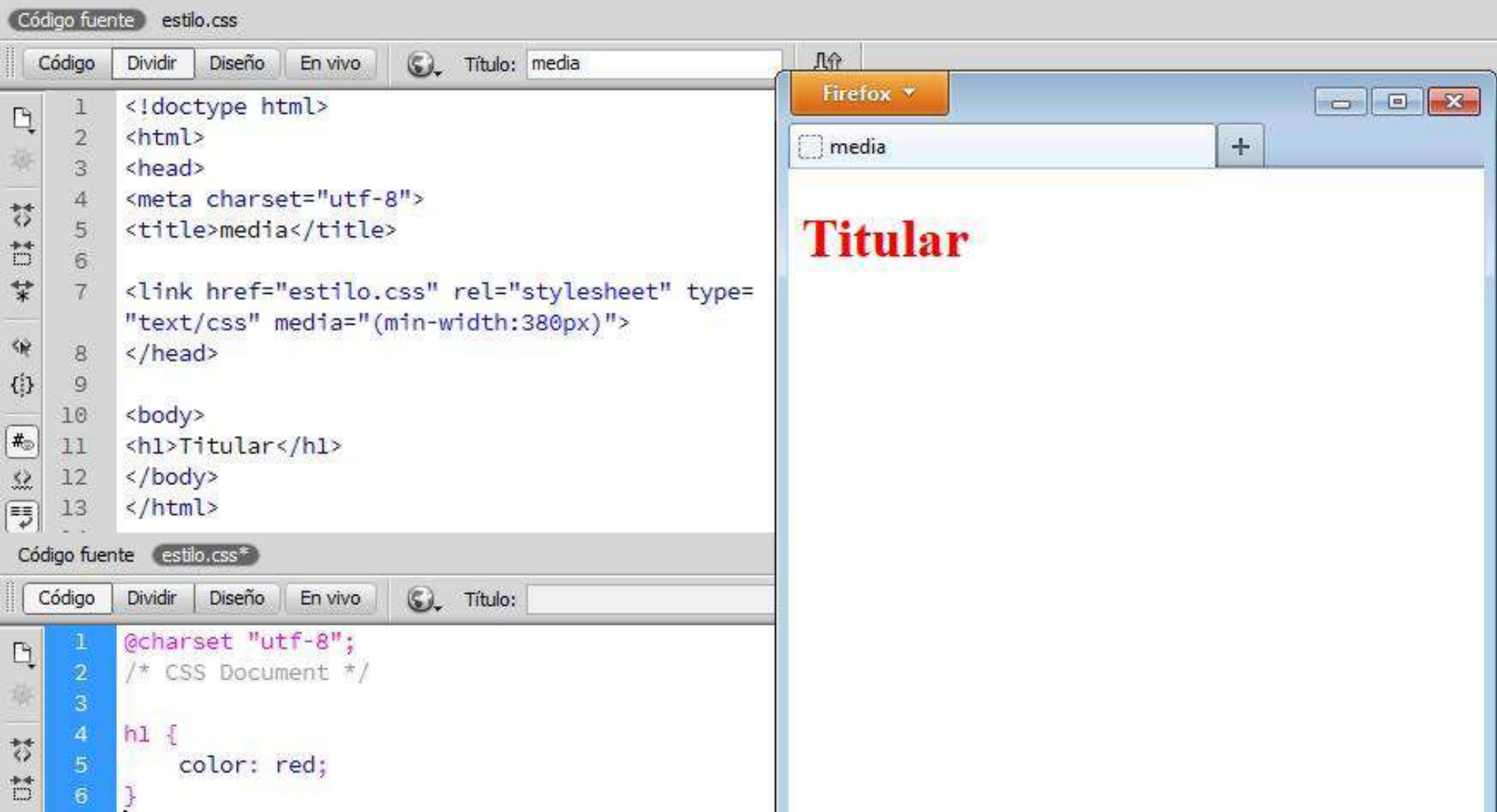
```
<body style="color: red; ">
```

```
<p style="font-size: 16px; color: blue; font-family: Arial,  
Helvetica, sans-serif;">
```

```
Mi primera página</p>
```

CSS

Lo más aconsejable para mantener la separación entre contenido y presentación, es una hoja de estilo CSS separada del HTML, que se importa con un elemento **<link>** en la cabecera.



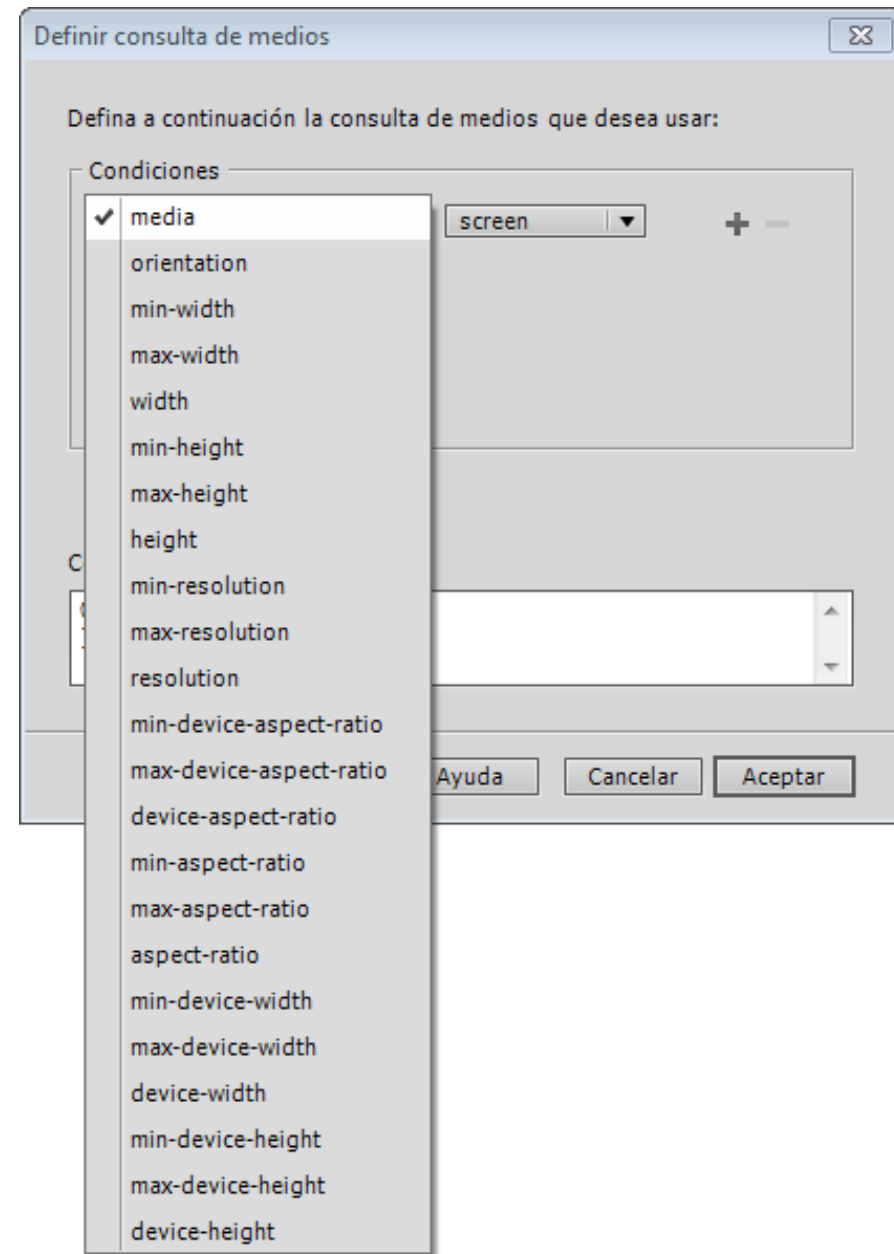
CSS

CSS Permite adaptar la presentación a PCs, móviles, tabletas o impresoras con el atributo **media** que activará el estilo específico de cada dispositivo.

```
<link rel="stylesheet"
type="text/css" media="screen"
href="sans-serif.css">
```

```
<link href="estilo.css"
rel="stylesheet" type="text/css"
media="(min-width:380px)">
```

<http://www.w3.org/TR/css3-mediaqueries/>
https://developer.mozilla.org/es/docs/CSS/Media_queries



Estructura HTML

Los elementos html pueden visualizarse por defecto como bloque o como en línea.

Los elementos bloques **block** ocupan todo el ancho de la página y fuerzan a una nueva línea antes y después.

`<article> <aside> <canvas> <div> <footer> <h1>, <h2>, <h3>, <h4>, <h5>, <h6> <header> <p> <pre> <section> <ul>...`

https://developer.mozilla.org/es/docs/Web/HTML/Block-level_elements

Los elementos en línea **inline** sólo ocupan el ancho necesario y no fuerzan nuevas líneas.

`<em> <a> <br> <img> <span> <button> <input> <select> <textarea>...`

https://developer.mozilla.org/es/docs/Web/HTML/Elementos_en_l%C3%ADnea

Estructura HTML

DIV

El elemento **<div>** es un elemento de bloque que se emplea para contener en su interior otros elementos **HTML**. No tiene ningún significado, es semánticamente neutro.

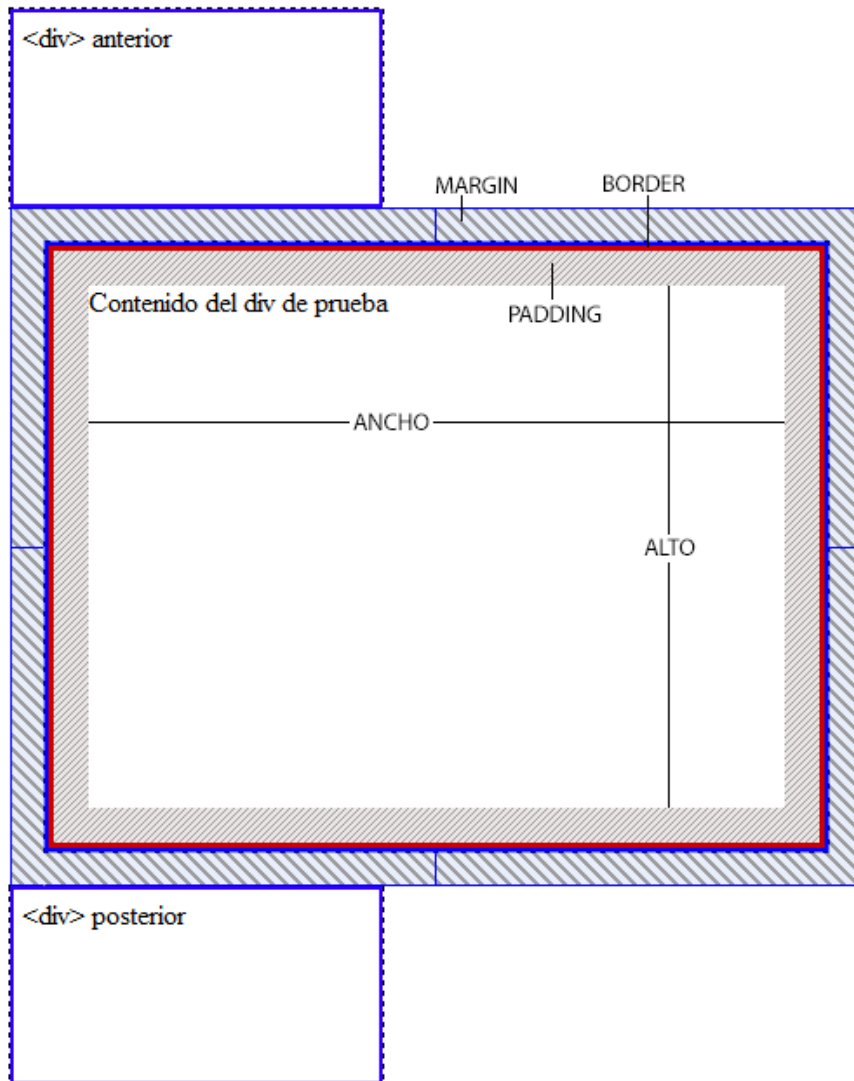
<div id="caja" class="card"> </div>

Atributo **ID**: identifica unívocamente un elemento HTML en una página. Un elemento HTML solo debe tener un atributo **ID** y este debe ser único en la página. El selector CSS **#caja** se refiere al elemento con atributo **id="caja"**, como en **#caja {color:blue}**

Atributo **CLASS**: define una clase de elementos HTML. Un elemento puede tener varios atributos **CLASS**. El selector CSS **.card** se refiere a todos los elemento de la clase **card** (que llevan el atributo **class="card"**) , como en **.card {color:red}**

div CSS

Padding, margin y border

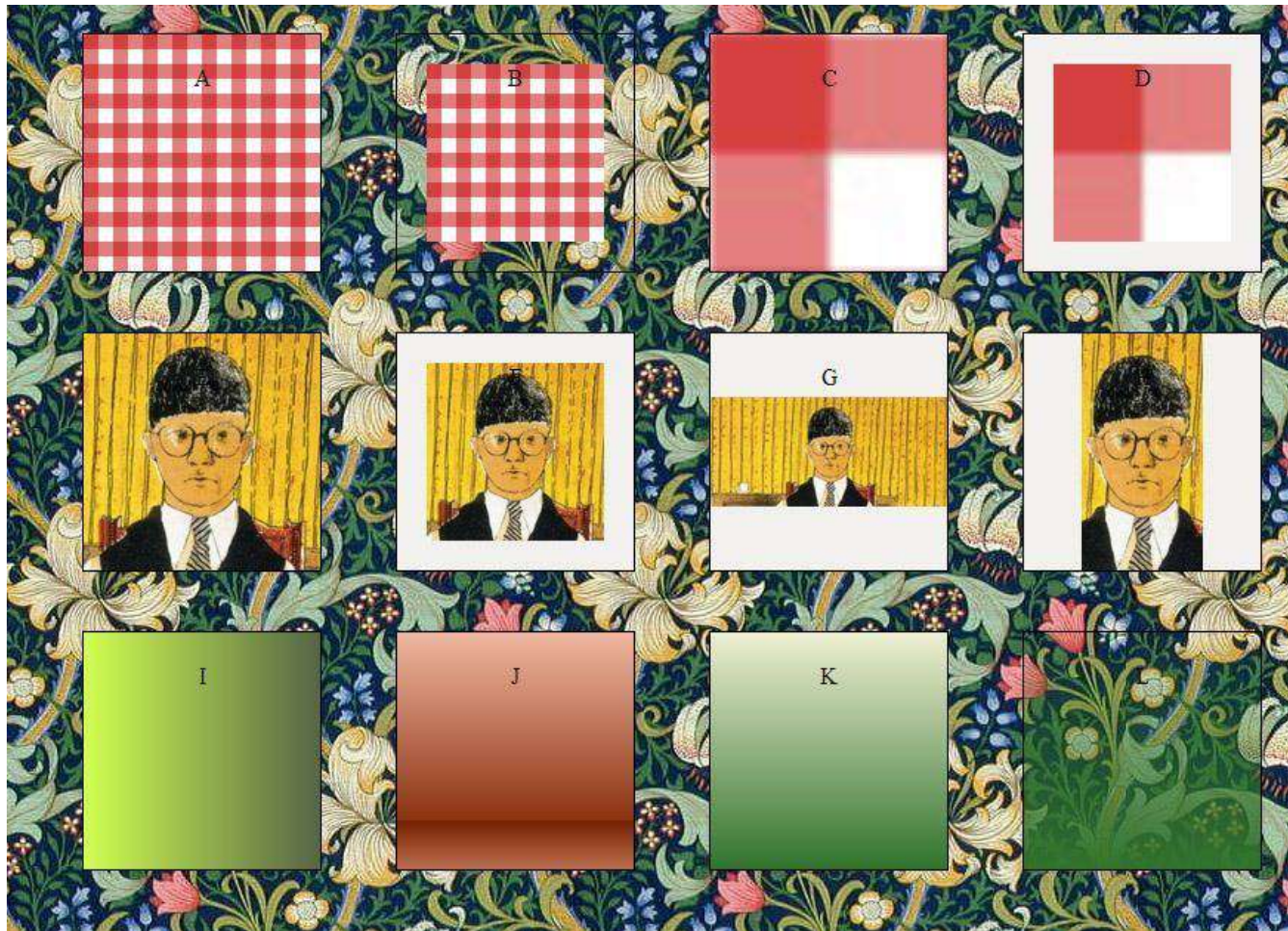


```
<!doctype html>
<html>
<head>
<meta charset="utf-8">
<title>div</title>
<style type="text/css">
.muestra {
    height: 300px;
    width: 400px;
    border: 5px solid #C00;
    margin: 20px;
    padding: 20px;
}
.anterior, .posterior {
    height: 100px;
    width: 200px;
    border: 2px solid #30F;
    padding: 5px;
}
</style>
</head>

<body>
<div class="anterior">&lt;div&gt; anterior</div>
<div class="muestra">Contenido del div de prueba</div>
<div class="posterior">&lt;div&gt; posterior</div>
</body>
</html>
```

Estructura CSS

Background



 `back_mantel.png`

 `dh.png`

 `dh_2.png`

 `dh_3.png`

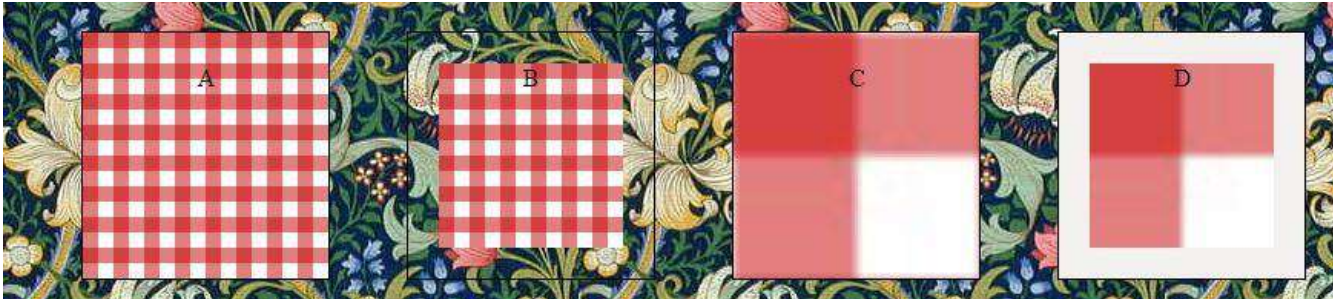
 `ac.jpg`

<http://www.css3files.com/background/>

e/s/d/
madrid

Estructura CSS

Background

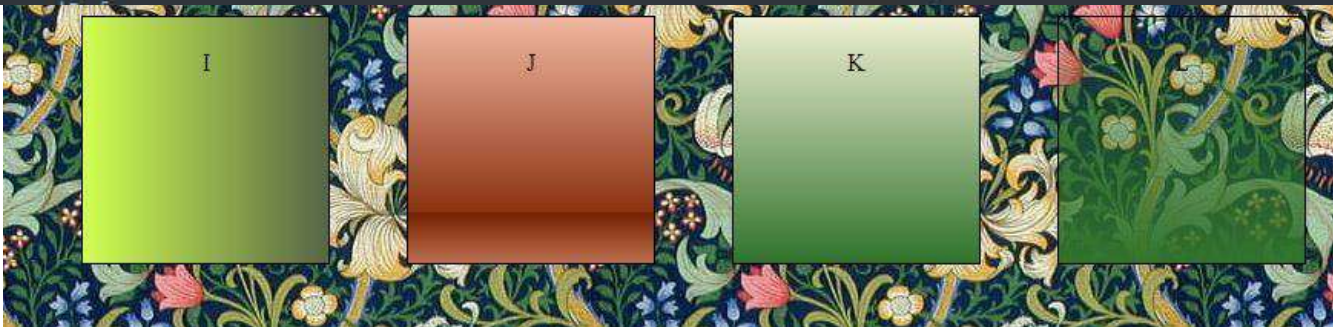


 `back_mantel.png`

 `dh.png`

```
19 body {  
20     background-image: url(images/ac.jpg);  
21 }  
22 .modA {  
23     background-image: url(images/back_mantel.jpg);  
24 }  
25 .modB {  
26     background-image: url(images/back_mantel.jpg);  
27     background-clip: content-box;  
28 }
```

```
29 .modC {  
30     background-image: url(images/back_mantel.jpg);  
31     background-size:100% 100%;  
32 }  
33 .modD {  
34     background-image: url(images/back_mantel.jpg);  
35     background-repeat:no-repeat;  
36     background-size:100% 100%;  
37     background-origin:content-box;  
38 }
```



 `dh_3.png`

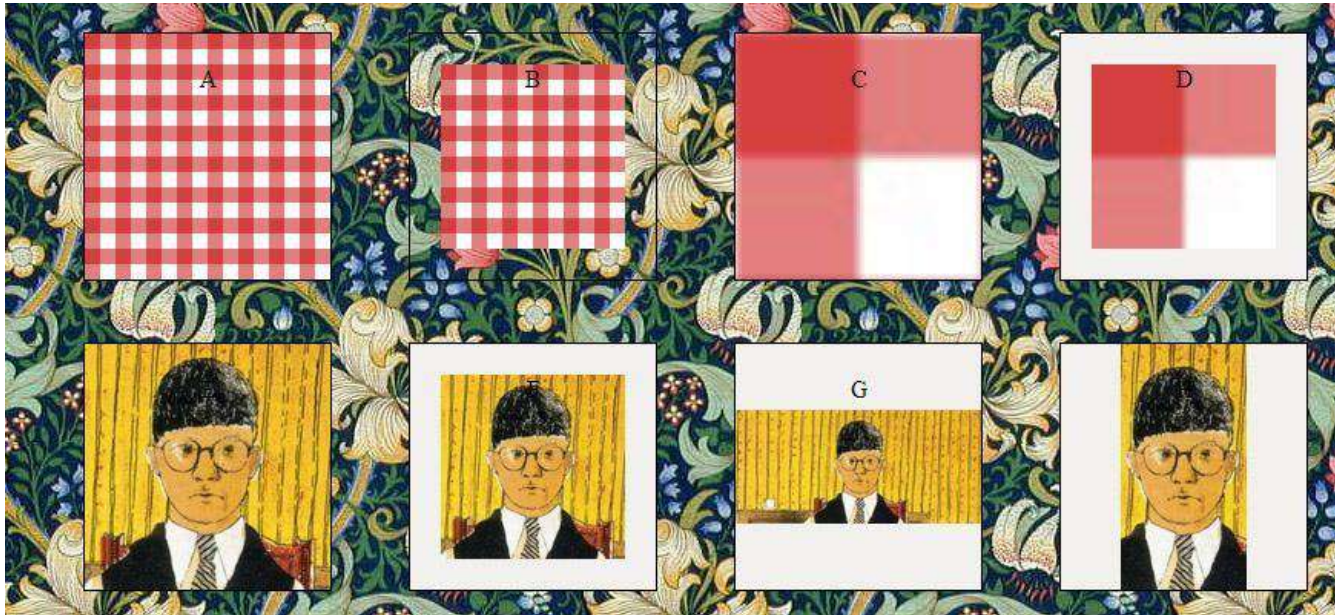
 `ac.jpg`

<http://www.css3files.com/background/>

e/s/d/
madrid

Estructura CSS

Background



 back_mantel.png

 dh.png

 dh_2.png



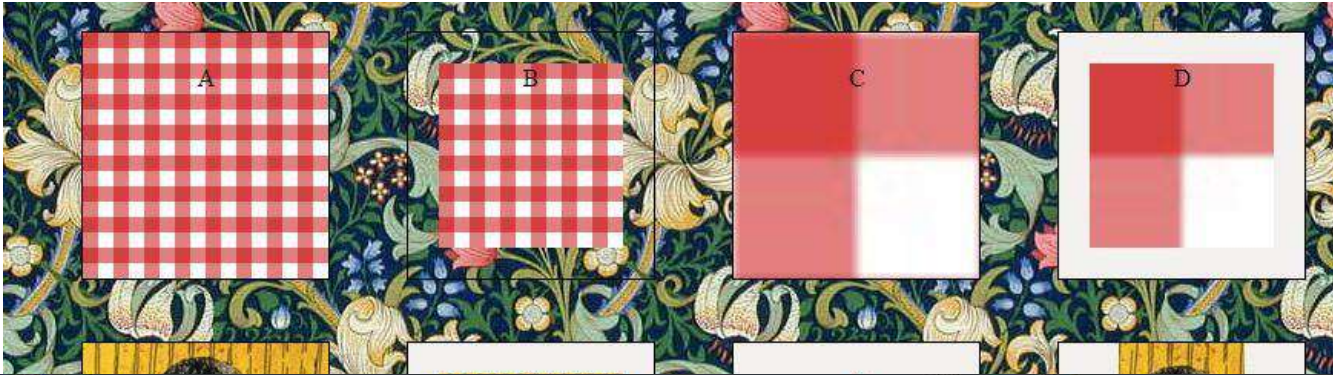
```
39 .modE {  
40     background-image: url(images/dh.jpg);  
41     background-repeat: no-repeat;  
42     background-size: cover;  
43 }  
44 .modF {  
45     background-image: url(images/dh.jpg);  
46     background-repeat: no-repeat;  
47     background-size: cover;  
48     background-origin: content-box;  
49 }
```

```
50 .modG {  
51     background-image: url(images/dh_2.jpg);  
52     background-repeat: no-repeat;  
53     background-size: contain;  
54     background-position: center;  
55 }  
56 .modH {  
57     background-image: url(images/dh_3.jpg);  
58     background-repeat: no-repeat;  
59     background-size: contain;  
60     background-position: center;  
61 }
```

<http://www.css3files.com/background/>

Estructura CSS

Background



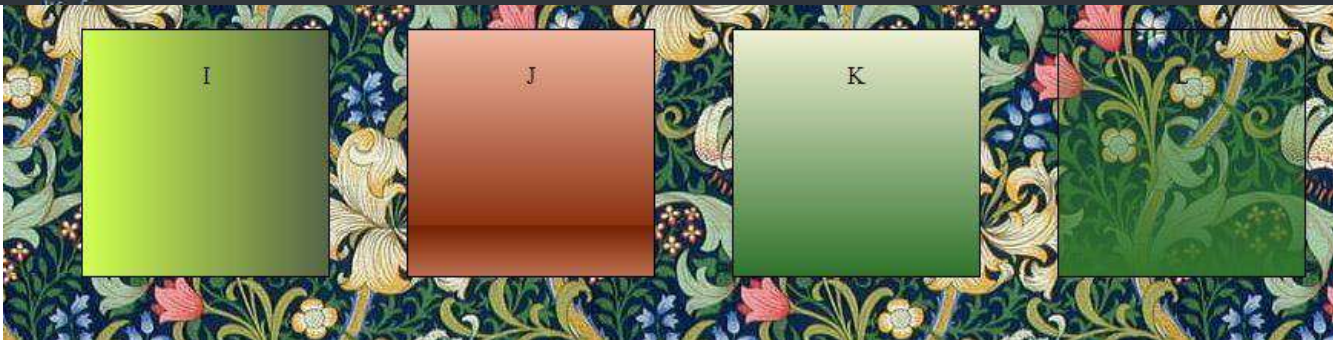
 `back_mantel.png`

 `dh.png`



```
62 .modI {  
63     background: linear-gradient(to right, #d2ff52 0%, #566647 100%);  
64 }  
65 .modJ {  
66     background: linear-gradient(to bottom, rgba(240,183,161,1)  
0%,rgba(140,51,16,1) 80%,rgba(117,34,1,1) 51%,rgba(191,110,78,1)  
100%);  
67 }  
68
```

```
68 .modK {  
69     background: linear-gradient(to top, rgba(50,116,45,1) 0%,  
70     rgba(239,239,210,1) 100%);  
71 }  
72 .modL {  
73     background: linear-gradient(to top, rgba(50,116,45,1) 0%,  
74     rgba(239,239,210,1) 100%);  
75 }
```



 `dh_3.png`



`ac.jpg`

<http://www.css3files.com/background/>

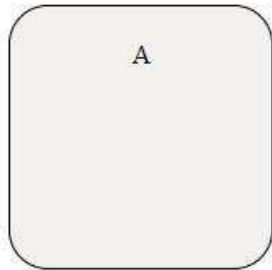
e/s/d/
madrid



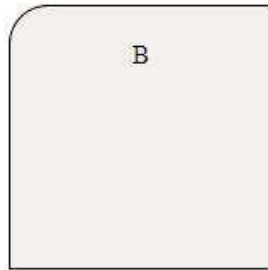
```
div {  
  background: url(texto/f01.png) left bottom  
             repeat-x,  
             url(texto/f02.png) left bottom no-repeat,  
             url(texto/f03.png) left top repeat-x;  
  background-color: #a4d3ff;  
}
```

Estructura CSS

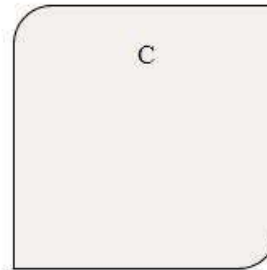
CSS3 permite añadir **bordes y bordes redondeados**



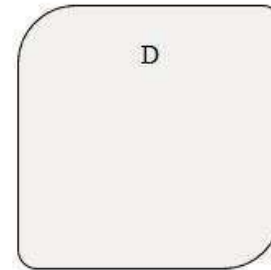
`border-radius: 24px;`



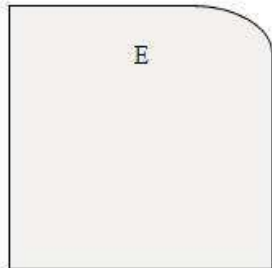
`border-top-left-radius: 24px;`



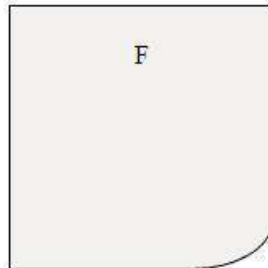
`border-radius: 24px 0;`



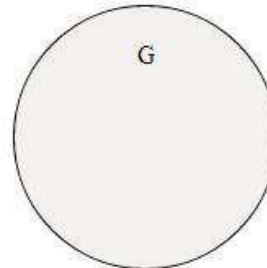
`border-radius: 36px 12px;`



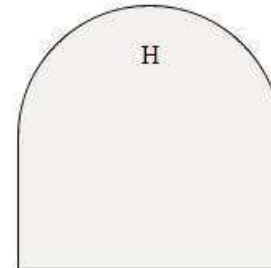
`border-top-right-radius: 50px 30px;`



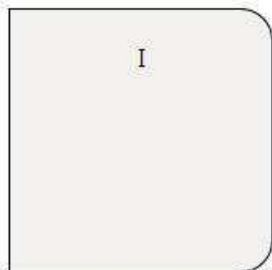
`border-bottom-right-radius: 50px 30px;`



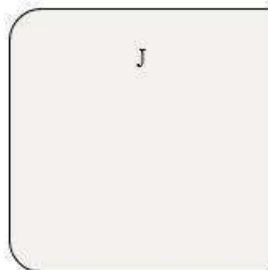
`border-radius: 50%;`



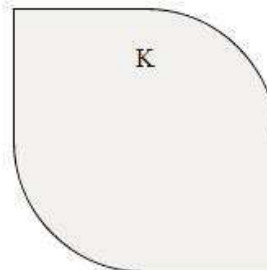
`border-radius: 50% 50% 0 0;`



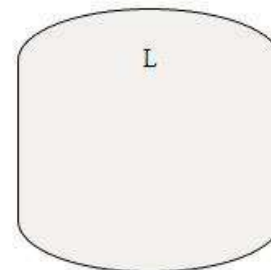
`border-radius: 0 20px 20px 0;`



`border-radius: 20px 0 0 20px;`



`border-radius: 0 50% 0 50%;`



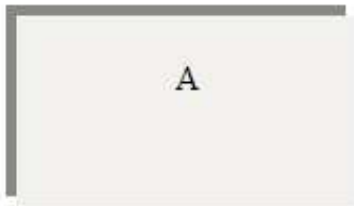
`border-top-left-radius: 50% 20%;`
`border-top-right-radius: 50% 20%;`
`border-bottom-left-radius: 50% 20%;`
`border-bottom-right-radius: 50% 20%;`

<http://www.cssmatic.com/es/border-radius>

Estructura CSS

CSS3 permite añadir **sombras**

`box-shadow: none | offset-x offset-y blur-radius spread-radius color | inset | initial | inherit;`



`box-shadow: -5px -5px #888;`



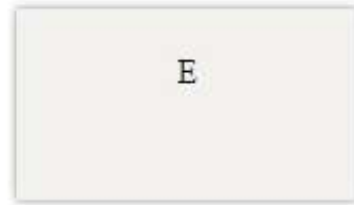
`box-shadow: -5px -5px 5px #888;`



`box-shadow: -5px -5px 0 5px #888;`



`box-shadow: -5px -5px 5px 5px #888;`



`box-shadow: 0 0 5px #888;`



`box-shadow: 0 0 5px 5px #888;`



`box-shadow: inset -5px -5px #888;`



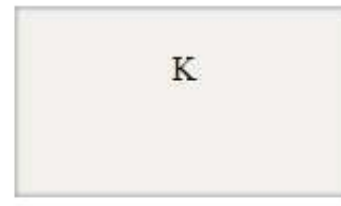
`box-shadow: inset -5px -5px 5px #888;`



`box-shadow:
inset -5px -5px 0 5px #888;`



`box-shadow:
inset -5px -5px 5px 5px #888;`



`box-shadow: inner 0 0 5px #888;`



`box-shadow: inset 0 0 5px 5px #888;`

<http://dev.w3.org/csswg/css-backgrounds/>

Estructura HTML

En **HTML5**, los bloques **div** son sustituidos por bloques semánticos que muestran significado por si mismos.

<header>

<nav>

<section>

<article>

Cabecera

THE TIMES

332,146
SUBSCRIBERS

JOIN NOW
FROM JUST
£2 A WEEK

Navegación

Business | Money | Sport | Life | Arts | Puzzles | Papers

Friday, August 2 | London | Max 26°C

Latest News

Ten charged with grooming offences against teenage girls

Contenido

Home Office investigated over stop-and-search checks

Sadie Gray
Last updated at 3:49PM, August 2 2013

The Home Office is to be investigated by the UK's equality watchdog after random spot-checks for illegal immigrants were reportedly aimed only at people from ethnic minorities. The stop-and-search style checks, performed yesterday outside London railway stations, contributed to a total of 139 arrests in a series of immigration raids around the country. The Equality and Human Rights Commission will investigate the spot-checks and a controversial advertisement campaign in which posters bearing the words "illegal immigrants – go home or face arrest" are displayed on the side of vans. A spokesman said this afternoon: "The commission is writing today to the Home Office about these reported operations, confirming that it will be



Ashes: Aussies get Root and Bresnan

Minimum 30 years jail for Daniel Pelka killer couple

China 'building aircraft carrier'

Satellite images of a shipyard near Shanghai have persuaded leading Western military experts that China may have begun building its first indigenous aircraft

Destacados

Fracking: drilling starts in Sussex as protests continue

Berlusconi: party relaunch despite looming house arrest

New RBS chief: on £1m-a-year - but he won't get a bonus

Zimbabwe: African observers give mixed verdict on election

Hotel deaths: cyanide suicide theory for Edinburgh couple

Terror alert: US to shut Mid East embassies



Follow reaction to The #Ashes with our live dashboard

Free to non-subscribers

THE TIMES

Stay connected by tablet, phone and web

The Digital Pack

Pie

13 | Version 4.6.0.0(86651)

My Account | RSS | Classified advertising | Display advertising | The Times Whisky Club | Encounters Dating | Sunday Times Wine Club | Privacy & Cookie Policy | Syndication | Site Map | FAQ | Terms & Conditions | Contact us | iPhone | Android smartphone | Blackberry | Windows phone | Android tablet | Kindle | Kindle Fire | Place an announcement in The Times | The Times Bookshop

<aside>

<footer>

e/s/d/
madrid

Estructura HTML

Ejemplo de estructura de una página realizada con los bloques semánticos **HTML5**

The image shows a web development environment with a code editor on the left and a browser preview on the right. The code editor displays the following HTML5 structure:

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <link href="html5_estilo.css" rel="stylesheet" type="text/css">
6 <title>estructura HTML5</title>
7 </head>
8 <body>
9 <div class="caja">
10 <header>
11 <h1>Cabecera: header</h1>
12 </header>
13 <nav>
14 <ul>
15 <li><a href="#">Menú 1</a></li>
16 <li><a href="#">Menú 2</a></li>
17 <li><a href="#">Menú 3</a></li>
18 </ul>
19 </nav>
20 <article>Contenido de la página:
21 <strong>article &gt; section</strong></article>
22 <aside>Área de contenido destacado
23 y relacionado: <strong>aside</strong></aside>
24 <footer>Pie: <strong>footer</strong></footer>
25 </div>
26 </body>
27 </html>
```

Below the code editor, a red box contains the text "+ CSS", with a red line pointing to the browser preview.

The browser preview (Firefox) shows the rendered output of the HTML structure:

- Cabecera: header**
- Menú 1 Menú 2 Menú 3**
- Contenido de la página: article > section**
- Área de contenido destacado y relacionado: aside**
- Pie: footer**

Estructura HTML

La estructura **HTML**, debe realizarse en el sentido lógico de lectura del documento, cuando la página se vea sin aplicar los estilos no pierda su significado.

theguardian

News Sport Comment Culture Business Money Life & style Travel Environment Tech TV Video Dating Offers Jobs

News UK World Development US Politics Media Education Society Science Women Law Data Football Observer

Breaking news: Review: Motorola phone notable for customization - AP

Twitter issues new rules to crack down on abuse



After rape and bomb threats, site to update rules, roll out 'report abuse' button more widely and use extra staff to police tweets

Lucy Mangan: net trolls and the currency of hate

Immigration spotchecks 'not racist'

Home Office contests claims that non-white people are being targeted by border agency at tube stations

- Farage: immigrant checks 'deeply disturbing'
- Alex Andreou: a new low for the Home Office

Telecoms firms pass details to GCHQ



Leading companies, including BT, Vodafone and Verizon, secretly collaborating with Britain's spy agency

1102 comments

- Peter Sommer: who's watching GCHQ?
- Analysis: GCHQ's room for manoeuvre
- Inside top secret world of UK's biggest spy agency

British nationals told to leave Yemen



Foreign Office say any Britons who stay in Yemen are unlikely to be able to be evacuated if unrest gets worse

US on alert over al-Qaida threat

Chaos over Tory welfare reform



DWP employees criticise

The Ashes England v Australia day three - live!



guardianjobs

Guardian Jobs
Reach quality candidates today
Click here »

Online dating



Find your soulmate with the Guardian's dating site
Join Guardian Soulmates

Subscribe

SUBSCRIBE

Learn more about subscribing to the Guardian in print

Beginner's blogging course



Interactive Celebrity selfies



Will Self
On happy pills



Sport

Gerrard to Suárez: stay at Liverpool



Football: Steven Gerrard says disgruntled team-mate Luis Suárez should stay at Anfield and earn a dream move

- Wenger: €100m Bale bid is a joke
- Newcastle agree deal to sign Gormis
- Sheffield Utd 2-1 Notts County

Championship preview: 2013-14



Jacob Steinberg casts his eye over the second-tier league's chances as the Football League kicks off again

Your search terms... The Guardian Search

theguardian

Twitter issues new rules to crack down on abuse



After rape and bomb threats, site to update rules, roll out 'report abuse' button more widely and use extra staff to police tweets

- Lucy Mangan: net trolls and the currency of hate

Immigration spotchecks 'not racist'

Home Office contests claims that non-white people are being targeted by border agency at tube stations

- Farage: immigrant checks 'deeply disturbing'
- Alex Andreou: a new low for the Home Office

Telecoms firms pass details to GCHQ



Leading companies, including BT, Vodafone and Verizon, secretly collaborating with Britain's spy agency

1102 comments

- Peter Sommer: who's watching GCHQ?
- Analysis: GCHQ's room for manoeuvre
- Inside top secret world of UK's biggest spy agency

British nationals told to leave Yemen



Foreign Office say any Britons who stay in Yemen are unlikely to be able to be evacuated if unrest gets worse

e/s/d/
madrid

Recursos CSS

CSS3 Generator

<http://css3generator.com/>

CSS3 Gradient Generator

<http://gradients.glrzad.com/>

{CSS} Portal - on line generator

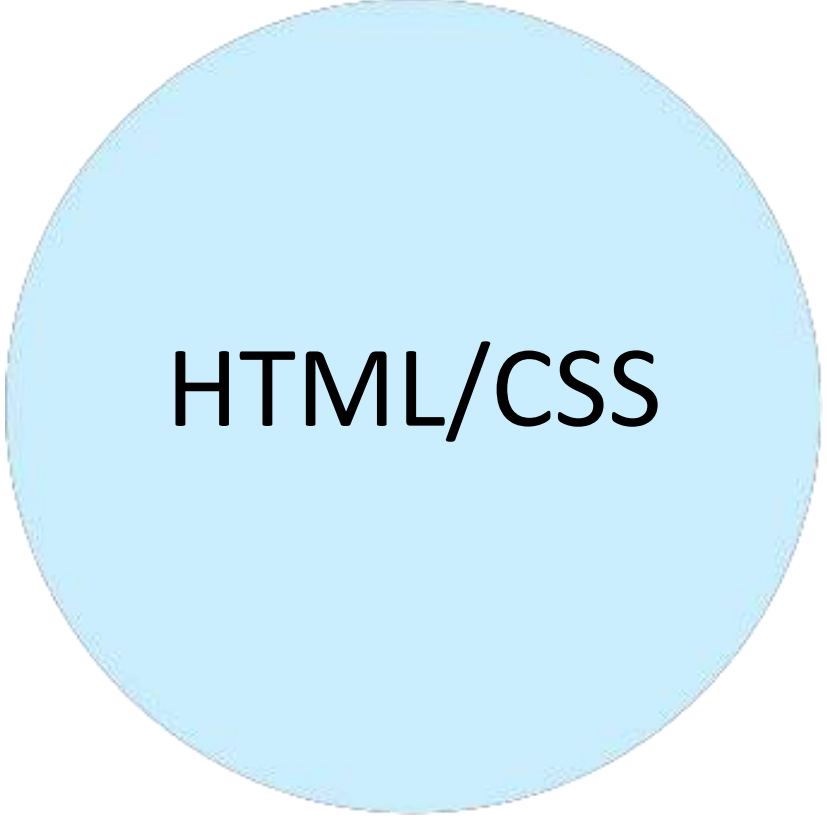
<http://www.cssportal.com/css3-rounded-corner/>

CSS3 MAKER:

<http://www.css3maker.com>

CSS3.com

<http://www.css3.com/>



HTML/CSS

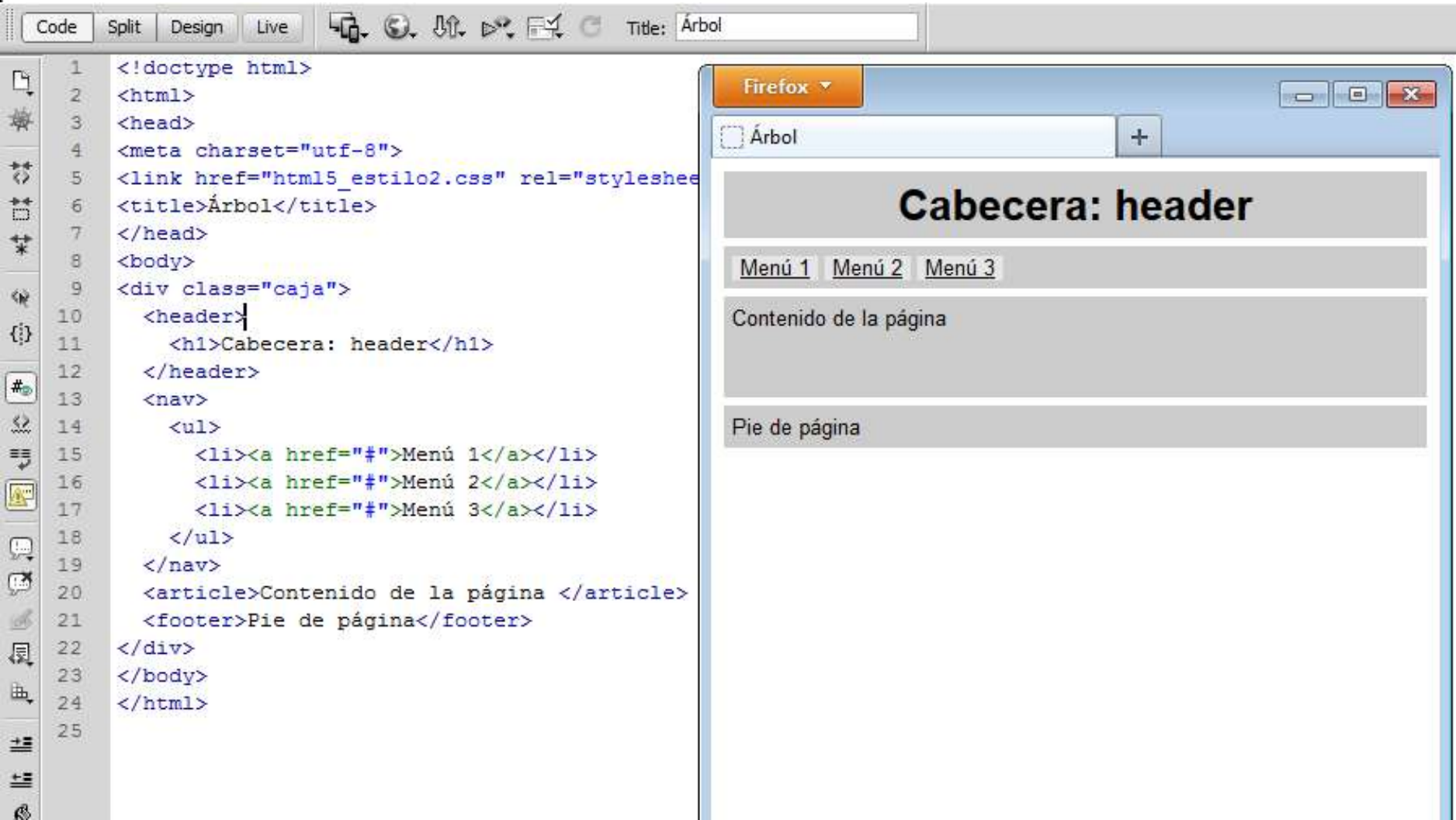


HTML

Herencia CSS

Herencia CSS

Para entender como funcionan los selectores y la herencia CSS es necesario entender qué es el árbol del documento.



The image shows a web browser window with two panes. The left pane displays the HTML source code, and the right pane shows the rendered page.

HTML Source Code (Left Pane):

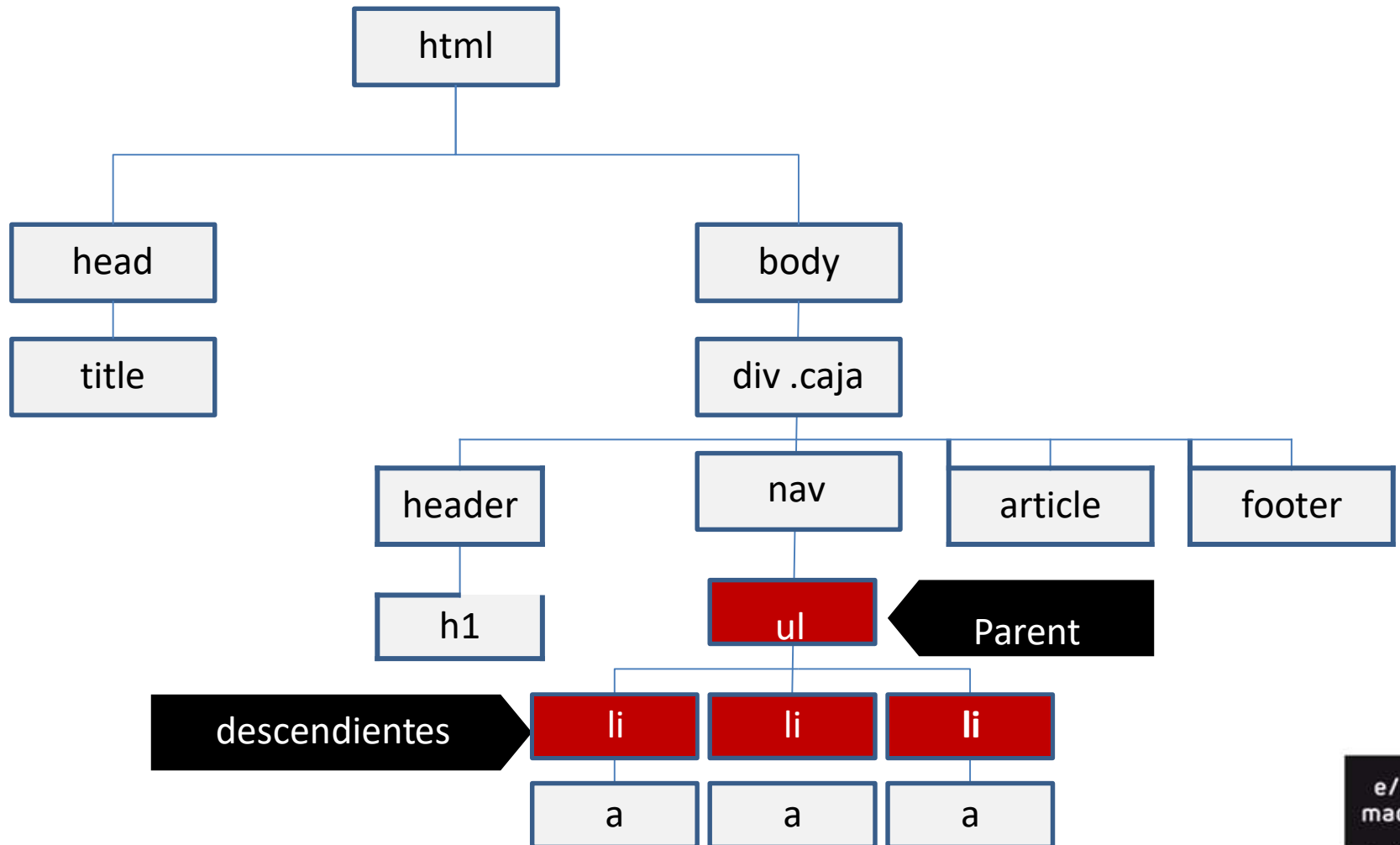
```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <link href="html5_estilo2.css" rel="stylesheet">
6 <title>Árbol</title>
7 </head>
8 <body>
9 <div class="caja">
10   <header>
11     <h1>Cabecera: header</h1>
12   </header>
13   <nav>
14     <ul>
15       <li><a href="#">Menú 1</a></li>
16       <li><a href="#">Menú 2</a></li>
17       <li><a href="#">Menú 3</a></li>
18     </ul>
19   </nav>
20   <article>Contenido de la página </article>
21   <footer>Pie de página</footer>
22 </div>
23 </body>
24 </html>
```

Rendered Page (Right Pane):

The rendered page shows the visual output of the HTML code. It features a header section with the title "Cabecera: header", a navigation menu with three items: "Menú 1", "Menú 2", and "Menú 3", a main content area labeled "Contenido de la página", and a footer area labeled "Pie de página".

Herencia CSS

Parent es el elemento conectado y directamente sobre un elemento en la estructura del documento.



Herencia CSS

Si definimos un estilo en el **body**, todos los elementos situados debajo en el árbol del documento, heredan esa propiedad.

The image shows a web browser window with the title "Herencia". The browser's address bar shows "Herencia". The page content is as follows:

Cabecera: header

- Opción 1
- Opción 2
- Opción 3

Contenido de la página

Pie de página

The browser window also shows a code editor on the left with the following HTML code:

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Herencia</title>
6 <style type="text/css">
7   body {
8     color: red;
9   }
10 </style>
11 </head>
12 <body>
13 <div class="caja">
14   <header>
15     <h1>Cabecera: header</h1>
16   </header>
17   <nav>
18     <ul>
19       <li>Opción 1</li>
20       <li>Opción 2</li>
21       <li>Opción 3</li>
22     </ul>
23   </nav>
24   <article>Contenido de la página </article>
25   <footer>Pie de página</footer>
26 </div>
27 </body>
28 </html>
```

Selectores CSS

El selector aplica a todos los elementos HTML de la página con esa etiqueta (p).

```
p {  
    color: #F00;  
}
```

El **selector múltiple** de CSS, incluye varios selectores separados por coma (,), para aplicar propiedades comunes: **h1, h2, h3**.

```
h1, h2, h3 {  
    color: #F00;  
}
```

El selector descendente puede incluir etiquetas **separadas solo por espacios**. Se aplicará solo a elementos que estén dentro de otros anteriores, ancestros, en el ejemplo, **.caja** deberá ser ancestro de **nav** y **nav** deberá ser ancestro de **ul**.

```
.caja nav ul {  
    color: red;  
}
```

Selectores CSS

Selector **universal** `*` afecta a todos los elementos.

```
* { margin: 0; padding: 0; }
```

Las **clases** se usan para aplicar estilos a un elemento determinado.

```
<p class="rojo">Párrafo rojo</p>  
.rojo { color: red; }
```

También se pueden aplicar estilos a un **id**.

```
<p id="texto">Párrafo especial</p>  
#texto { color: blue; }
```

La selección de hijos es similar al selector descendente, que se aplica sólo a los elementos que son hijos directos.

```
li > a { color: red; }
```

Prioridad CSS

Cuando dos declaraciones afectan a un mismo elemento. ¿cual de ellas se interpreta en el navegador como más importante?

CodeSplitDesignLive

Herencia

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Herencia</title>
6 <style type="text/css">
7 #caja header h1 { color: blue; }
8 #caja .cabecera h1 { color: red; }
9 header h1 { color: lime; }
10 h1 { color: purple; }
11 h1 { color: grey; }
12 </style>
13 </head>
14 <body>
15 <div id="caja">
16   <header class="cabecera">
17     <h1>Cabecera: header</h1>
18   </header>
19 </div>
20 </body>
21 </html>
22
```

A = 0 estilos en
línea B = 0 ID
C = 0 clases
D = 1 elemento
Puntuación =
0,0,0,1

Hay que calcular la tupla (A, B, C, D) ganadora de todas las reglas CSS que compiten. A tiene máximo peso y D mínimo. Si hay empate en A, se mira B y así sucesivamente.

A = estilo en línea
B = número de IDs
C = número de clases
D = número de marcas HTML

Prioridad CSS

Cuando dos declaraciones afectan a un mismo elemento. ¿cual de ellas se interpreta en el navegador como más importante?

CodeSplitDesignLive

Herencia

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Herencia</title>
6 <style type="text/css">
7 #caja header h1 { color: blue; }
8 #caja .cabecera h1 { color: red; }
9 header h1 { color: lime; }
10 h1 { color: purple; }
11 h1 { color: grey; }
12 </style>
13 </head>
14 <body>
15 <div id="caja">
16     <header class="cabecera">
17         <h1>Cabecera: header</h1>
18     </header>
19 </div>
20 </body>
21 </html>
22
```

A = 0 estilos en
línea B = 0 ID
C = 0 clases
D = 2 marcas
Puntuación =
0,0,0,2

Hay que calcular la tupla (A, B, C, D) ganadora de todas las reglas CSS que compiten. A tiene máximo peso y D mínimo. Si hay empate en A, se mira B y así sucesivamente.

A = estilo en línea
B = número de IDs
C = número de clases
D = número de marcas HTML

Prioridad CSS

Cuando dos declaraciones afectan a un mismo elemento. ¿cual de ellas se interpreta en el navegador como más importante?

CodeSplitDesignLive

Herencia

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Herencia</title>
6 <style type="text/css">
7 #caja header h1 { color: blue; }
8 #caja .cabecera h1 { color: red; }
9 header h1 { color: lime; }
10 h1 { color: purple; }
11 h1 { color: grey; }
12 </style>
13 </head>
14 <body>
15 <div id="caja">
16     <header class="cabecera">
17         <h1>Cabecera: header</h1>
18     </header>
19 </div>
20 </body>
21 </html>
22
```

A = 0 estilos en
línea B = 1 ID
C = 1 clase
D = 1 elemento
Puntuación =
0,1,1,1

Hay que calcular la tupla (A, B, C, D) ganadora de todas las reglas CSS que compiten. A tiene máximo peso y D mínimo. Si hay empate en A, se mira B y así sucesivamente.

A = estilo en línea
B = número de IDs
C = número de clases
D = número de marcas HTML

Prioridad CSS

Cuando dos declaraciones afectan a un mismo elemento. ¿cual de ellas se interpreta en el navegador como más importante?

CodeSplitDesignLive

Herencia

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Herencia</title>
6 <style type="text/css">
7 #caja header h1 { color: blue; }
8 #caja .cabecera h1 { color: red; }
9 header h1 { color: lime; }
10 h1 { color: purple; }
11 h1 { color: grey; }
12 </style>
13 </head>
14 <body>
15 <div id="caja">
16   <header class="cabecera">
17     <h1>Cabecera: header</h1>
18   </header>
19 </div>
20 </body>
21 </html>
22
```

A = 0 estilos en
línea B = 1 ID
C = 0 clases
D = 2 marcas
Puntuación =
0,1,0,2

Hay que calcular la tupla (A, B, C, D) ganadora de todas las reglas CSS que compiten. A tiene máximo peso y D mínimo. Si hay empate en A, se mira B y así sucesivamente.

A = estilo en línea
B = número de IDs
C = número de clases
D = número de marcas HTML

Prioridad CSS

La ganadora es, #caja .cabecera h1 = 0,1,1,1

The screenshot shows a web browser window with the title "Herencia". The browser displays the text "Cabecera: header" in red. A red box highlights the text "Ganadora:" followed by the breakdown of the selector's specificity: "A = 0 estilos en línea", "B = 1 ID", "C = 1 clase", "D = 1 elemento", and "Puntuación = 0,1,1,1". A red line connects this box to the CSS code in the background, specifically to the line "#caja .cabecera h1 { color: red; }".

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Herencia</title>
6 <style type="text/css">
7 #caja header h1 { color: blue; }
8 #caja .cabecera h1 { color: red; }
9 header h1 { color: lime; }
10 h1 { color: purple; }
11 h1 { color: grey; }
12 </style>
13 </head>
14 <body>
15 <div id="caja">
16   <header class="cabecera">
17     <h1>Cabecera: header</h1>
18   </header>
19 </div>
20 </body>
21 </html>
22
```

Prioridad CSS

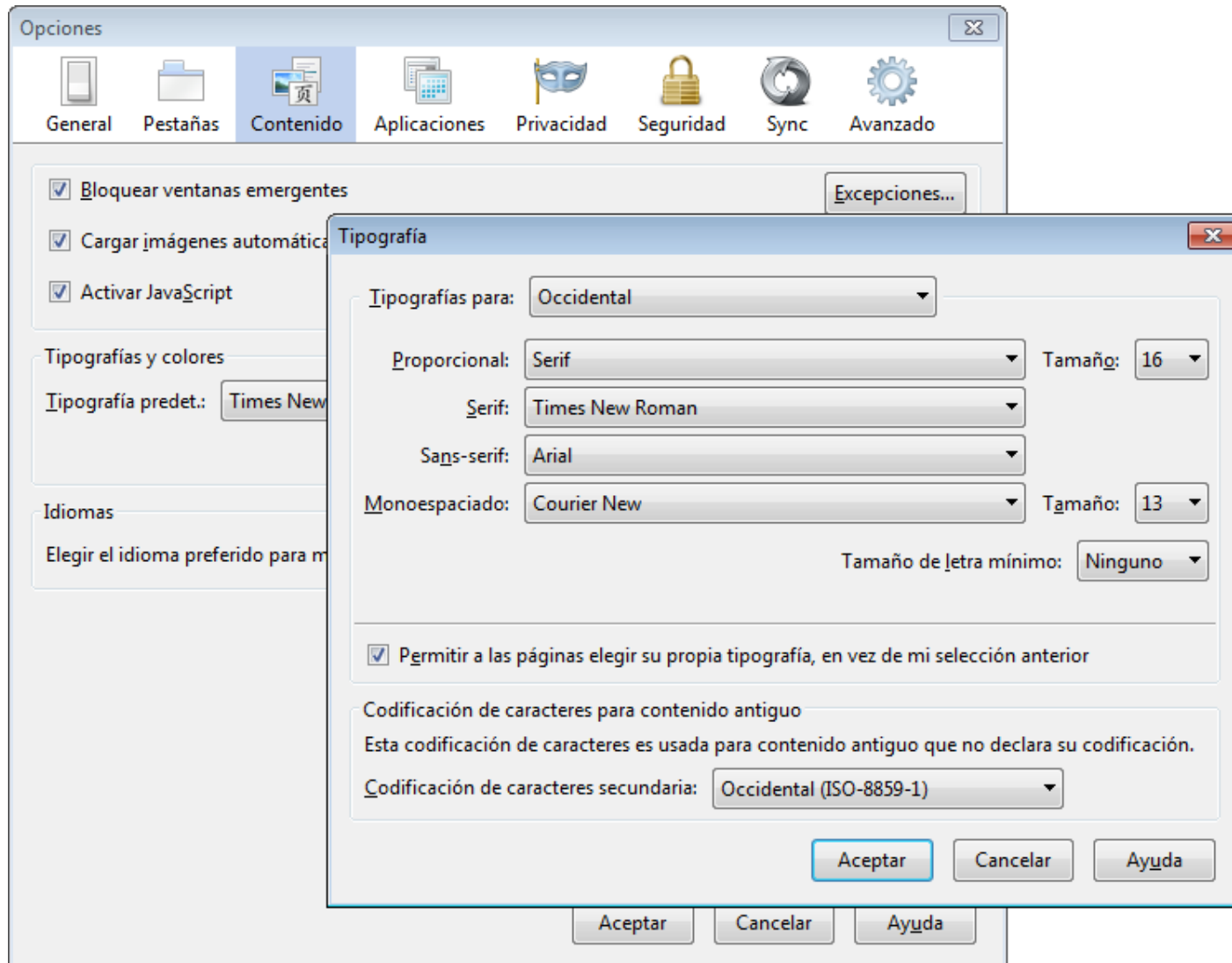
Cuando dos declaraciones tienen el mismo valor:
Será la última especificada

The screenshot shows a web browser window with the title "Herencia". The browser's address bar shows "Herencia" and a plus sign. The main content area displays the text "Cabecera: header" in a large, bold, grey font. The browser's developer tools are open, showing the HTML code for the page. The code is as follows:

```
1 <!doctype html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>Herencia</title>
6 <style type="text/css">
7 h1 { color: purple; }
8 h1 { color: grey; }
9 </style>
10 </head>
11 <body>
12 <div id="caja">
13   <header class="cabecera">
14     <h1>Cabecera: header</h1>
15   </header>
16 </div>
17 </body>
18 </html>
19
```

Reglas CSS

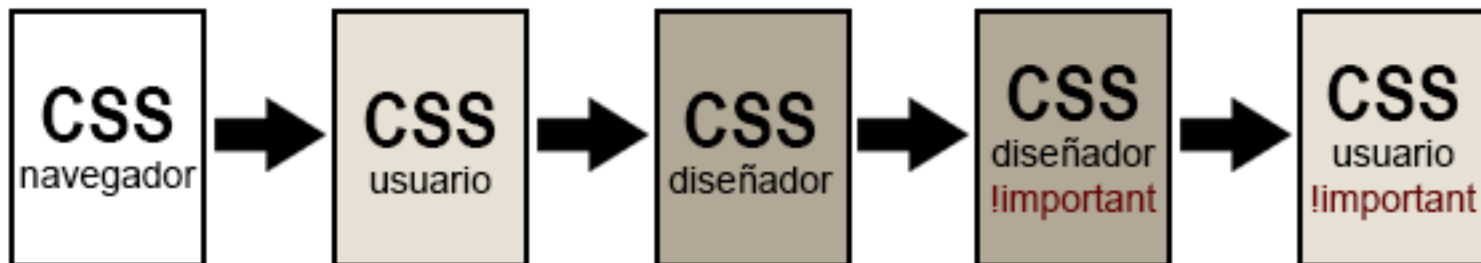
Modificación de estilos, por el usuario en el navegador: tipografías



Prioridad CSS

Reglas adicionales de prioridad de las declaraciones CSS ordenadas de menor a mayor:

- CSS **por defecto del Navegador** (navegador)
- CSS en **preferencias de usuario del navegador** (usuario)
- CSS en **página HTML o script CSS** (diseñador)
- CSS en **página HTML o script CSS** con **!important** (diseñador), p. e. **body {color:blue !important;}**
- CSS en **preferencias de usuario del navegador** con **!important** (usuario), p. e. **body {color:blue !important;}**



Recursos CSS

Kseso CSS: CSS básico: Cascada, especificidad y herencia

<http://ksesocss.blogspot.com/2012/05/css-basico-cascada-especificidad-y.html>

Dev.Opera: Recorriendo el árbol DOM

<http://dev.opera.com/articles/view/traversing-the-dom-es/>

LIBROS WEB: Árbol de nodo

http://librosweb.es/javascript/capitulo_5/arbol_de_nodos.html

UOC: Herencia y cascada

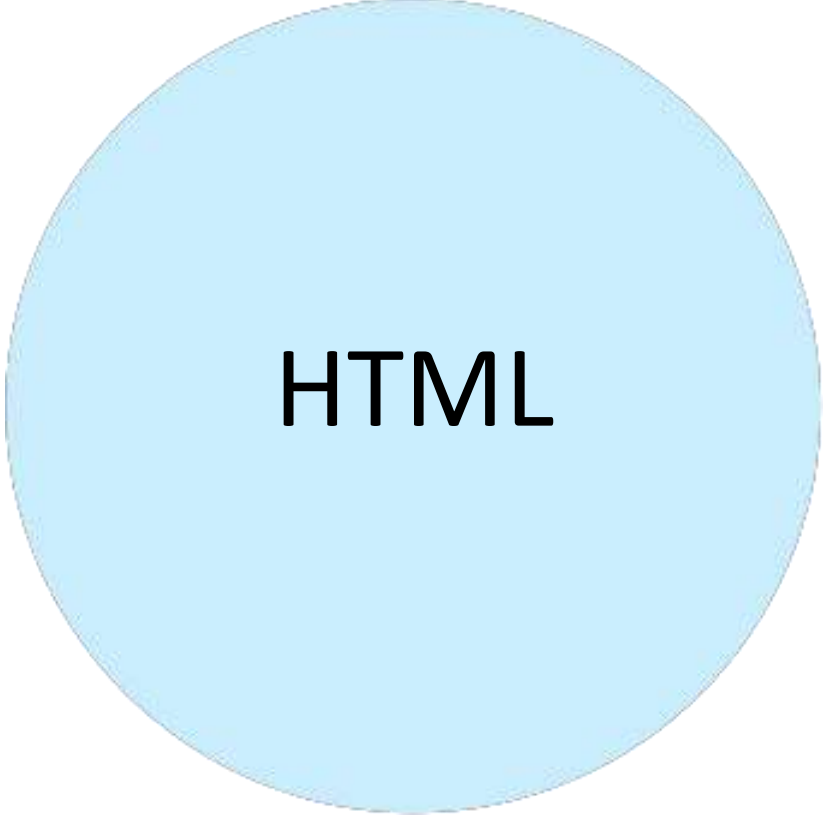
<http://mosaic.uoc.edu/ac/le/es/m6/ud2/>

LIBROSWEB: Colisiones de estilos

http://librosweb.es/css/capitulo_2/colisiones_de_estilos.html

CSS: Specificity Wars

http://www.stuffandnonsense.co.uk/archives/css_specificity_wars.html



HTML

Recurso 1 — Mapa de decisiones para escoger tipos de <input>

Objetivo. Guiarte, paso a paso, para elegir el tipo de campo HTML5 correcto y su configuración mínima accesible (atributos, autocomplete, inputmode, ayudas y errores). Es una guía, no una actividad.

Cómo usar este recurso (3 pasos)

1. Identifica el dato que necesitas (correo, teléfono, número, fecha, archivo, texto...).
2. Sigue el diagrama (abajo) o la versión solo texto para escoger el tipo nativo.
3. Aplica la configuración recomendada (atributos, tokens de autocomplete, mensajes claros).

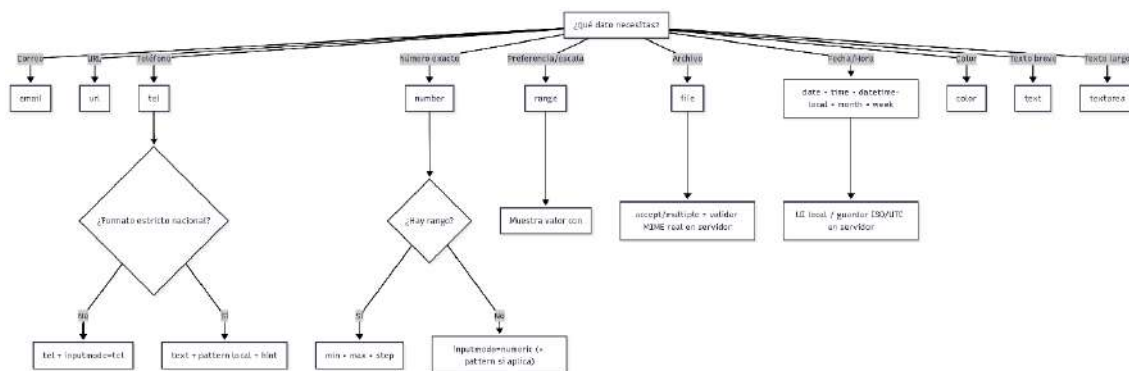


Figura 1. *Árbol de decisión para seleccionar el tipo de control HTML5* (Elaboración propia del autor, Damián Nicolalde, 2025)

- Correo → type="email"
- URL → type="url"
- Teléfono → type="tel" (si tu país exige formato rígido, usa type="text" + pattern + pista clara)
- Número exacto → type="number" (+ min/max/step si hay rango; si no, inputmode="numeric")
- Preferencia/escala → type="range" (+ <output> que muestre el valor)
- Fecha/hora → date / time / datetime-local / month / week (UI local; guardar en ISO/UTC)
- Archivo → type="file" (accept, multiple; en servidor validar MIME real y tamaño)
- Color → type="color"
- Texto breve → type="text" (+ minlength/maxlength)
- Texto largo → <textarea>

Tabla 1. *Tipo recomendado, atributos clave y sugerencias de A11y/UX* (Elaboración propia del autor, Damián Nicolalde, 2025)

Dato	Tipo recomendado	Atributos clave	Sugerencias A11y/UX
Correo	email	required	autocomplete="email", mensaje claro al fallar
URL	url	(opc.) required	Agrega ejemplo en placeholder (p. ej., https://...)
Teléfono	tel	—	inputmode="tel"; si hay formato rígido, usar pattern + hint
Número exacto	number	min max step	Evita bloquear mientras escribe; usa :invalid/:valid
Preferencia/escala	range	min max step	Muestra el valor con <output>
Fecha/Hora	date/time/datetime-local	—	UI local; persiste ISO/UTC en servidor
Archivo	file	accept multiple	Validar tipo real y tamaño en servidor
Color	color	—	Útil en personalización sencilla
Texto breve	text	minlength maxlength	inputmode según caso (numeric, email, etc.)
Texto largo	textarea	rows/cols	Evita validaciones intrusivas al teclear

Snippets mínimos por caso

Todos los ejemplos incluyen label, ayuda/error vinculados y comentarios.

Correo

```
<label for="e">Correo</label>
<input id="e" name="e" type="email" required
  autocomplete="email" aria-describedby="e-hint e-err">
<small id="e-hint">Usa tu correo institucional.</small>
<span id="e-err" role="alert" hidden>Formato: usuario@dominio.</span>
```

Teléfono (flexible)

```
<label for="t">Teléfono</label>
<input id="t" name="t" type="tel" inputmode="tel"
  aria-describedby="t-hint">
<small id="t-hint">Incluye código de área si aplica.</small>
```

Teléfono (formato estricto local — ejemplo)

```
<label for="t2">Teléfono (EC)</label>
<input id="t2" name="t2" type="text" inputmode="tel"
  pattern="^\+593\s?\d{2}\s?\d{3}\s?\d{4}$"
  aria-describedby="t2-hint t2-err">
<small id="t2-hint">Ej.: +593 22 345 6789</small>
<span id="t2-err" role="alert" hidden>Usa el formato del ejemplo.</span>
```

Número exacto (con rango)

```
<label for="n">Créditos</label>
<input id="n" name="n" type="number" min="1" max="40" step="1" required>
```

Escala/preferencia (mostrar valor)

```
<label for="s">Satisfacción</label>
<input id="s" type="range" min="1" max="5" step="1"
  oninput="s_out.value = this.value" value="3" aria-describedby="s-hint">
<small id="s-hint">1 = bajo, 5 = alto</small>
<output id="s_out">3</output>
```

Fecha (UI local; guardar ISO/UTC en servidor)

```
<label for="f">Fecha del turno</label>
<input id="f" name="f" type="date" required>
```

Archivo (validar en servidor tipo real y tamaño)

```
<label for="cv">CV (PDF)</label>
<input id="cv" name="cv" type="file" accept="application/pdf">
```

URL

```
<label for="u">Sitio web</label>
<input id="u" name="u" type="url" placeholder="https://ejemplo.edu.ec">
```

Texto breve

```
<label for="nick">Alias</label>
<input id="nick" name="nick" type="text" minlength="3" maxlength="20"
  aria-describedby="nick-hint">
<small id="nick-hint">3–20 caracteres, sin espacios.</small>
```

Texto largo

```
<label for="bio">Presentación</label>
<textarea id="bio" name="bio" rows="4"
  aria-describedby="bio-hint"></textarea>
<small id="bio-hint">Cuenta brevemente sobre ti (máx. 600 caracteres).</small>
```

Plantilla “campo bien formado” (rellena y reutiliza)

```
<label for="X">Etiqueta clara</label>
<input id="X" name="X" type="TIPO"
  autocomplete="TOKEN" inputmode="MODOS"
  required minlength="N" maxlength="M" min="A" max="B" step="S"
  aria-describedby="X-hint X-err">
<small id="X-hint">Pista breve (regla o ejemplo).</small>
<span id="X-err" role="alert" hidden>Mensaje de corrección.</span>
```

Notas clave

- **label for** define el nombre accesible del control.
- **aria-describedby** enlaza ayuda y error para que el lector los anuncie junto al campo.
- **Nativo primero**: si existe un tipo (email, url, date, number...), úsalo antes que pattern.
- **Móvil**: usa inputmode para mejorar el teclado (numeric, tel, email).

- **Archivos:** accept orienta, pero valida en servidor el MIME real y el tamaño.
- **Fechas:** UI local, persistencia ISO/UTC.

Recurso 2 — Biblioteca de mensajes accesibles de error (plantilla)

Objetivo. Proveer un patrón reutilizable (HTML+CSS+JS) para mostrar mensajes claros y anunciabiles, basado en la API nativa de validación, con extensiones para reglas cruzadas y validación asíncrona.

Patrón base (HTML + CSS)

```
<form id="f1" novalidate aria-labelledby="t-form">
  <h2 id="t-form">Registro</h2>

  <div class="field">
    <label for="email">Correo institucional</label>
    <input id="email" name="email" type="email" required
      autocomplete="email" aria-describedby="email-hint email-err">
    <small id="email-hint">Formato: usuario@dominio.</small>
    <span id="email-err" class="error" role="alert" hidden></span>
  </div>

  <div class="field">
    <label for="pass">Contraseña</label>
    <input id="pass" name="pass" type="password" required minlength="8"
      aria-describedby="pass-hint pass-err">
    <small id="pass-hint">Mín. 8 caracteres.</small>
    <span id="pass-err" class="error" role="alert" hidden></span>
  </div>

  <button type="submit">Crear cuenta</button>
</form>

/* Visibilidad de estados y foco */
.field input:invalid { border:2px solid #c00; }
.field input:valid { border:2px solid #0a0; }
:focus-visible { outline:3px solid #000; outline-offset:2px; }

/* Mensajes de error */
.error[hidden]{ display:none; }
.error{ display:block; margin-top:.25rem; }

/* No dependas solo del color: mantén el texto claro */
```

Maapeo de `ValidityState` → mensaje recomendado

```
const messages = {
  valueMissing: 'Completa este campo.',
  typeMismatch: 'Revisa el formato (p. ej., usuario@dominio).',
  patternMismatch: 'Sigue el formato indicado.',
  tooShort: (el) => `Añade ${el.minLength - el.value.length} caracteres más.`,
  tooLong: (el) => `Reduce a ${el.maxLength} caracteres.`,
  rangeUnderflow: (el) => `El valor mínimo es ${el.min}.`,
  rangeOverflow: (el) => `El valor máximo es ${el.max}.`,
  stepMismatch: 'Ajusta al paso permitido.',
  badInput: 'Usa solo números en este campo.',
  customError: (el) => el.validationMessage || 'Revisa este dato.'
};
```

Utilitario JS para un formulario (plug-and-play)

```

function attachA11yValidation(form){
  const fields = form.querySelectorAll('input, select, textarea');

  function showError(el, msg){
    const errId = (el.getAttribute('aria-describedby') || '').split(' ')
      .find(id => id && id.endsWith('-err'));
    const errEl = errId ? document.getElementById(errId) : null;
    el.setCustomValidity(msg || '');
    if (msg){
      el.setAttribute('aria-invalid', 'true');
      if (errEl){ errEl.textContent = msg; errEl.hidden = false; }
    } else {
      el.removeAttribute('aria-invalid');
      if (errEl){ errEl.hidden = true; errEl.textContent=''; }
    }
  }

  function nativeMessage(el){
    const v = el.validity;
    if (v.valid) return '';
    for (const key of Object.keys(messages)){
      if (v[key]){
        const m = messages[key];
        return typeof m === 'function' ? m(el) : m;
      }
    }
    return 'Revisa este dato.';
  }

  fields.forEach(el=>{
    el.addEventListener('input', ()=> showError(el, nativeMessage(el)));
    el.addEventListener('blur', ()=> showError(el, nativeMessage(el)));
  });

  form.addEventListener('submit', (e)=>{
    // Recorre y muestra el primer error
    for (const el of fields){
      const msg = nativeMessage(el);
      if (msg){
        e.preventDefault();
        showError(el, msg);
        el.focus(); // foco al primer campo con error
        break;
      }
    }
  });
}

// Activación
attachA11yValidation(document.getElementById('f1'));

```

Reglas cruzadas (ejemplo: “fin ≥ inicio”)

```
<label for="ini">Inicio</label>
<input id="ini" type="date" required>
<label for="fin">Fin</label>
<input id="fin" type="date" required aria-describedby="fin-err">
<span id="fin-err" role="alert" hidden></span>

<script>
const ini = document.getElementById('ini');
const fin = document.getElementById('fin');
function cross(){
  fin.setCustomValidity("");
  if (ini.value && fin.value && fin.value < ini.value){
    fin.setCustomValidity('La fecha de fin debe ser posterior a inicio. ');
  }
  fin.dispatchEvent(new Event('input')); // para refrescar mensaje con utilitario
}
ini.addEventListener('change', cross);
fin.addEventListener('change', cross);
</script>
```

Validación asíncrona (esqueleto con role="status")

```
<input id="mail2" type="email" required aria-describedby="st-mail2 mail2-err">
<small id="st-mail2" role="status" aria-live="polite"></small>
<span id="mail2-err" role="alert" hidden></span>

<script>
let debounce;
mail2.addEventListener('input', ()=>{
  clearTimeout(debounce);
  st-mail2.textContent = "";
  mail2.setCustomValidity("");
  if (!mail2.checkValidity()) return;

  debounce = setTimeout(async ()=>{
    st-mail2.textContent = 'Comprobando...';
    // const {exists} = await
    fetch('/api/check?e='+encodeURIComponent(mail2.value)).then(r=>r.json());
    const exists=false; // simula
    st-mail2.textContent = exists ? 'Este correo ya tiene cuenta.' : 'Disponible.';
    if (exists){ mail2.setCustomValidity('Correo ya registrado. '); }
    mail2.dispatchEvent(new Event('input'));
  }, 300);
});
</script>
```

Buenas prácticas de redacción (mensajes): “regla + cómo corrijo”, lenguaje simple, sin mayúsculas “a gritos”, sin tecnicismos innecesarios.

Recurso 3 — Checklist WCAG AA para formularios

Objetivo. Hoja de verificación operativa para revisar rápidamente formularios conforme a POUR (AA) antes de publicar.

Tabla 2. *Checklist por principio, con verificación y nota* (Elaboración propia del autor, Damián Nicolalde, 2025)

Ítem (POUR)	Cómo verificar	OK	N/A	Notas
P: Cada imagen informativa tiene alt útil; si requiere contexto, figure+figcaption.	Inspección + lector de pantalla.	☐	☐	
P: Contraste de texto $\geq 4.5:1$ (normal) / $3:1$ (grande).	Lighthouse/Axe.	☐	☐	
P: Zoom 200–400% sin romper (sin scroll horizontal general).	Zoom del navegador.	☐	☐	
O: Se puede completar todo con teclado (Tab/Shift+Tab/Enter/Espacio).	Recorrido manual.	☐	☐	
O: Foco visible y orden lógico.	Estilos :focus-visible.	☐	☐	
O: Modales accesibles (foco atrapado legítimo, cerrar con Esc, foco vuelve).	Prueba manual.	☐	☐	
U: Todos los campos tienen label for .	Inspección.	☐	☐	
U: Ayudas/errores vinculados (aria-describedby) y anunciables (role="alert"/aria-live).	Prueba de error.	☐	☐	
U: Jerarquía de encabezados correcta (h1→h2→h3).	Vista de encabezados.	☐	☐	
R: Semántica nativa; ARIA solo si falta (sin roles redundantes).	Validador + DevTools.	☐	☐	
R: Validación nativa encendida; reglas JS claras; servidor revalida.	Prueba checkValidity() + envío real.	☐	☐	

Prueba rápida

1. **Teclado:** completa el formulario sin mouse; confirma que el foco se ve en todo.
2. **Errores:** fuerza 2–3 errores; verifica que se lean y expliquen cómo corregir.
3. **Zoom 200%:** comprueba que no se rompa el layout.
4. **Lighthouse A11y:** ≥ 90 como barómetro interno (no sustituye revisión manual).

Prueba extendida (15 minutos)

- **Contraste** en componentes clave (botones, errores, links).
- **Lectores** (si es posible): NVDA (Win) o VoiceOver (Mac) en 1 flujo.
- **Validación asíncrona:** un caso real (p. ej., “correo ya registrado”) con role="status" y sin bloqueos.

Criterios de aceptación (pasa/no pasa)

- Al menos **90** en Lighthouse (Accessibility) + todos los ítems críticos **OK** (label, ayudas vinculadas, foco visible, teclado completo, zoom 200%).
- Sin **tablas para maquetar**; sin **placeholders como etiquetas**; sin quitar el **outline**.



FORMULARIO DE PRODUCCIÓN DE CONTENIDOS

NOMBRE DE LA ASIGNATURA: Desarrollo en plataformas
NOMBRE COMPLETO DEL DOCENTE CONTENIDISTA: Damián Aníbal Nicolalde Rodríguez

DESARROLLO

1. INTRODUCCIÓN DE LA CLASE (2 párrafos)

En esta tercera clase profundizaremos en los principios que permiten que una interfaz web sea flexible, escalable y adaptable a distintos dispositivos. Comprenderás cómo el modelo de caja (Box Model) define la estructura básica de todo elemento HTML y cómo su manipulación mediante propiedades de espaciado, bordes y márgenes influye directamente en la estética y funcionalidad de un sitio. Desde esta base, daremos el paso hacia los sistemas de organización modernos —Flexbox y CSS Grid— que reemplazan los antiguos métodos basados en flotaciones, ofreciendo control total sobre la disposición de los elementos tanto en una como en dos dimensiones.

Asimismo, exploraremos el concepto de diseño responsivo y el uso estratégico de Media Queries para crear experiencias coherentes en móviles, tabletas y escritorios. Analizaremos también el papel de los frameworks CSS contemporáneos (como Bootstrap y Tailwind) en la estandarización y aceleración del desarrollo, destacando sus ventajas y limitaciones frente al CSS puro. Al finalizar la clase, serás capaz de construir interfaces visualmente consistentes, accesibles y adaptables, integrando estructura, estilo y respuesta al contexto en una sola arquitectura visual.

Clase 3: CSS3 y diseño adaptable

RDA 1: Programar aplicaciones web dinámicas, interactivas y con diseño visualmente adaptable, que permita mejorar la experiencia del usuario.

Reto # 1

3.1. CSS3 y diseño adaptable

El **lenguaje CSS3** (Cascading Style Sheets nivel 3) es el estándar utilizado para definir la presentación visual de los documentos HTML. Su importancia radica en que permite separar la estructura del contenido (HTML) de la forma en que se muestra (CSS), facilitando la creación de sitios web modernos, accesibles y visualmente atractivos.

Uno de los conceptos más relevantes al trabajar con CSS3 es el diseño adaptable (adaptive design), que consiste en lograr que una interfaz pueda ajustarse a distintos



dispositivos y tamaños de pantalla de manera automática, sin sacrificar usabilidad ni estética.

1. ¿Qué es CSS3 y qué aportó frente a versiones anteriores?

CSS3 introdujo un conjunto de módulos independientes que ampliaron notablemente las capacidades del lenguaje. A diferencia de CSS2, que era más rígido, CSS3 permite evolucionar cada módulo por separado (tipografía, fondos, animaciones, transformaciones, layout, etc.).

Tabla 1. Comparación de CSS2 vs CSS3

Característica	CSS2	CSS3
Estructura	Monolítica, todas las reglas en un bloque	Modular, dividido en diferentes especificaciones
Tipografía	Limitada a fuentes del sistema	@font-face permite usar fuentes personalizadas
Layout	Flotantes y posiciones absolutas	Flexbox y Grid para layouts complejos
Colores	Paleta básica y hexadecimal	RGBA, HSLA, gradientes lineales y radiales
Animaciones	Necesitaban JavaScript	Nativas con <code>transition</code> y <code>@keyframes</code>

Fuente: elaboración propia (2025)

2. Principios del diseño adaptable

El diseño adaptable busca que un sitio web funcione y se vea bien en cualquier contexto, desde un celular pequeño hasta una pantalla 4K.



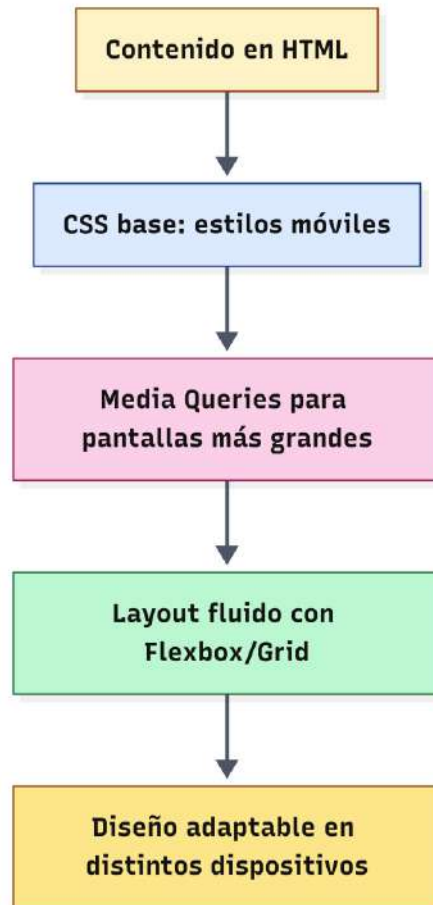


Imagen 1. Idea general del diseño adaptable

Fuente: creación propia (2025).

Principios fundamentales:

1. **Mobile-first:** Se diseña primero pensando en dispositivos pequeños y luego se agregan reglas para pantallas más grandes.
2. **Fluidez:** Se emplean unidades relativas (% , em, rem, vw, vh) en lugar de píxeles fijos.
3. **Imágenes y videos flexibles:** `max-width: 100%` garantiza que no desborden.
4. **Jerarquía tipográfica adaptable:** se utilizan escalas fluidas, por ejemplo:
5. `h1 { font-size: clamp(1.5rem, 2vw, 2.5rem); }`

Con esto, el tamaño mínimo es 1.5 rem, el máximo 2.5 rem y entre esos límites se ajusta al ancho de la ventana.

3. Diferencia entre diseño adaptable y diseño responsivo

Aunque suelen usarse como sinónimos, no son idénticos:

- **Adaptable:** utiliza diseños predefinidos que “saltan” entre versiones según el dispositivo (móvil, tableta, escritorio).
- **Responsivo:** ajusta el contenido de manera fluida y continua, reaccionando a cada variación en la ventana.

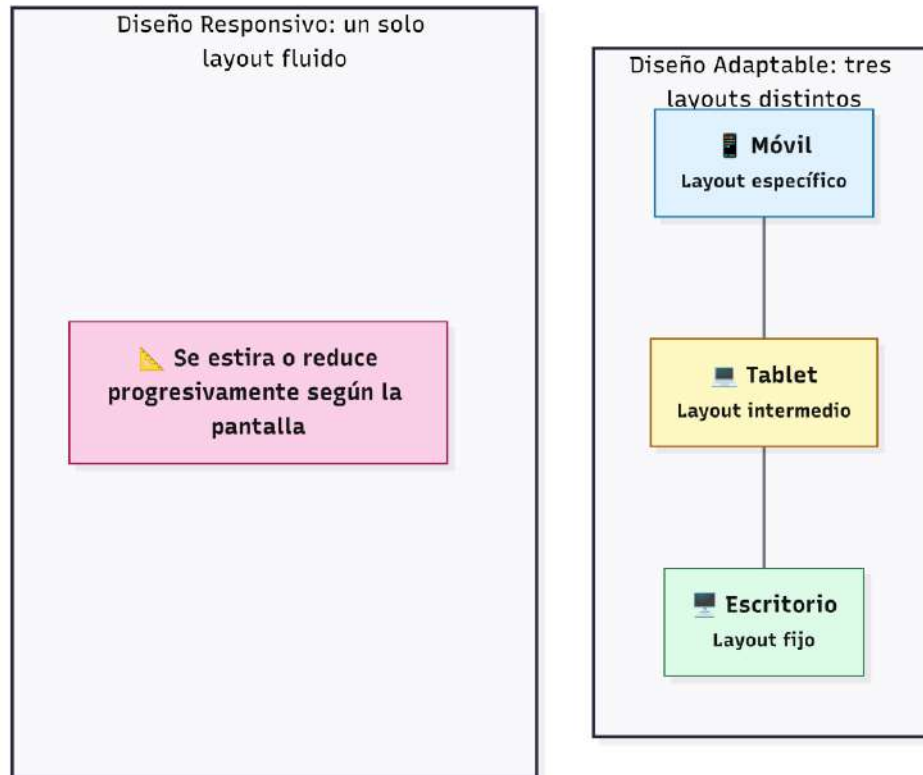


Imagen 2. Adaptable vs. responsivo

Fuente: elaboración propia (2025).

Título del enlace relacionado: Guía completa de diseño web responsivo (MDN Web Docs)

Descripción del enlace relacionado: Recurso oficial de Mozilla Developer Network que explica de manera práctica cómo aplicar CSS3 y Media Queries para construir sitios web responsivos. Incluye ejemplos y buenas prácticas.

Enlace: https://developer.mozilla.org/es/docs/Learn/CSS/CSS_layout/Responsive_Design



4. Unidades relativas y su rol en la adaptabilidad

Uno de los errores comunes en principiantes es fijar medidas en píxeles. CSS3 recomienda trabajar con unidades relativas, que permiten escalabilidad.

Tabla 2. Unidades más comunes

Unidad	Referencia	Ejemplo	Uso recomendado
%	Elemento contenedor	<code>width: 80%</code>	Layouts fluidos
em	Tamaño de la fuente del elemento	<code>padding: 2em</code>	Espaciados relativos a texto local
rem	Tamaño de la fuente raíz	<code>font-size: 1.2rem</code>	Tipografía consistente
vw	1% del ancho de la ventana	<code>font-size: 5vw</code>	Texto que crece con viewport
vh	1% de la altura de la ventana	<code>height: 50vh</code>	Secciones que llenan pantalla

Fuente: elaboración propia (2025).

5. Buenas prácticas para un diseño adaptable

1. Evitar anchos fijos:

```
contenedor { max-width: 1200px; margin: auto; }
```

Esto asegura que el contenido nunca sea más ancho de lo necesario.

2. Hacer que las imágenes sean flexibles:

```
img { max-width: 100%; height: auto; }
```

3. Diseñar sistemas de espaciado:

Usar variables CSS:

```
:root {
  --space-1: 0.5rem;
  --space-2: 1rem;
  --space-3: 2rem;
}
```

4. Mantener la legibilidad:

Líneas de 60–80 caracteres, `line-height: 1.5;`, y contraste suficiente entre texto y fondo.



```
body {  
  font-family: sans-serif;  
  margin: 0 auto;  
  max-width: 1200px;  
  padding: 1rem;  
}  
  
h1 {  
  font-size: clamp(1.5rem, 2vw, 2.5rem);  
}
```

Imagen 3. Ejemplo de layout fluido básico

Fuente: elaboración propia (2025).

6. Errores frecuentes

- **Medidas rígidas:** usar `width: 960px;` en vez de `max-width`.
- **Olvidar probar en distintos dispositivos:** diseñar solo en un monitor grande.
- **No usar media queries:** confiar en que el navegador “lo ajustará solo”.
- **Abusar de píxeles:** dificulta la escalabilidad.
- **Ignorar accesibilidad:** contraste insuficiente o texto muy pequeño.

7. Caso práctico narrado

Situación: un estudiante diseña una página de recetas con un título, imagen y lista de ingredientes.

1. En escritorio, todo se ve perfecto: imagen grande a la izquierda, ingredientes a la derecha.
2. En celular, la imagen ocupa toda la pantalla y los ingredientes se pierden abajo.
3. Con CSS3 adaptable, se corrige:
 - Se establece `max-width: 100%` para la imagen.
 - Se define un layout con `display: flex` que pasa de columnas en escritorio a filas en móvil usando `@media (max-width: 600px)`.

Resultado: en cualquier dispositivo, la receta se lee con claridad.

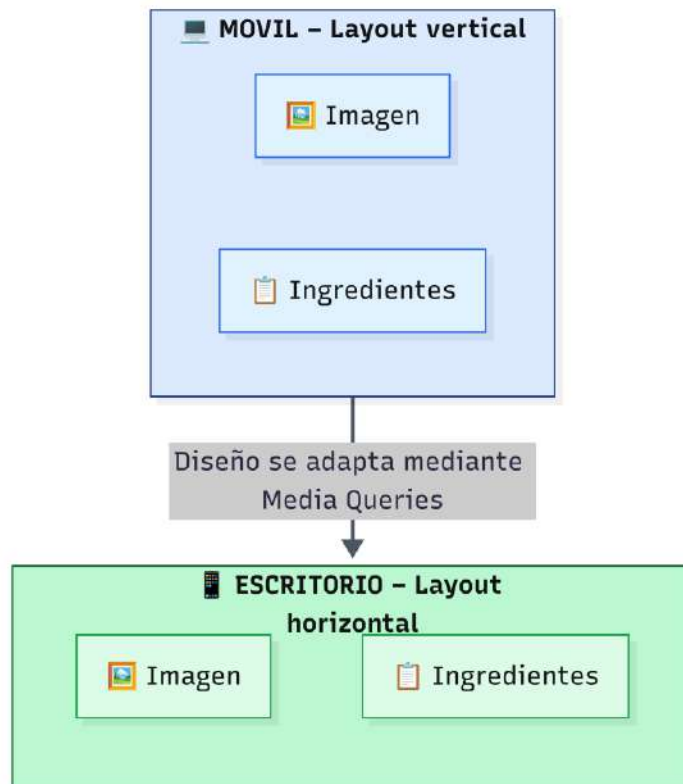


Imagen 4. Caso práctico de adaptación

Fuente: elaboración propia (2025).

8. Resumen

El diseño adaptable con CSS3 no es un lujo, sino una necesidad en la web actual. Un usuario puede entrar desde un celular, una tableta, una laptop o un televisor, y la interfaz debe responder de manera óptima en todos los casos.

CSS3 proporciona las herramientas necesarias: unidades relativas, tipografía fluida, imágenes flexibles, Media Queries, Flexbox y Grid, además de prácticas de accesibilidad.

3.2. Box model, Flexbox y Grid

La forma en que los navegadores renderizan los elementos HTML depende de un concepto central: el modelo de caja (box model). Cada etiqueta en una página se interpreta como una caja rectangular que tiene contenido, relleno, borde y márgenes.

CSS3 introduce herramientas más modernas para organizar estas cajas en layouts complejos: **Flexbox** y **Grid**, que reemplazan a técnicas antiguas como los `float` o el uso excesivo de tablas.

1. El modelo de caja (Box model)

Cada elemento de una página web está envuelto en un "box". Comprender este modelo es esencial para controlar el espacio, las alineaciones y los diseños.

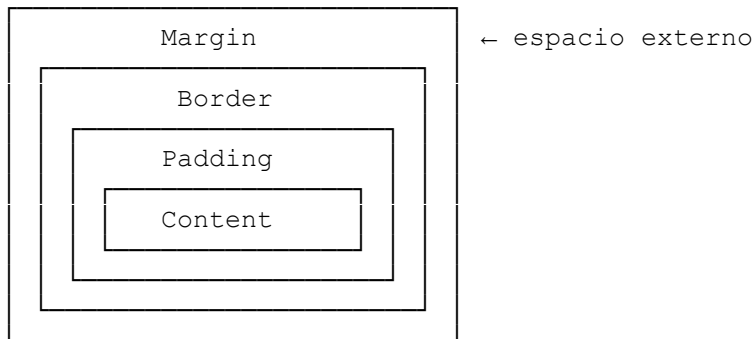


Imagen 5. Esquema del modelo de caja

Fuente: elaboración propia (2025).

Componentes principales:

1. **Content:** el texto, imagen o elemento principal.
2. **Padding:** espacio interno entre el contenido y el borde.
3. **Border:** línea que rodea el padding y contenido.
4. **Margin:** espacio externo, separa la caja de otras cajas.

Ejemplo en CSS:

```
.caja {
  width: 200px;
  padding: 20px;
  border: 2px solid black;
  margin: 10px;
}
```

Este bloque tendrá:

- 200px de ancho de contenido.
- +20px de padding en cada lado.
- +2px de borde.
- +10px de margen alrededor.

Ancho real total = 200 + (20x2) + (2x2) = 244px.

Para simplificar, se suele usar:

```
* { box-sizing: border-box; }
```



Así el ancho total incluye padding y borde, evitando cálculos manuales.

2. Flexbox: diseño unidimensional

Flexbox (Flexible Box Layout) organiza cajas en una dirección principal: fila o columna. Es ideal para alinear elementos dentro de un contenedor.

```
Fila (row):  [Caja 1][Caja 2][Caja 3]

Columna (col):

[Caja 1]
[Caja 2]
[Caja 3]
```

Imagen 6. Idea de Flexbox

Fuente: elaboración propia (2025).

Propiedades clave del contenedor flex:

```
.contenedor {
  display: flex;
  flex-direction: row; /* o column */
  justify-content: space-between; /* distribución horizontal */
  align-items: center; /* alineación vertical */
}
```

- **display: flex:** activa Flexbox.
- **flex-direction:** eje principal (row o column).
- **justify-content:** organiza en el eje principal (start, center, end, space-between, space-around).
- **align-items:** organiza en el eje cruzado (stretch, center, flex-start, flex-end).

Propiedades de los hijos flex:

```
.item {
  flex: 1; /* ocupa mismo espacio que los demás */
}
```

Tabla 1. Justify-content vs Align-items

Propiedad	Función	Ejemplo visual
justify-content	Ordena horizontalmente	[A B C]
align-items	Ordena verticalmente	A B C

Fuente: elaboración propia (2025)

```
header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

Imagen 7. Ejemplo práctico Flexbox

Fuente: elaboración propia (2025).

Resultado: el logo queda a la izquierda y el menú de navegación a la derecha, perfectamente alineados.

3. Grid: diseño bidimensional

Mientras que Flexbox organiza elementos en una sola dimensión (fila o columna), CSS Grid Layout lo hace en dos dimensiones (filas y columnas simultáneamente).

A	B	C	← Fila 1
D	E	F	← Fila 2

Imagen 8. Idea de Grid

Fuente: elaboración propia (2025).

Propiedades del contenedor grid:

```
.grid {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr; /* 3 columnas */  
  grid-template-rows: auto 200px; /* 2 filas */  
  gap: 10px;  
}
```

- **display: grid:** activa Grid.
- **grid-template-columns / rows:** define estructura.
- **fr:** unidad fraccional (divide espacio disponible).
- **gap:** separa filas y columnas.

Propiedades de los hijos grid:

```
.item-a {  
  grid-column: 1 / 3; /* ocupa 2 columnas */  
}  
.item-b {  
  grid-row: 1 / 3; /* ocupa 2 filas */  
}
```



Tabla 2. Flexbox vs Grid

Característica	Flexbox	Grid
Dirección	Una (fila o columna)	Dos (filas y columnas)
Complejidad	Más simple	Más potente
Uso típico	Menús, barras, alineación de items	Tableros, layouts completos de página

Fuente: elaboración propia (2025).

4. Ejemplo integrador

Código:

```
body {
  display: grid;
  grid-template-columns: 1fr 3fr;
  grid-template-rows: auto 1fr auto;
  gap: 10px;
  min-height: 100vh;
}

header {
  grid-column: 1 / 3;
  background: lightblue;
}

aside {
  background: lightgray;
}

main {
  background: white;
}

footer {
  grid-column: 1 / 3;
  background: lightblue;
}
```



Imagen 9. Layout con Grid

Fuente: elaboración propia (2025).

5. Buenas prácticas

1. Usar Flexbox para alineación local (por ejemplo, elementos dentro de un `header`).
2. Usar Grid para estructura global (por ejemplo, toda la página).



3. Definir breakpoints con media queries para reorganizar el Grid en móviles.
4. Combinar ambos: Grid para macroestructura + Flexbox para microdetalles.

6. Resumen

- El Box Model es la base: todo elemento HTML es una caja con contenido, padding, borde y margen.
- Flexbox simplifica la alineación y distribución en una dimensión.
- Grid permite organizar elementos en dos dimensiones y construir layouts completos.
- En la práctica: usa Grid para la estructura global y Flexbox dentro de cada sección.

Un dominio claro de estas herramientas permite pasar de páginas rígidas a interfaces flexibles, escalables y elegantes, requisito esencial en el desarrollo de plataformas modernas.

3.3. Media Queries y diseño responsivo

El diseño responsivo es la adaptación fluida y progresiva de una interfaz web a distintos tamaños y características de dispositivos. Su herramienta principal en CSS3 son las Media Queries, que permiten aplicar reglas de estilo condicionales dependiendo de propiedades del dispositivo o del navegador.

Esto asegura que una página web se vea bien en un celular de 5 pulgadas, en una tablet de 10 pulgadas, en un monitor de escritorio o incluso en un televisor 4K.

1. Concepto de Media Query

Una *Media Query* es una instrucción en CSS que define condiciones (queries) sobre el entorno de visualización. Si esas condiciones se cumplen, se aplican ciertos estilos.

Sintaxis básica

```
@media (condición) {  
  /* Estilos que se aplican solo si la condición es verdadera */  
}
```

Ejemplo:

```
@media (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```



Cuando el ancho de la pantalla es 600px o menos, el fondo de la página cambia a celeste.

2. Principio de Mobile First

En el diseño moderno se recomienda comenzar con estilos para móviles (pantallas pequeñas) y luego agregar reglas progresivas para pantallas más grandes.

Esto se logra usando min-width:

```
/* Estilos base: móvil */
body { font-size: 16px; }

/* Tablet y superiores */
@media (min-width: 768px) {
  body { font-size: 18px; }
}

/* Escritorio y superiores */
@media (min-width: 1200px) {
  body { font-size: 20px; }
}
```

ESTILOS BASE → min-width:768px → min-width:1200px
(Móvil) (Tablet) (Desktop)

Imagen 10. Esquema Mobile First

Fuente: elaboración propia (2025).

3. Propiedades comunes en Media Queries

Los navegadores permiten condicionar estilos en base a muchas características:

Tabla 1. Propiedades comunes en Media Queries

Propiedad	Descripción	Ejemplo
width / max-width	Ancho de la ventana	(max-width: 600px)
height / max-height	Alto de la ventana	(min-height: 800px)
orientation	Orientación del dispositivo	(orientation: landscape)
aspect-ratio	Relación ancho/alto	(min-aspect-ratio: 16/9)
prefers-color-scheme	Tema claro u oscuro del usuario	(prefers-color-scheme: dark)
prefers-reduced-motion	Preferencia de animaciones reducidas	(prefers-reduced-motion: reduce)

Fuente: elaboración propia (2025).

4. Ejemplo práctico de Media Queries

Código

```
.card {
  background: white;
  padding: 1rem;
  margin: 1rem;
}

/* Tablets */
@media (min-width: 768px) {
  .card {
    display: flex;
    flex-direction: row;
  }
}

/* Escritorio */
@media (min-width: 1200px) {
  .card {
    max-width: 1000px;
    margin: 2rem auto;
  }
}
```

En móviles, cada tarjeta se apila en columna.

En tablets, las tarjetas usan `flex-row`.

En escritorio, se centran con un ancho máximo.

```
MÓVIL:
[ Tarjeta ]
[ Tarjeta ]

TABLET:
[ Tarjeta | Tarjeta ]

ESCRITORIO:
[ Tarjeta | Tarjeta | Tarjeta ]
```

Imagen 11. Adaptación de tarjetas con Media Queries

Fuente: elaboración propia (2025).

5. Breakpoints comunes

No hay reglas universales, pero existen convenciones de la industria que sirven como referencia inicial.

Tabla 2. Breakpoints sugeridos

Dispositivo	Ancho	Media Query recomendada
Móvil pequeño	0-480px	@media (max-width: 480px)
Móvil grande	481-767px	@media (min-width: 481px)
Tablet	768-1023px	@media (min-width: 768px)
Laptop	1024-1199px	@media (min-width: 1024px)
Desktop	1200px+	@media (min-width: 1200px)

Fuente: elaboración propia (2025).

6. Media Queries avanzadas

CSS3 va más allá del tamaño de pantalla e incluye preferencias del usuario:

1. Tema oscuro/claro:

```
@media (prefers-color-scheme: dark) {
  body { background: #121212; color: white; }
}
```

2. Reducción de movimiento:

```
@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; }
}
```

Esto respeta la accesibilidad y evita mareos en personas sensibles.

```
LIGHT MODE: fondo blanco, texto negro
DARK MODE:  fondo negro, texto blanco
```

Imagen 12. Ejemplo de tema claro/oscuro con Media Queries

Fuente: elaboración propia (2025).

7. Buenas prácticas en diseño responsivo

1. No uses demasiados breakpoints: elige los necesarios, basados en contenido, no en dispositivos específicos.
2. Empieza simple: diseña un flujo vertical para móvil, luego mejora en pantallas más grandes.
3. Usa unidades relativas: `rem`, `%`, `fr`, `vw` escalan mejor que `px`.
4. Prueba en distintos dispositivos: simuladores y navegadores ofrecen herramientas para inspeccionar tamaños.
5. Piensa en accesibilidad: Media Queries como `prefers-reduced-motion` son tan importantes como las de tamaño.

8. Caso: menú de navegación

- En móvil, el menú aparece oculto en un botón “hamburguesa”.
- En tablet, los ítems se muestran en fila pero con texto pequeño.
- En escritorio, el menú ocupa toda la barra superior con espaciado amplio.

Código

```
nav ul {
  display: none;
}

/* Tablet */
@media (min-width: 768px) {
  nav ul {
    display: flex;
    gap: 1rem;
  }
}

/* Desktop */
@media (min-width: 1200px) {
  nav ul {
    gap: 2rem;
    font-size: 1.2rem;
  }
}
```

MÓVIL: ≡
 TABLET: [Home] [Blog] [Contacto]
 DESKTOP: [Home | Blog | Contacto]

Imagen 13. Menú responsivo con Media Queries

Fuente: elaboración propia (2025).

9. Resumen

- El diseño responsivo asegura que las páginas se adapten a todos los dispositivos.
- Media Queries son la base de esta adaptabilidad, permitiendo condicionar estilos según ancho, alto, orientación o incluso preferencias del usuario.
- La estrategia Mobile First es la más recomendada.
- Al aplicar buenas prácticas, los proyectos se vuelven más inclusivos, accesibles y escalables.



3.4. Frameworks CSS modernos: Bootstrap y Tailwind

En la actualidad, el desarrollo de interfaces web no solo depende de conocer CSS3 puro, sino también de aprovechar frameworks CSS modernos, que aceleran el trabajo y garantizan coherencia visual.

Dos de los más utilizados son:

- **Bootstrap**, creado en 2011 por desarrolladores de Twitter.
- **Tailwind CSS**, surgido en 2017 con un enfoque *utility-first* (clases pequeñas reutilizables).

Ambos permiten crear sitios adaptables, accesibles y con buenas prácticas de diseño, pero tienen filosofías distintas.

1. ¿Qué es un framework CSS?

Un framework CSS es un conjunto predefinido de reglas, componentes y utilidades que simplifica el diseño de páginas web.

Ventajas principales:

- Ahorra tiempo: ya trae botones, formularios, rejillas y tipografía.
- Consistencia: mantiene un estilo uniforme en todo el proyecto.
- Adaptabilidad: casi todos incluyen un sistema responsivo listo.

2. Bootstrap: rejilla y componentes listos

Bootstrap es el framework más popular del mundo, usado en millones de sitios. Se centra en ofrecer una rejilla responsiva (grid system) y una gran cantidad de componentes preconstruidos.

12 columnas → puedes combinar: [4 4 4] → tres iguales [6 6] → dos iguales [3 9] → una pequeña y otra grande
--

Imagen 14. Esquema de la rejilla de Bootstrap

Fuente: elaboración propia (2025).



Ejemplo de uso de la rejilla

```
<div class="container">
  <div class="row">
    <div class="col-4">Columna 1</div>
    <div class="col-4">Columna 2</div>
    <div class="col-4">Columna 3</div>
  </div>
</div>
```

Con pocas clases, se logra un layout adaptable.

Tabla 1. Ventajas y desventajas de Bootstrap

Ventajas	Desventajas
Gran comunidad y documentación	Puede sentirse rígido si no se personaliza
Componentes listos: menús, botones, modales	CSS algo pesado si no se optimiza
Rejilla responsiva fácil de usar	El diseño puede lucir genérico si no se adapta

Fuente: elaboración propia (2025).

3. Tailwind CSS: utilidades primero

A diferencia de Bootstrap, Tailwind CSS no trae componentes listos. Su enfoque es utility-first, es decir, ofrece clases pequeñas y específicas para estilos (colores, márgenes, tipografía, etc.), que el desarrollador combina para crear interfaces personalizadas.

Ejemplo en Tailwind

```
<div class="max-w-sm mx-auto bg-white rounded-xl shadow-md overflow-hidden p-4">
  <h1 class="text-xl font-bold text-gray-900">Título</h1>
  <p class="text-gray-600">Este es un párrafo con Tailwind.</p>
</div>
```

Aquí no hay CSS adicional: todo se define con clases.

Bootstrap:	<code><div class="card"> ... </div></code>
Tailwind:	<code><div class="bg-white p-4 shadow-md rounded-lg"> ... </div></code>

Imagen 15. Filosofía Utility-first

Fuente: elaboración propia (2025).

Tabla 2. Ventajas y desventajas de Tailwind

Ventajas	Desventajas
Gran flexibilidad, diseños únicos	Curva de aprendizaje inicial
Código más ligero (si se usa purge)	Puede verse sobrecargado de clases
Excelente integración con React, Vue, etc.	No trae componentes listos



Fuente: elaboración propia (2025).

4. Comparación entre Bootstrap y Tailwind

Bootstrap → Componentes listos (rápido)
Tailwind → Estilos utilitarios (flexible)

Imagen 16. Comparación visual

Fuente: elaboración propia (2025).

Tabla 3. Bootstrap vs Tailwind

Característica	Bootstrap	Tailwind
Filosofía	Componentes predefinidos	Clases utilitarias
Velocidad inicial	Muy rápida	Requiere aprender combinación de clases
Personalización	Limitada sin sobrescribir	Muy alta
Peso inicial	Mayor	Menor (con purge)
Comunidad	Muy amplia	En rápido crecimiento

5. Ejemplo comparativo

Bootstrap

```
<button class="btn btn-primary">Enviar</button>
```

Tailwind

```
<button class="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2
px-4 rounded">
  Enviar
</button>
```

Bootstrap ofrece un botón listo, mientras que Tailwind permite definirlo con clases utilitarias.

6. Buenas prácticas con frameworks

1. No depender al 100%: aprende primero CSS puro.
2. Personaliza: cambia colores, tipografías y variables para que tu sitio no se vea genérico.
3. Optimiza: en producción, elimina clases o componentes que no uses.
4. Accesibilidad: revisa contraste, foco visible y compatibilidad con lectores de pantalla.
5. Combina lo mejor: puedes usar Tailwind para la base y Bootstrap para algunos componentes si el proyecto lo permite.



7. Caso: página de inicio

Un estudiante debe diseñar una landing page con un encabezado, un bloque central y un pie de página.

- Con **Bootstrap**, puede usar `container`, `row` y `col` para organizar el layout, y añadir un `navbar` ya construido.
- Con **Tailwind**, define `grid`, `gap-4`, `text-center` y colores directamente en las clases HTML, logrando un estilo único.

Bootstrap:	Layout estándar con componentes prehechos
Tailwind:	Layout único con clases utilitarias

Imagen 17. Resultado visual

Fuente: elaboración propia (2025).

8. Resumen

- Los frameworks CSS modernos son herramientas que aceleran el diseño.
- **Bootstrap** es ideal para empezar rápido, con componentes listos y una rejilla responsiva.
- **Tailwind** ofrece máxima flexibilidad con un enfoque utility-first, favoreciendo la personalización.
- La elección depende del proyecto: rapidez y estandarización con Bootstrap, o flexibilidad y unicidad con Tailwind.

Título del enlace relacionado: Bootstrap vs Tailwind: comparación práctica (CSS Tricks)

Descripción del enlace relacionado: Artículo que compara los frameworks CSS más populares, Bootstrap y Tailwind, destacando sus diferencias en filosofía, ventajas y usos prácticos en proyectos modernos.

Enlace: <https://css-tricks.com/tailwind-vs-bootstrap/>

Dominar estos frameworks es fundamental para un desarrollador moderno, pero siempre recordando que la base sigue siendo **CSS3 puro**.

Referencias citadas en la Clase 3.

- Frain, B. (2020). Responsive Web Design with HTML5 and CSS. Packt Publishing.
- Cederholm, D. (2018). CSS Grid Layout. A Book Apart.
- W3C. (2023). CSS Cascading Style Sheets Level 3 (CSS3) Specification. <https://www.w3.org/TR/css-2023/>



- Mozilla Developer Network (MDN). (2024). CSS: Cascading Style Sheets. <https://developer.mozilla.org/es/docs/Web/CSS>
- W3C. (2018). Media Queries Level 4. <https://www.w3.org/TR/mediaqueries-4/>.

Definición de términos

- **Box model**: concepto fundamental de CSS que describe cómo los navegadores calculan el espacio que ocupa cada elemento HTML, considerando contenido, padding, borde y márgenes.
 - **Media Query**: regla condicional en CSS3 que aplica estilos en función de características del dispositivo (ancho, alto, orientación, preferencias de usuario), permitiendo el diseño responsivo.
- Recursos de profundización

Recurso: *Profundizacion1_clase3.docx*

Diseño web adaptable

Damián Nicolalde

danicolalde@puce.edu.ec

Web móvil

- La introducción en el mercado del iPhone en el año 2007 cambió el mundo para siempre. Hasta ese momento, solo disponíamos de móviles con pantallas pequeñas y sin la capacidad suficiente de descargar y mostrar sitios web.
- Pero cuando el iPhone apareció en el mercado, los sitios web con un diseño fijo como estos no se podían leer en una pantalla tan pequeña. La primera solución fue desarrollar dos sitios web separados, uno para ordenadores y otro para iPhones, pero la introducción en el mercado de nuevos dispositivos, como el iPad en el año 2010, forzó nuevamente a los desarrolladores a buscar mejores alternativas.
- La solución final debía incluir elementos flexibles, además de la posibilidad de adaptar el diseño a cada dispositivo. Así es como nació lo que hoy conocemos como diseño web adaptable, o por su nombre en inglés *responsive web design*.
- El diseño web adaptable es una técnica que combina diseños flexibles con una herramienta provista por CSS llamada *Media Queries* (consulta de medios), que nos permite detectar el tamaño de la pantalla y realizar los cambios necesarios para adaptar el diseño a cada situación.

Media Queries

- Una Media Query es una regla reservada en CSS que se incorporó con el propósito de permitir a los desarrolladores detectar el medio en el que se muestra el documento. Por ejemplo, usando Media Queries podemos determinar si el documento se muestra en un monitor o se envía a una impresora, y asignar los estilos apropiados para cada caso. Para este propósito, las Media Queries ofrecen las siguientes palabras clave.
- **all**—Las propiedades se aplican en todos los medios.
- **print**—Las propiedades se aplican cuando la página web se envía a una impresora.
- **screen**—Las propiedades se aplican cuando la página web se muestra en una pantalla color.
- **speech**—Las propiedades se aplican cuando la página web se procesa por un sintetizador

Media Queries

- Lo que hace que las Media Queries sean útiles para el diseño web es que también pueden detectar el tamaño del medio. Con las Media Queries podemos detectar el tamaño del área de visualización (la parte de la ventana del navegador donde se muestran nuestras páginas web) y definir diferentes reglas CSS para cada dispositivo. Existen varias palabras clave que podemos usar para detectar estas características. Las siguientes son las que más se usan para desarrollar un sitio web con diseño web adaptable.

Media Queries

- **width**—Esta palabra clave determina el ancho en que se aplican las propiedades.
- **height**—Esta palabra clave determina la altura a la que se aplican las propiedades.
- **min-width**—Esta palabra clave determina el ancho mínimo desde el cual que se aplican las propiedades.
- **max-width**—Esta palabra clave determina el ancho máximo hasta el cual que se aplican las propiedades.
- **aspect-ratio**—Esta palabra clave determina la proporción en la cual que se aplican las propiedades.
- **orientation**—Esta palabra clave determina la orientación en la cual que se aplican las propiedades. Los valores disponibles son portrait (vertical) y landscape (horizontal).
- **resolution**—Esta palabra clave determina la densidad de píxeles en la cual que se aplican las propiedades. Acepta valores en puntos por pulgada (dpi), puntos por centímetro (dpcm) o por proporción en píxeles (dppx). Por ejemplo, para detectar una pantalla de tipo retina con una escala de 2, podemos usar el valor 2dppx.

Media Queries

- Existen dos maneras de declarar Media Queries: desde el documento usando el atributo media del elemento <link>, o desde la hoja de estilo con la regla @media. Cuando usamos el elemento <link>, podemos seleccionar el archivo CSS con la hoja de estilo que queremos cargar para una configuración específica. Por ejemplo, el siguiente documento carga dos archivos CSS, uno que contiene los estilos generales que aplicaremos en toda situación y medios, y otro con los estilos requeridos para presentar la página en un dispositivo de pantalla pequeña (480 píxeles de ancho o menos).

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <title>Este texto es el título del documento</title>
5      <meta charset="utf-8">
6      <meta name="description" content="Este es un documento HTML5">
7      <meta name="keywords" content="HTML, CSS, JavaScript">
8      <link rel="stylesheet" href="adaptatabletodos.css">
9      <link rel="stylesheet" media="(max-width: 480px)" href="adaptatablecelulares.css">
10 </head>
11 <body>
12     <section>
13         <div>
14             <article>
15                 <h1>Título Primer Artículo</h1>
16                 
17             </article>
18         </div>
19     </section>
20     <aside>
21         <div>
22             <h1>Información</h1>
23             <p>Cita del artículo uno</p>
24             <p>Cita del artículo dos</p>
25         </div>
26     </aside>
27     <div class="recuperar"></div>
28 </body>
29 </html>

```

```

1  body {
2      margin: 0px;
3      padding: 0px;
4      background-color: #990000;
5  }

```

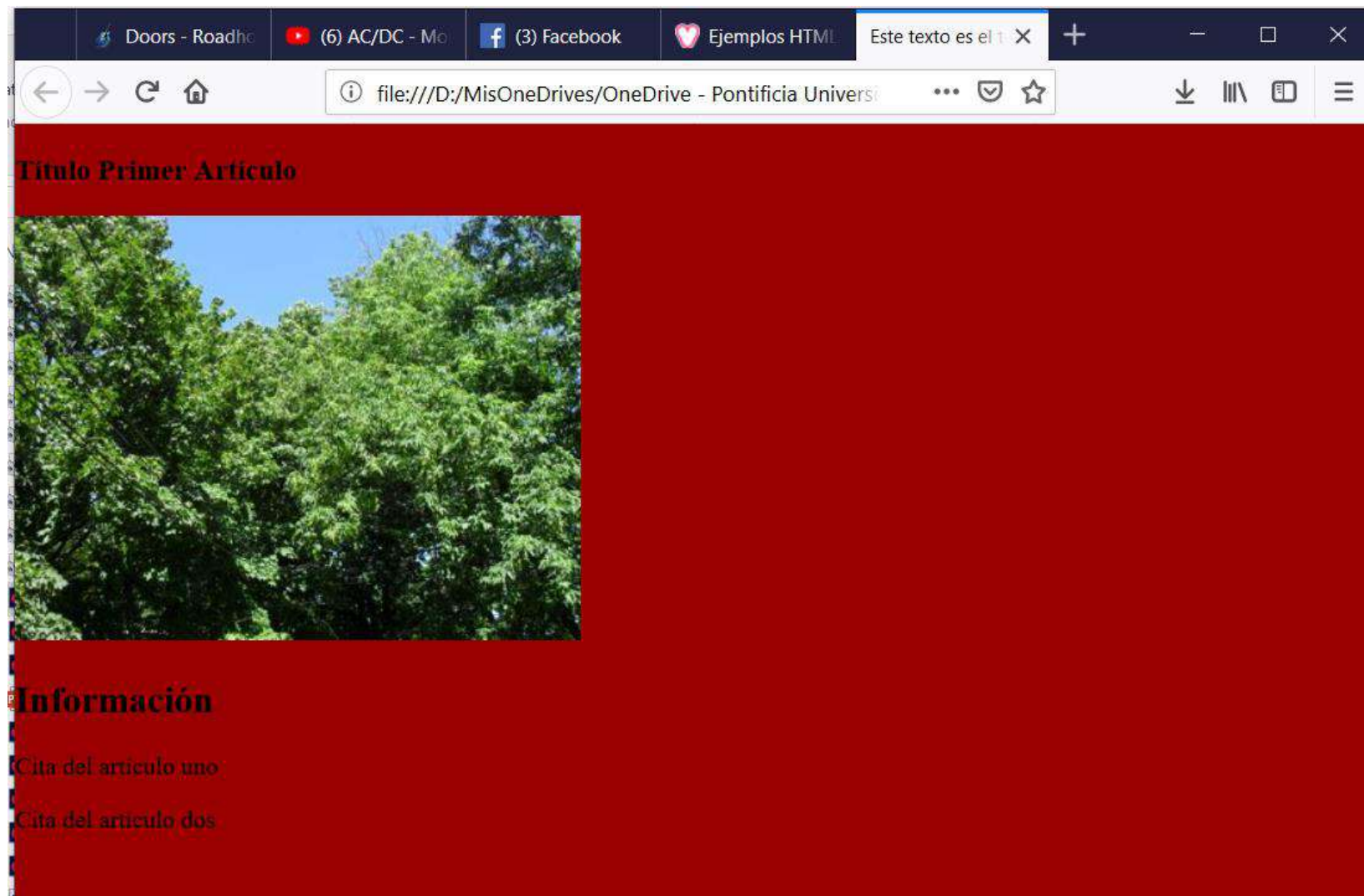
```

1  body {
2      background-color: #3333FF;
3  }

```

Cuando el navegador lee este documento, primero carga el archivo adaptatabletodos.css y luego, si el ancho del área de visualización es de 480 píxeles o inferior, carga el archivo adaptatablecelulares.css y aplica los estilos al documento.

CSS



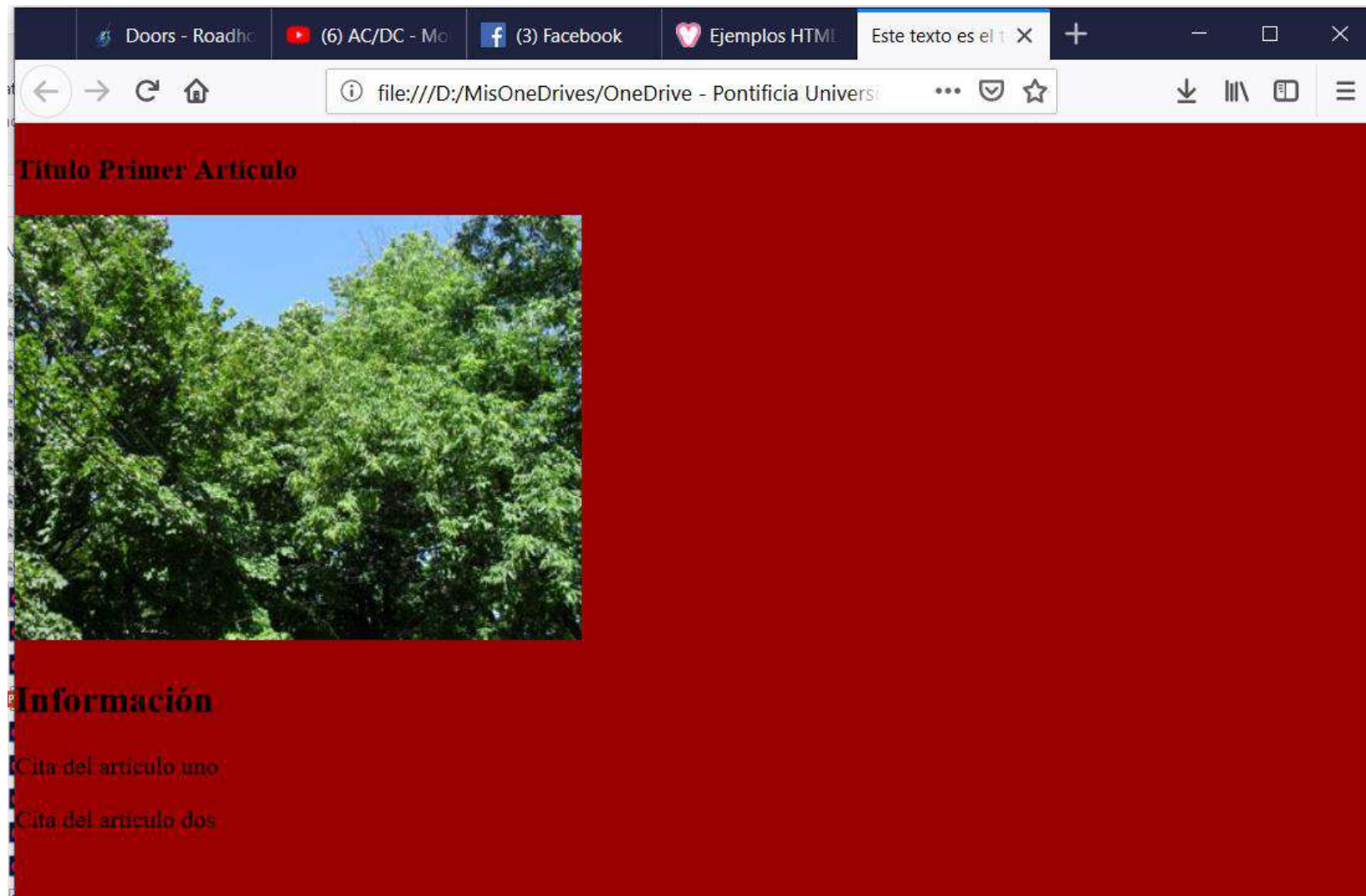
Media Queries

- Con el elemento `<link>` y el atributo `media` podemos cargar diferentes hojas de estilo para cada situación, pero cuando solo se modifican unas pocas propiedades, podemos incluir todas en la misma hoja de estilo y definir las Media Queries con la regla `@media`, tal como ilustra el siguiente ejemplo.
- (primero mirar la siguiente lámina) La Media Query se declara con la regla `@media` seguida de las palabras clave y valores que definen las características del medio y las propiedades entre llaves. Cuando el navegador lee esta hoja de estilo, asigna las propiedades de la regla `body` al elemento `<body>` y luego, si el ancho del área de visualización es de 480 píxeles o inferior, cambia el color de fondo del elemento `<body>` a `#3333FF` (azul).

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <title>Este texto es el título del documento</title>
5      <meta charset="utf-8">
6      <meta name="description" content="Este es un documento HTML5">
7      <meta name="keywords" content="HTML, CSS, JavaScript">
8      <link rel="stylesheet" href="adaptatabletodos2.css">
9  </head>
10 <body>
11     <section>
12         <div>
13             <article>
14                 <h1>Título Primer Artículo</h1>
15                 
16             </article>
17         </div>
18     </section>
19     <aside>
20         <div>
21             <h1>Información</h1>
22             <p>Cita del artículo uno</p>
23             <p>Cita del artículo dos</p>
24         </div>
25     </aside>
26     <div class="recuperar"></div>
27 </body>
28 </html>
```



```
1  body {
2      margin: 0px;
3      padding: 0px;
4      background-color: #990000;
5  }
6  @media (max-width: 480px) {
7      body {
8          background-color: #3333FF;
9      }
10 }
```



Puntos de interrupción

- Debemos establecer puntos máximos o mínimos en los cuales el diseño debe cambiar significativamente, y utilizar el modelo de caja para adaptar el tamaño de los elementos al espacio disponible cuando el ancho de la pantalla se encuentra entre estos puntos. Estos puntos máximos y mínimos se especifican con las palabras clave max-width y min-width, y se llaman *puntos de interrupción* (*breakpoints* en inglés).
- Los puntos de interrupción son aquellos en los que el diseño de una página web requiere cambios significativos para poder adaptarse al tamaño de la pantalla. Estos puntos son los tamaños en los cuales la flexibilidad de los elementos no es suficiente para ajustar las cajas al espacio disponible y tenemos que mover los elementos de un lugar a otro, o incluso excluirllos del diseño para que nuestro sitio web siga siendo legible. No existen estándares establecidos que podamos seguir para determinar estos puntos; todo depende de nuestro diseño. Por ejemplo, podríamos decidir que cuando el ancho del área de visualización es igual o menor que 480 píxeles tenemos que usar solo una columna para presentar la información, pero esto solo es posible si el diseño original contiene más de una columna. Los valores más comunes son 320, 480, 768, y 1024.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <title>Este texto es el título del documento</title>
5      <meta charset="utf-8">
6      <meta name="description" content="Este es un documento HTML5">
7      <meta name="keywords" content="HTML, CSS, JavaScript">
8      <link rel="stylesheet" href="adaptabledos3.css">
9  </head>
10 <body>
11     <section>
12         <div>
13             <article>
14                 <h1>Título Primer Artículo</h1>
15                 
16             </article>
17         </div>
18     </section>
19     <aside>
20         <div>
21             <h1>Información</h1>
22             <p>Cita del artículo uno</p>
23             <p>Cita del artículo dos</p>
24         </div>
25     </aside>
26     <div class="recuperar"></div>
27 </body>
28 </html>
```



```
1  body {
2      margin: 0px;
3      padding: 0px;
4      background-color: #990000;
5  }
6  @media (max-width: 1024px) {
7      body {
8          background-color: #3333FF;
9      }
10 }
11 @media (max-width: 768px) {
12     body {
13         background-color: #FF33FF;
14     }
15 }
16 @media (max-width: 480px) {
17     body {
18         background-color: #339933;
19     }
20 }
21 @media (max-width: 320px) {
22     body {
23         background-color: #CCCCCC;
24     }
25 }
```

Miríadax -

Este texto es el título del documento

file:///D:/MisOneDrives/OneDrive - Pontificia Universidad Católica del Ecu

Título Primer Artículo



Información

Cita del artículo uno

Cita del artículo dos

Miríadax -

Este texto es el título del documento

file:///D:/MisOneDrives/OneDrive - Pontific

Título Primer Artículo



Información

Cita del artículo uno

Cita del artículo dos

Miríadax -

Este texto es el título del documento

file:///D:/MisOne

Título Primer Artículo



Información

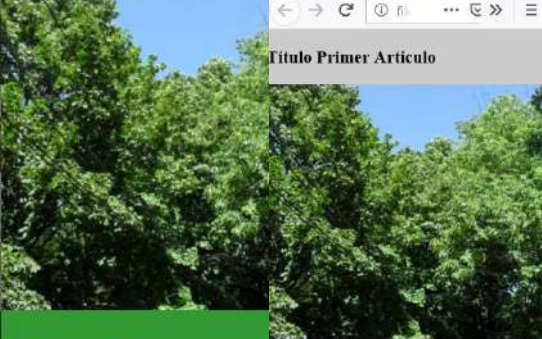
Cita del artículo uno

Cita del artículo dos

Este texto es el título del documento

file:///D:/MisOne

Título Primer Artículo



Información

Cita del artículo uno

Cita del artículo dos

Este texto es el título del documento

file:///D:/MisOne

Título Primer Artículo



Información

Cita del artículo uno

Cita del artículo dos

Área de visualización

- El área de visualización (*viewport* en inglés) es la parte de la ventana del navegador donde se muestran nuestras páginas web. Debido a la relación entre la ventana y el área de visualización, ambos deberían presentar el mismo tamaño en dispositivos móviles, pero esto no se cumple en todos los casos. Por defecto, algunos dispositivos asignan un ancho de 980 píxeles al área de visualización, sin importar su tamaño real o el tamaño real de la pantalla. Esto significa que las Media Queries de nuestras hojas de estilo verán un ancho de 980 píxeles cuando en realidad el tamaño del área de visualización es totalmente diferente.
- Para normalizar esta situación y forzar al navegador a definir el tamaño del área de visualización igual al tamaño real de la pantalla, tenemos que declarar el elemento `<meta>` en la cabecera de nuestros documentos con el nombre `viewport` y valores que determinan el ancho y la escala que queremos ver. Los dos valores requeridos son `width` e `initial-scale` para declarar el ancho del área de visualización y su escala. El siguiente ejemplo ilustra cómo debemos configurar el elemento `<meta>` para pedirle al navegador que defina el tamaño y la escala reales de la pantalla del dispositivo.

```
<head>
  <title>Este texto es el título del documento</title>
  <meta charset="utf-8">
  <meta name="description" content="Este es un documento HTML5">
  <meta name="keywords" content="HTML, CSS, JavaScript">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="stylesheet" href="adaptatabletodos.css">
</head>
```

Flexibilidad

- En un diseño web adaptable tenemos que permitir que nuestras páginas sean flexibles, de modo que los elementos se adapten al espacio disponible cuando el ancho del área de visualización se encuentra entre los puntos de interrupción. Esto se puede lograr con el modelo de caja flexible: solo tenemos que crear contenedores flexibles y declarar sus tamaños como flexibles con la propiedad flex, pero este modelo de caja no lo reconocen algunos navegadores y, por lo tanto, la mayoría de los desarrolladores aún implementan el modelo de caja tradicional.
- Aunque el modelo de caja tradicional no fue diseñado para trabajar con elementos flexibles, podemos volver flexibles los elementos declarando sus tamaños en porcentaje. Cuando declaramos el tamaño de un elemento en porcentaje, el navegador calcula el tamaño real en píxeles a partir del tamaño de su contenedor. Por ejemplo, si asignamos un ancho de 80 % a un elemento que se encuentra dentro de otro elemento con un tamaño de 1000 píxeles, el ancho del elemento será de 800 píxeles (80 % de 1000). Debido a que el tamaño de los elementos cambia cada vez que lo hace el tamaño de sus contenedores, estos se vuelven flexibles.
- El siguiente ejemplo crea dos columnas con los elementos `<section>` y `<aside>` y declara sus anchos en porcentaje para hacerlos flexibles.

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <title>Este texto es el título del documento</title>
5      <meta charset="utf-8">
6      <meta name="description" content="Este es un documento HTML5">
7      <meta name="keywords" content="HTML, CSS, JavaScript">
8      <meta name="viewport" content="width=device-width, initial-scale=1">
9      <link rel="stylesheet" href="adaptatabletodos4.css">
10 </head>
11 <body>
12     <section>
13         <div>
14             <article>
15                 <h1>Título Primer Artículo</h1>
16                 
17             </article>
18         </div>
19     </section>
20     <aside>
21         <div>
22             <h1>Información</h1>
23             <p>Cita del artículo uno</p>
24             <p>Cita del artículo dos</p>
25         </div>
26     </aside>
27     <div class="recuperar">Este es mi texto</div>
28 </body>
29 </html>

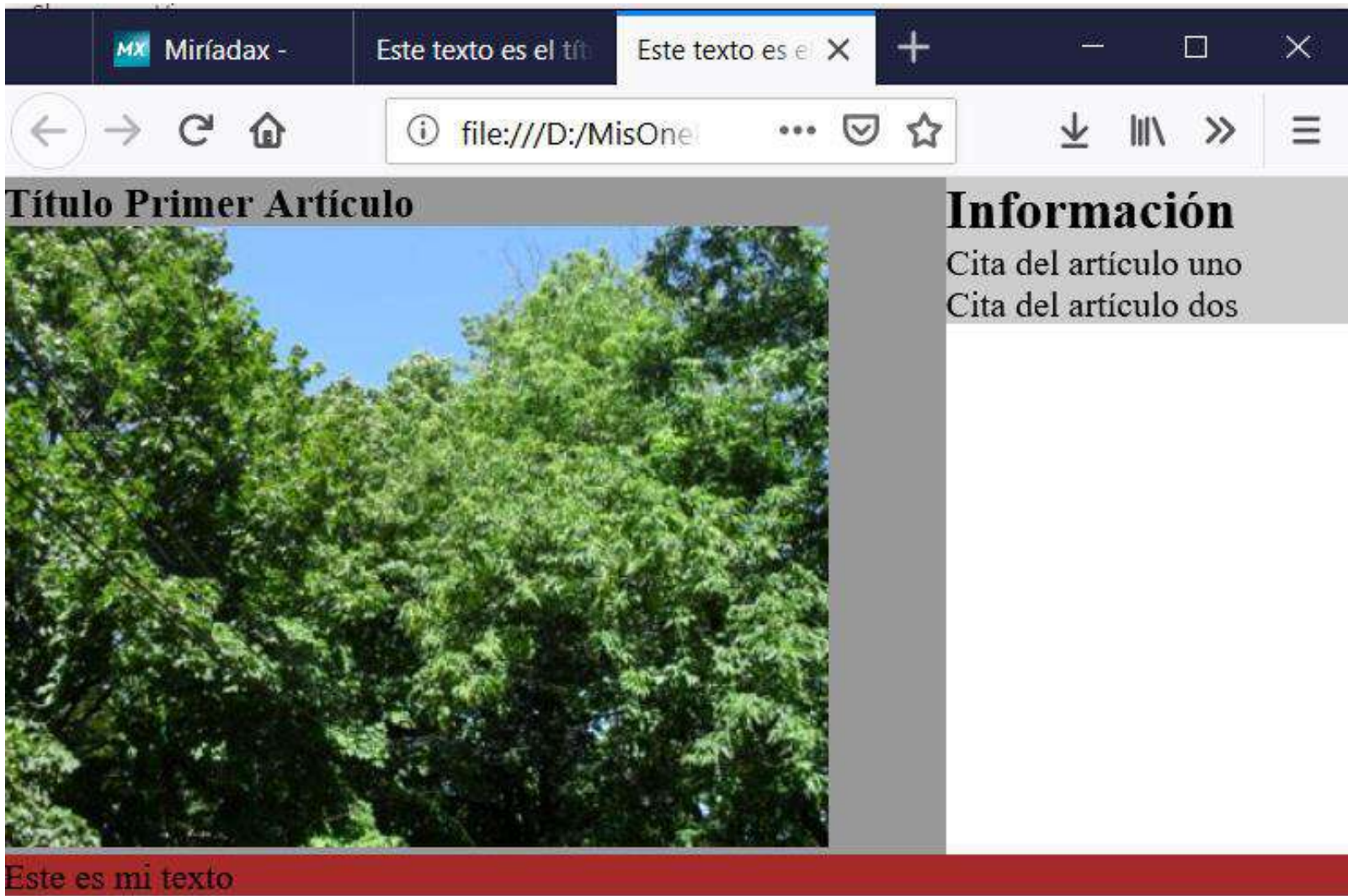
```

```

1  * {
2      margin: 0px;
3      padding: 0px;
4  }
5  section {
6      float: left;
7      width: 70%;
8      background-color: #999999;
9  }
10 aside {
11     float: left;
12     width: 30%;
13     background-color: #CCCCCC;
14 }
15 .recuperar {
16     clear: both;
17     background-color: brown;
18 }

```

Cada vez que declaramos el ancho del elemento en porcentajes, tenemos que asegurarnos de que la suma total de los valores no exceda el 100 %.



```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <title>Este texto es el título del documento</title>
5      <meta charset="utf-8">
6      <meta name="description" content="Este es un documento HTML5">
7      <meta name="keywords" content="HTML, CSS, JavaScript">
8      <meta name="viewport" content="width=device-width, initial-scale=1">
9      <link rel="stylesheet" href="adaptatabletodos5.css">
10 </head>
11 <body>
12 <section>
13 <div>
14 <article>
15 <h1>Título Primer Artículo</h1>
16 
17 </article>
18 </div>
19 </section>
20 <aside>
21 <div>
22 <h1>Información</h1>
23 <p>Cita del artículo uno</p>
24 <p>Cita del artículo dos</p>
25 </div>
26 </aside>
27 <div class="recuperar">Este es mi texto</div>
28 </body>
29 </html>
```

```
1  * {
2      margin: 0px;
3      padding: 0px;
4  }
5  section {
6      float: left;
7      width: 61%;
8      padding: 2%;
9      margin-right: 5%;
10     background-color: #999999;
11 }
12 aside {
13     float: left;
14     width: 26%;
15     padding: 2%;
16     background-color: #CCCCCC;
17 }
18 .recuperar {
19     padding: 5%;
20     clear: both;
21     background-color: brown;
22 }
```

MX

Mirfada

Este texto es

Este texto es

Este texto X

+

—

□

×

←

→

↺

🏠

📄 file:///D:/MisOne

⋮

🛡️

☆

⬇️

📖

➡️

☰

Título Primer Artículo



Información

Cita del artículo uno

Cita del artículo dos

Este es mi texto

Box-sizing

- **box-sizing**—Esta propiedad nos permite decidir cómo se calculará el tamaño de un elemento y forzar a los navegadores a incluir el relleno y el borde en el tamaño declarado por la propiedad `width`. Los valores disponibles son `content-box` y `border-box`.
- Por defecto, el valor de la propiedad `box-sizing` se declara como `content-box`, lo cual significa que el navegador agregará los valores de las propiedades `padding` y `border` al tamaño especificado por la propiedad `width`. Si en cambio asignamos a esta propiedad el valor `border-box`, el navegador incluye el relleno y el borde como parte del ancho, y entonces disponemos de otras opciones como combinar anchos definidos en porcentaje con rellenos en píxeles, tal como se muestra en el siguiente ejemplo.

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <title>Este texto es el título del documento</title>
5   <meta charset="utf-8">
6   <meta name="description" content="Este es un documento HTML5">
7   <meta name="keywords" content="HTML, CSS, JavaScript">
8   <meta name="viewport" content="width=device-width, initial-scale=1">
9   <link rel="stylesheet" href="adaptatabletodos6.css">
10 </head>
11 <body>
12   <section>
13     <div>
14       <article>
15         <h1>Título Primer Artículo</h1>
16         
17       </article>
18     </div>
19   </section>
20   <aside>
21     <div>
22       <h1>Información</h1>
23       <p>Cita del artículo uno</p>
24       <p>Cita del artículo dos</p>
25     </div>
26   </aside>
27   <div class="recuperar">Este es mi texto</div>
28 </body>
29 </html>
```

```
1 * {
2   margin: 0px;
3   padding: 0px;
4 }
5 section {
6   float: left;
7   width: 65%;
8   padding: 20px;
9   margin-right: 5%;
10  background-color: #999999;
11  box-sizing: border-box;
12 }
13 aside {
14   float: left;
15   width: 30%;
16   padding: 20px;
17   background-color: #CCCCCC;
18   box-sizing: border-box;
19 }
20 .recuperar {
21   clear: both;
22   background-color: brown;
23   padding: 5px;
24 }
```



Fijo y flexible

- En los anteriores ejemplos hemos visto cómo definir cajas flexibles con rellenos fijos, pero muchas veces nos encontraremos con la necesidad de tener que combinar columnas completas con valores flexibles y fijos. Nuevamente, esto es muy fácil de lograr con el modelo de caja flexible; simplemente tenemos que crear un contenedor flexible, declarar el tamaño de los elementos que queremos que sean flexibles con la propiedad `flex` y aquellos que queremos que tengan un tamaño fijo con la propiedad `width`. Pero el modelo de caja tradicional no se diseñó para realizar esta clase de tareas. Por suerte existen algunos trucos que podemos implementar para lograr este propósito. La alternativa más popular es la de declarar un relleno en la columna flexible con el que hacemos sitio para ubicar la columna de tamaño fijo y agregar un margen negativo que desplaza esta columna al espacio vacío dejado por el relleno. Las siguientes son las reglas que necesitamos para declarar un ancho flexible para el elemento `<section>` y un ancho fijo para el elemento `<aside>` de nuestro documento.

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <title>Este texto es el título del documento</title>
5      <meta charset="utf-8">
6      <meta name="description" content="Este es un documento HTML5">
7      <meta name="keywords" content="HTML, CSS, JavaScript">
8      <meta name="viewport" content="width=device-width, initial-scale=1">
9      <link rel="stylesheet" href="adaptatableops7.css">
10 </head>
11 <body>
12     <section>
13         <div>
14             <article>
15                 <h1>Título Primer Artículo</h1>
16                 
17             </article>
18         </div>
19     </section>
20     <aside>
21         <div>
22             <h1>Información</h1>
23             <p>Cita del artículo uno</p>
24             <p>Cita del artículo dos</p>
25         </div>
26     </aside>
27     <div class="recuperar">Este es mi texto</div>
28 </body>
29 </html>

```

```

1  * {
2      margin: 0px;
3      padding: 0px;
4  }
5  section {
6      float: left;
7      width: 100%;
8      padding-right: 260px;
9      margin-right: -240px;
10     box-sizing: border-box;
11 }
12 section > div {
13     padding: 20px;
14     background-color: #999999;
15 }
16 aside {
17     float: left;
18     width: 240px;
19     padding: 20px;
20     background-color: #CCCCCC;
21     box-sizing: border-box;
22 }
23 .recuperar {
24     clear: both;
25     background-color: brown;
26     padding: 5px;
27 }

```

Las reglas asignan un ancho de 100 % y un relleno del lado derecho de 260 píxeles al elemento <section>. Como usamos la propiedad box-sizing para incluir el relleno en el valor de la propiedad width, el contenido del elemento ocupará solo el lado izquierdo, dejando un espacio vacío a la derecha donde podemos ubicar la columna con tamaño fijo. Finalmente, para mover la columna creada por el elemento <aside> a este espacio, declaramos un margen negativo de 240 píxeles en el lado derecho del elemento <section>.



Título Primer Artículo

Información

Cita del artículo uno
Cita del artículo dos



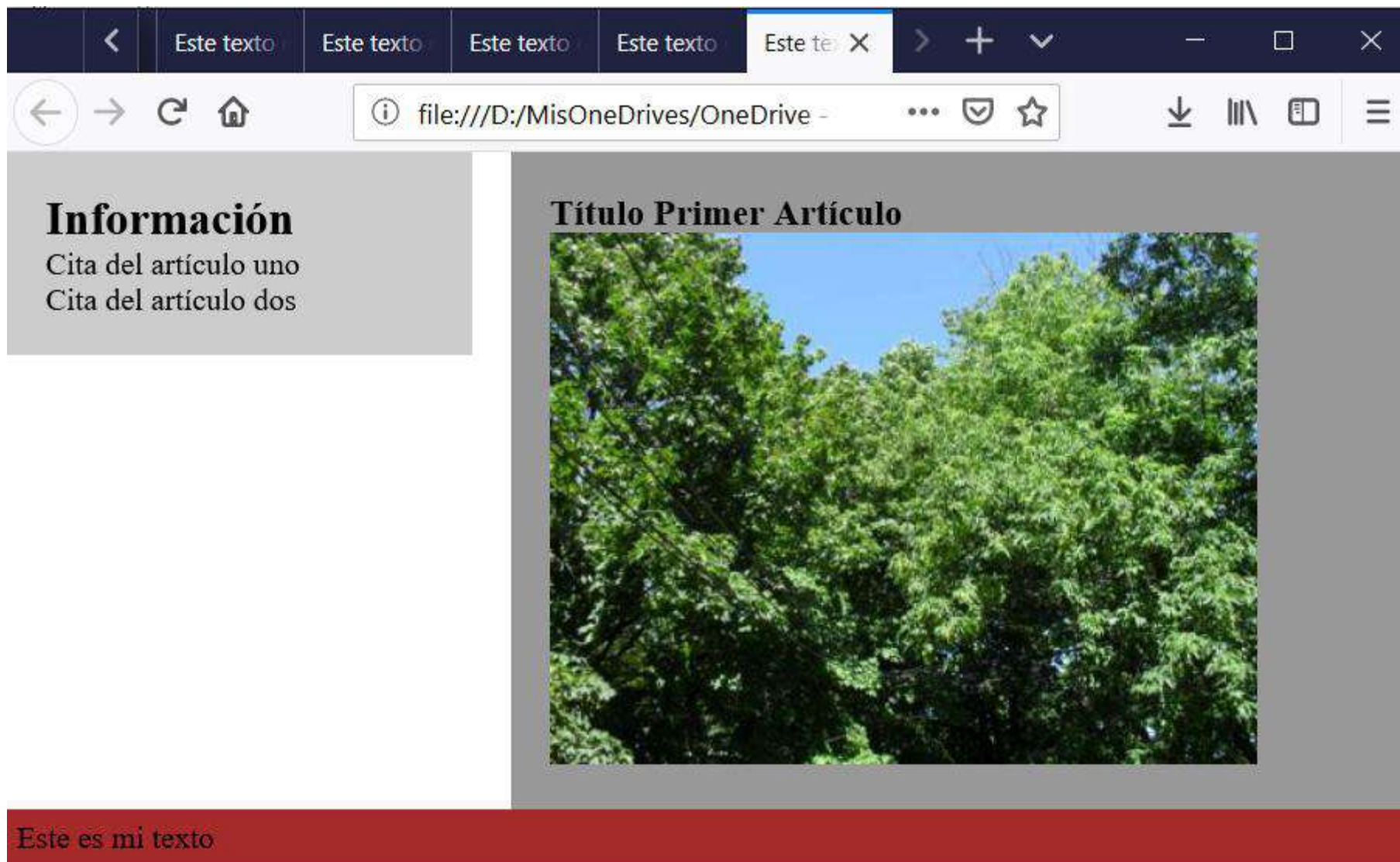
Este es mi texto

Fijo y flexible

- Si queremos ubicar la columna de tamaño fijo a la izquierda en lugar de a la derecha, el proceso es el mismo, pero tenemos que declarar la columna izquierda debajo de la columna derecha en el código y luego configurar sus posiciones con la propiedad float, tal como hacemos en el siguiente ejemplo.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <title>Este texto es el título del documento</title>
5      <meta charset="utf-8">
6      <meta name="description" content="Este es un documento HTML5">
7      <meta name="keywords" content="HTML, CSS, JavaScript">
8      <meta name="viewport" content="width=device-width, initial-scale=1">
9      <link rel="stylesheet" href="adaptatabletodos8.css">
10 </head>
11 <body>
12 <section>
13 <div>
14 <article>
15 <h1>Título Primer Artículo</h1>
16 
17 </article>
18 </div>
19 </section>
20 <aside>
21 <div>
22 <h1>Información</h1>
23 <p>Cita del artículo uno</p>
24 <p>Cita del artículo dos</p>
25 </div>
26 </aside>
27 <div class="recuperar">Este es mi texto</div>
28 </body>
29 </html>
```

```
1  * {
2      margin: 0px;
3      padding: 0px;
4  }
5  section {
6      float: right;
7      width: 100%;
8      padding-left: 260px;
9      margin-left: -240px;
10     box-sizing: border-box;
11 }
12 section > div {
13     padding: 20px;
14     background-color: #999999;
15 }
16 aside {
17     float: left;
18     width: 240px;
19     padding: 20px;
20     background-color: #CCCCCC;
21     box-sizing: border-box;
22 }
23 .recuperar {
24     clear: both;
25     background-color: brown;
26     padding: 5px;
27 }
```



Fijo y flexible

- Si lo que queremos es declarar dos columnas con tamaños fijos a los lados y una columna flexible en el medio usando este mismo mecanismo, tenemos que agrupar las columnas dentro de un contenedor y luego aplicar las propiedades a los elementos dos veces, dentro y fuera del contenedor. En el siguiente documento, agrupamos dos elementos `<section>` dentro de un elemento `<div>` identificado con el nombre contenedor para contener las columnas izquierda y central. Estas dos columnas se han identificado con los nombres `columnaizquierda` y `columnacentral`.
- El código CSS para este documento es sencillo. Los elementos contenedor y `columnacentral` tienen que declararse como flexibles, por lo que tenemos que asignar al contenedor el relleno y el margen adecuados para ubicar el elemento `<aside>` a la derecha, y luego definir el relleno y el margen del elemento `columnacentral` para hacer sitio para la columna fija del lado izquierdo.

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <title>Este texto es el título del documento</title>
5   <meta charset="utf-8">
6   <meta name="description" content="Este es un documento HTML5">
7   <meta name="keywords" content="HTML, CSS, JavaScript">
8   <meta name="viewport" content="width=device-width, initial-scale=1">
9   <link rel="stylesheet" href="misestilos.css">
10 </head>
11 <body>
12   <div id="contenedor">
13     <section id="columnacentral">
14       <div>
15         <article>
16           <h1>Título Primer Artículo</h1>
17           
18         </article>
19       </div>
20     </section>
21     <section id="columnaizquierda">
22       <div>
23         <h1>Opciones</h1>
24         <p>Opción 1</p>
25         <p>Opción 2</p>
26       </div>
27     </section>
28     <div class="recuperar">
29       <footer>
30         Contactos
31       </footer>
32     </div>
33   </div>
34   <aside>
35     <div>
36       <h1>Información</h1>
37       <p>Cita del artículo uno</p>
38       <p>Cita del artículo dos</p>
39     </div>
40   </aside>
41 </body>
42 </html>

```

```

1 * {
2   margin: 0px;
3   padding: 0px;
4 }
5 #contenedor {
6   float: left;
7   width: 100%;
8   padding-right: 260px;
9   margin-right: -240px;
10  box-sizing: border-box;
11 }
12 aside {
13   float: left;
14   width: 240px;
15   padding: 20px;
16   background-color: #CCCCCC;
17   box-sizing: border-box;
18 }
19 #columnacentral {
20   float: right;
21   width: 100%;
22   padding-left: 220px;
23   margin-left: -200px;
24   box-sizing: border-box;
25 }
26 #columnacentral > div {
27   padding: 20px;
28   background-color: #999999;
29 }
30 #columnaizquierda {
31   float: left;
32   width: 200px;
33   padding: 20px;
34   background-color: #CCCCCC;
35   box-sizing: border-box;
36 }
37 .recuperar {
38   clear: both;
39   background-color: brown;
40   padding: 5px;
41   width: 150%; /*aquí podemos
42 }

```

Este texto es el título del document X

+

− □ ×

← → ↺ 🏠

file:///D:/MisOneDrives/OneDrive - Pontificia Universidad Cat ... ☆

⬇ 📁 📄 ☰

Opciones

Opción 1

Opción 2

Título Primer Artículo



Información

Cita del artículo uno

Cita del artículo dos

Contactos

- En el ejemplo anterior declaramos el elemento `columnaizquierda` después del elemento `columnacentral`. Con esto nos aseguramos de que `columnaizquierda` se presenta sobre `columnacentral` y, por lo tanto, es accesible. Como hemos hecho antes, las posiciones en la página web de estos elementos se definen con la propiedad `float`. En consecuencia, el elemento `columnaizquierda` se coloca en el lado izquierdo de la página con un ancho fijo de 200 píxeles, el elemento `columnacentral` se coloca en el centro con un ancho flexible y el elemento `<aside>` a la derecha con un ancho fijo de 240 píxeles.

Texto

- Otro aspecto del diseño que tenemos que adaptar a diferentes dispositivos es el texto. Cuando declaramos un tipo de letra con un tamaño fijo, como 14 píxeles, el texto se muestra siempre en ese tamaño, independientemente del dispositivo. En ordenadores personales puede verse bien, pero en pantallas pequeñas un texto de este tamaño será difícil de leer. Para ajustar el tamaño de la letra al dispositivo, podemos usar un tamaño de letra estándar. Este es un tamaño determinado por el navegador de acuerdo con el dispositivo en el que se ejecuta y ajustado a las preferencias del usuario. Por ejemplo, en ordenadores personales, el tamaño de la fuente por defecto es normalmente 16 píxeles. Para asegurarnos de que el texto de nuestra página es legible, podemos definir su tamaño en relación a este valor. CSS ofrece las siguientes unidades de medida con este propósito.
- **em**—Esta unidad representa una medida relativa al tamaño de la fuente asignado al elemento. Acepta números decimales. El valor 1 es igual al tamaño actual de la fuente.
- **rem**—Esta unidad representa una medida relativa al tamaño de la fuente asignada al elemento raíz (usualmente, el elemento `<body>`). Acepta números decimales. El valor 1 es igual al tamaño actual de la fuente.

Texto

- Las fuentes se definen del mismo modo que lo hemos hecho antes, pero la unidad px se debe reemplazar por la unidad em para declarar el tamaño de la fuente relativo al tamaño de fuente actual asignado al elemento. Por ejemplo, la siguiente hoja de estilo define tamaños de letra para los elementos `<h1>` y `<p>` en nuestro documento usando unidades em.
- La unidad em reemplaza a la unidad px y, por lo tanto, el navegador es responsable de calcular el tamaño del texto en píxeles a partir del tamaño de la fuente actualmente asignada al elemento. Para calcular este valor, el navegador multiplica el número de em por el tamaño actual de la fuente estándar. Por ejemplo, la regla article h1 del Listado 5-14 asigna un tamaño de 2em al elemento `<h1>` dentro de los elementos `<article>`. Esto significa que en un ordenador de escritorio el texto en estos elementos tendrá un tamaño de 32 píxeles ($16 * 2$).

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <title>Este texto es el título del documento</title>
5   <meta charset="utf-8">
6   <meta name="description" content="Este es un documento HTML5">
7   <meta name="keywords" content="HTML, CSS, JavaScript">
8   <meta name="viewport" content="width=device-width, initial-scale=1">
9   <link rel="stylesheet" href="adaptatabletodos9.css">
10 </head>
11 <body>
12   <section>
13     <div>
14       <article>
15         <h1>Título Primer Artículo</h1>
16         
17       </article>
18     </div>
19   </section>
20   <aside>
21     <div>
22       <h1>Información</h1>
23       <p>Cita del artículo uno</p>
24       <p>Cita del artículo dos</p>
25     </div>
26   </aside>
27   <div class="recuperar">Este es mi texto</div>
28 </body>
29 </html>
```

```
1 * {
2   margin: 0px;
3   padding: 0px;
4 }
5 section {
6   float: left;
7   width: 61%;
8   padding: 2%;
9   margin-right: 5%;
10  background-color: #999999;
11 }
12 aside {
13   float: left;
14   width: 26%;
15   padding: 2%;
16   background-color: #CCCCCC;
17 }
18 .recuperar {
19   clear: both;
20 }
21 article h1 {
22   font: bold 2em Georgia, sans-serif;
23 }
24 aside h1 {
25   font: bold 1.2em Georgia, sans-serif;
26 }
27 aside p {
28   font: 1em Georgia, sans-serif;
29 }
```

Este texto es el título del document X

←

→

↺

🏠

📄

- Pontificia Universidad Católica del Ecuador/GoogleDrive-B

⋮

🛡️

★

⬇️

📖

📄

☰

Título Primer Artículo



Información

Cita del artículo uno

Cita del artículo dos

Cita del artículo tres

Este es mi texto

Texto

- La unidad em define el tamaño relativo al tamaño de fuente asignado al elemento. Esto se debe a que los elementos heredan propiedades de sus contenedores y, por lo tanto, sus tamaños de letra pueden ser diferentes del estándar establecido por el navegador. Esto significa que si modificamos el tamaño de letra del contenedor de un elemento, el tamaño de letra asignado al elemento se verá afectado. Por ejemplo, las siguientes reglas asignan un tamaño de letra al elemento <article> y otro al elemento <h1> en su interior.
- La primera regla asigna un tamaño de letra de 1.5em al elemento <article>. Después de aplicarse esta propiedad, el contenido del elemento <article>, incluido el contenido del elemento <h1> en su interior, tendrá un tamaño de 1.5 veces el tamaño estándar (24 píxeles en un ordenador personal). El elemento <h1> hereda este valor y, por lo tanto, cuando se aplica la siguiente regla, el tamaño de la letra no se calcula desde el tamaño estándar sino desde el tamaño establecido por el contenedor ($16 * 1.5 * 2 = 48$).

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <title>Este texto es el título del documento</title>
5    <meta charset="utf-8">
6    <meta name="description" content="Este es un documento HTML5">
7    <meta name="keywords" content="HTML, CSS, JavaScript">
8    <meta name="viewport" content="width=device-width, initial-scale=1">
9    <link rel="stylesheet" href="adaptatabletodos10.css">
10 </head>
11 <body>
12   <section>
13     <div>
14       <article>
15         <h1>Título Primer Artículo</h1>
16         
17       </article>
18     </div>
19   </section>
20   <aside>
21     <div>
22       <h1>Información</h1>
23       <p>Cita del artículo uno</p>
24       <p>Cita del artículo dos</p>
25       <p>Cita del artículo tres</p>
26     </div>
27   </aside>
28   <div class="recuperar">Este es mi texto</div>
29 </body>
30 </html>
```

```
1  * {
2    margin: 0px;
3    padding: 0px;
4  }
5  section {
6    float: left;
7    width: 61%;
8    padding: 2%;
9    margin-right: 5%;
10   background-color: #999999;
11 }
12 aside {
13   float: left;
14   width: 26%;
15   padding: 2%;
16   background-color: #CCCCCC;
17 }
18 .recuperar {
19   clear: both;
20 }
21
22 article {
23   font: bold 1.5em Georgia, sans-serif;
24 }
25 article h1 {
26   font: bold 2em Georgia, sans-serif;
27 }
```

Este texto es el título del documenti X

Este texto es el título del documenti X



file:///D:/MisOneDrives/OneDrive - Pontificia Universidad Católica de



Título Primer Artículo



Información

Cita del artículo uno

Cita del artículo dos

Cita del artículo tres

Este es mi texto

Texto

- Si queremos cambiar este comportamiento y forzar al navegador a calcular el tamaño de letra siempre desde el valor estándar, podemos usar las unidades rem. Estas unidades son relativas al elemento raíz en lugar del elemento actual y, por lo tanto, los valores siempre se multiplican por el mismo valor base.

```
article {  
    font: bold 1.5rem Georgia, sans-serif;  
}  
article h1 {  
    font: bold 2rem Georgia, sans-serif;  
}
```

Imágenes

- Las imágenes se muestran por defecto en su tamaño original, lo que significa que se adaptan al espacio disponible a menos que lo declaremos de forma explícita. Para convertir una imagen fija en una imagen flexible tenemos que declarar su tamaño en porcentaje. Por ejemplo, la siguiente regla configura el ancho de los elementos `<img>` dentro de los elementos `<article>` de nuestro documento con un valor de 100 % y, por lo tanto, las imágenes tendrán un ancho igual al de su contenedor.

```
article img {  
    width: 100%;  
}
```

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <title>Este texto es el título del documento</title>
5      <meta charset="utf-8">
6      <meta name="description" content="Este es un documento HTML5">
7      <meta name="keywords" content="HTML, CSS, JavaScript">
8      <meta name="viewport" content="width=device-width, initial-scale=1">
9      <link rel="stylesheet" href="adaptatabletodos11.css">
10 </head>
11 <body>
12     <section>
13         <div>
14             <article>
15                 <h1>Título Primer Artículo</h1>
16                 
17             </article>
18         </div>
19     </section>
20     <aside>
21         <div>
22             <h1>Información</h1>
23             <p>Cita del artículo uno</p>
24             <p>Cita del artículo dos</p>
25             <p>Cita del artículo tres</p>
26         </div>
27     </aside>
28     <div class="recuperar">Este es mi texto</div>
29 </body>
30 </html>
```

```
1  * {
2      margin: 0px;
3      padding: 0px;
4  }
5  section {
6      float: left;
7      width: 61%;
8      padding: 2%;
9      margin-right: 5%;
10     background-color: #999999;
11 }
12 aside {
13     float: left;
14     width: 26%;
15     padding: 2%;
16     background-color: #CCCCCC;
17 }
18 .recuperar {
19     clear: both;
20 }
21 article {
22     font: bold 1.5em Georgia, sans-serif;
23 }
24 article h1 {
25     font: bold 2em Georgia, sans-serif;
26 }
27 article img {
28     width: 100%;
29 }
```



Título Primer Artículo



Información

Cita del artículo uno

Cita del artículo dos

Cita del artículo tres

Imágenes

- Al asignar un porcentaje al ancho de la imagen, se fuerza al navegador a calcular el tamaño de la imagen de acuerdo con el tamaño de su contenedor (el elemento <article> en nuestro documento). También podemos declarar valores menores a 100 %, pero el tamaño de la imagen siempre será proporcional al tamaño de su contenedor, lo cual significa que si el contenedor se expande, la imagen se podría presentar con un tamaño más grande que el original. Si queremos establecer los límites para expandir o encoger la imagen, tenemos que usar las propiedades max-width y min-width. Ya hemos visto estas propiedades aplicadas en el modelo de caja flexible, pero también se pueden emplear en el modelo de caja tradicional para declarar límites para elementos flexibles. Por ejemplo, si queremos que la imagen se reduzca solo cuando no hay espacio suficiente para mostrarla en su tamaño original, podemos usar la propiedad max-width.

```
article img {  
    max-width: 100%;  
}
```

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <title>Este texto es el título del documento</title>
5      <meta charset="utf-8">
6      <meta name="description" content="Este es un documento HTML5">
7      <meta name="keywords" content="HTML, CSS, JavaScript">
8      <meta name="viewport" content="width=device-width, initial-scale=1">
9      <link rel="stylesheet" href="adaptabletodos12.css">
10 </head>
11 <body>
12     <section>
13         <div>
14             <article>
15                 <h1>Título Primer Artículo</h1>
16                 
17             </article>
18         </div>
19     </section>
20     <aside>
21         <div>
22             <h1>Información</h1>
23             <p>Cita del artículo uno</p>
24             <p>Cita del artículo dos</p>
25             <p>Cita del artículo tres</p>
26         </div>
27     </aside>
28     <div class="recuperar">Este es mi texto</div>
29 </body>
30 </html>
```

```
1  * {
2      margin: 0px;
3      padding: 0px;
4  }
5  section {
6      float: left;
7      width: 61%;
8      padding: 2%;
9      margin-right: 5%;
10     background-color: #999999;
11 }
12 aside {
13     float: left;
14     width: 26%;
15     padding: 2%;
16     background-color: #CCCCCC;
17 }
18 .recuperar {
19     clear: both;
20 }
21 article {
22     font: bold 1.5em Georgia, sans-serif;
23 }
24 article h1 {
25     font: bold 2em Georgia, sans-serif;
26 }
27 article img {
28     max-width: 100%;
29 }
```

Este texto es el título del document X

← → ↺ 🏠

file:///D:/MisOneDrives/OneDrive - Pontificia Univ

⋮ 🛡️ ☆

⬇️ 📁 📖 ☰

Título Primer Artículo



Información

Cita del artículo uno
Cita del artículo dos
Cita del artículo tres

Este es mi texto

Este texto es el título del d X

← → ↺

file:

⋮ 🛡️ ☆

⬇️ >> ☰

Título Primer Artículo



Información

Cita del artículo uno
Cita del artículo dos
Cita del artículo tres

Este es mi texto

<

>

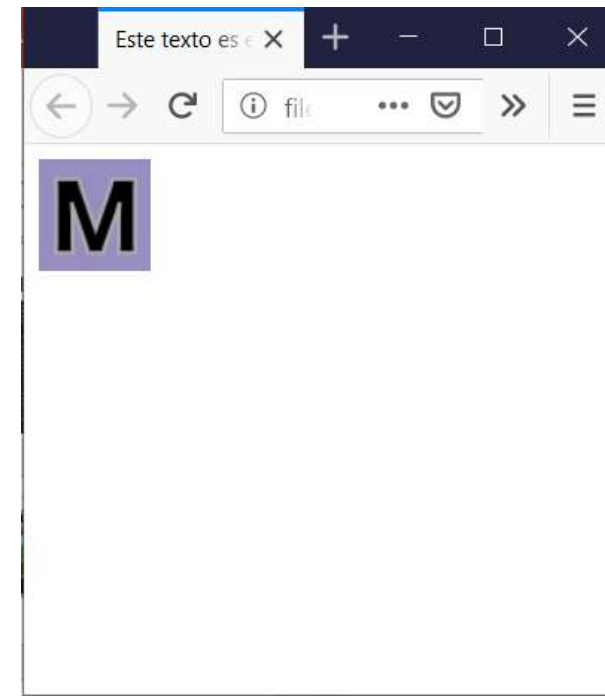
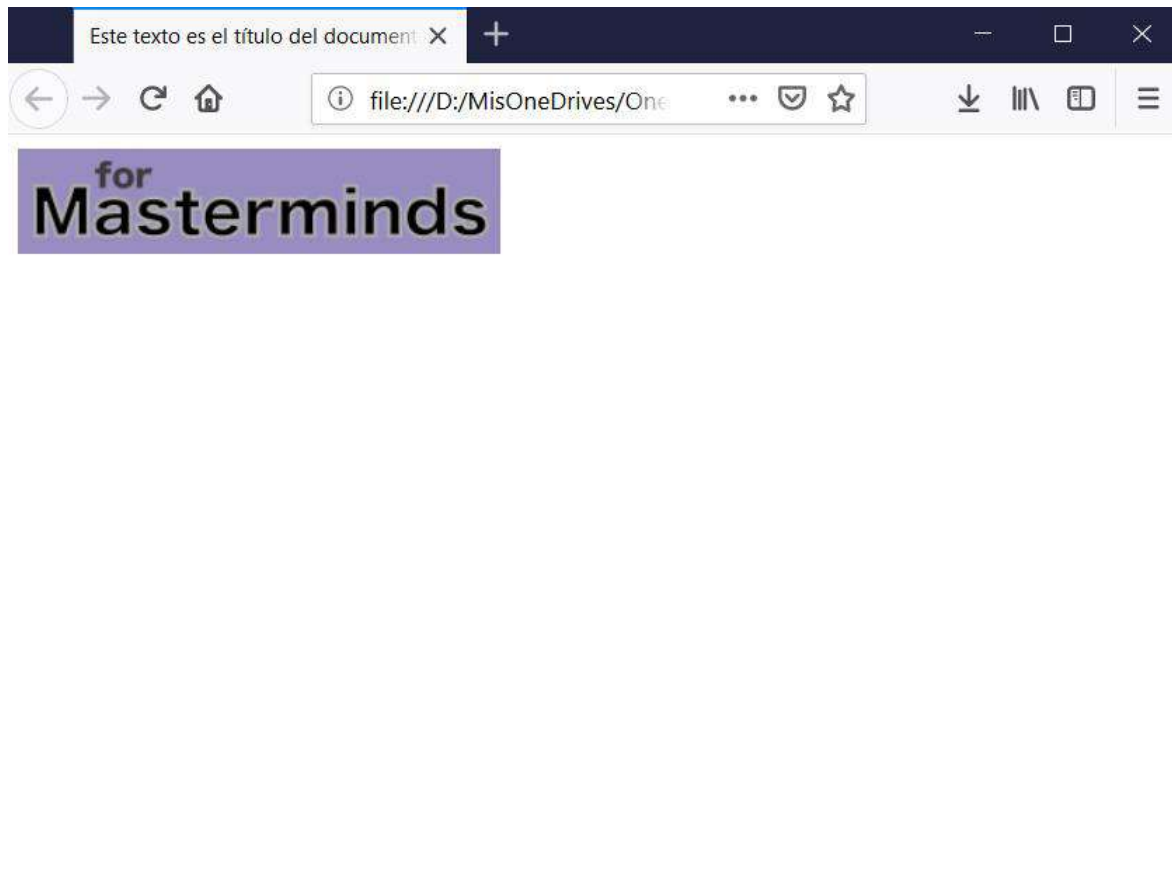
Imágenes

- Además de adaptar la imagen al espacio disponible, un sitio web adaptable también necesita que unas imágenes se reemplacen por otras cuando las condiciones cambian demasiado. Existen al menos dos situaciones en las que esto es necesario: cuando el espacio disponible no es suficiente para mostrar la imagen original (cuando el logo del sitio web tiene que ser reemplazado por una versión reducida, por ejemplo) o cuando la densidad de píxeles de la pantalla es diferente y necesitamos mostrar una imagen con una resolución más alta o más baja. Independientemente del motivo, HTML ofrece el siguiente elemento para seleccionar la imagen a mostrar.
- **<picture>**—Este elemento es un contenedor que nos permite especificar múltiples fuentes para el mismo elemento `<img>`.
- **<source>**—Este elemento define una posible fuente para el elemento `<img>`. Puede incluir el atributo `media`, para especificar la Media Query a la que la imagen estará asociada, y el atributo `srcset` para especificar la ruta de las imágenes que queremos declarar como fuentes del elemento.

Imágenes

- El elemento `<picture>` es solo un contenedor que debe incluir uno o más elementos `<source>` para especificar las posibles fuentes de la imagen y un elemento `<img>` al final para mostrar la imagen seleccionada en pantalla. El siguiente documento ilustra cómo reemplazar el logo de un sitio web por una versión simplificada en dispositivos con pantallas pequeñas usando estos elementos.

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <title>Este texto es el título del documento</title>
5          <meta charset="utf-8">
6          <meta name="description" content="Este es un documento HTML5">
7          <meta name="keywords" content="HTML, CSS, JavaScript">
8          <meta name="viewport" content="width=device-width, initial-scale=1">
9      </head>
10     <body>
11         <header>
12             <picture>
13                 <source media="(max-width: 480px)" srcset="logoreducido.jpg">
14                 
15             </picture>
16         </header>
17     </body>
18 </html>
```



Imágenes

- Aunque este es probablemente el escenario más común en el que una imagen debe ser reemplazada por otra de diferente tamaño o contenido, existe una condición más que debemos considerar cuando mostramos imágenes a nuestros usuarios. Desde que se introdujeron los monitores de tipo retina por parte de Apple en el año 2010, existen pantallas con una densidad más alta de píxeles. Una densidad más alta significa más píxeles por pulgada, lo que se traduce en imágenes más claras y definidas. Pero si queremos aprovechar esta característica, debemos facilitar imágenes de alta resolución en estas pantallas. Por ejemplo, una imagen de 300 píxeles de ancho por 200 píxeles de alto dentro de un área del mismo tamaño se verá bien en pantallas con una densidad normal, pero los dispositivos con alta densidad de píxeles tendrán que estirar la imagen para ocupar todos los píxeles disponibles en la misma área. Si queremos que esta imagen también se vea bien en pantallas retina con una escala de 2 (doble cantidad de píxeles por área), tenemos que reemplazarla por la misma imagen con una resolución dos veces superior (600 x 400 píxeles).
- Las Media Queries contemplan esta situación con la palabra clave `resolution`. Esta palabra clave requiere que el valor se declare con un número entero y la unidad `dppx`. El valor 1 representa una resolución estándar. Valores más altos definen la escala de la pantalla. Actualmente, las pantallas retina tienen escalas de 2 y 3. El siguiente documento declara una fuente específica para pantallas con una escala de 2 (las más comunes).

```
1 <!DOCTYPE html>
2 <html lang="es">
3   <head>
4     <title>Este texto es el título del documento</title>
5     <meta charset="utf-8">
6     <meta name="description" content="Este es un documento HTML5">
7     <meta name="keywords" content="HTML, CSS, JavaScript">
8     <meta name="viewport" content="width=device-width, initial-scale=1">
9     <link rel="stylesheet" href="adaptatabletodos13.css">
10  </head>
11  <body>
12    <section>
13      <div>
14        <article>
15          <h1>Título Primer Artículo</h1>
16          <picture>
17            <source media="(resolution: 2dppx)" srcset="miimagen@2x.jpg">
18            
19          </picture>
20        </article>
21      </div>
22    </section>
23    <aside>
24      <div>
25        <h1>Información</h1>
26        <p>Cita del artículo uno</p>
27        <p>Cita del artículo dos</p>
28      </div>
29    </aside>
30    <div class="recuperar"></div>
31  </body>
32 </html>
```

```
1 * {
2   margin: 0px;
3   padding: 0px;
4 }
5 section {
6   float: left;
7   width: 61%;
8   padding: 2%;
9   margin-right: 5%;
10  background-color: #999999;
11 }
12 aside {
13   float: left;
14   width: 26%;
15   padding: 2%;
16   background-color: #CCCCCC;
17 }
18 .recuperar {
19   clear: both;
20 }
21 article {
22   font: bold 1.5em Georgia, sans-serif;
23 }
24 article h1 {
25   font: bold 2em Georgia, sans-serif;
26 }
27 article img{
28   width: 100%;
29 }
```

Imágenes

- Existe una convención que sugiere declarar los nombres de los archivos que contienen las imágenes de alta resolución con el mismo nombre que la imagen estándar seguido del carácter @, el valor de la escala, y la letra x, como en @2x o @3x. Esta es la razón por la que asignamos el nombre miimagen@2x.jpg al archivo que contiene la imagen de alta resolución.

Imágenes

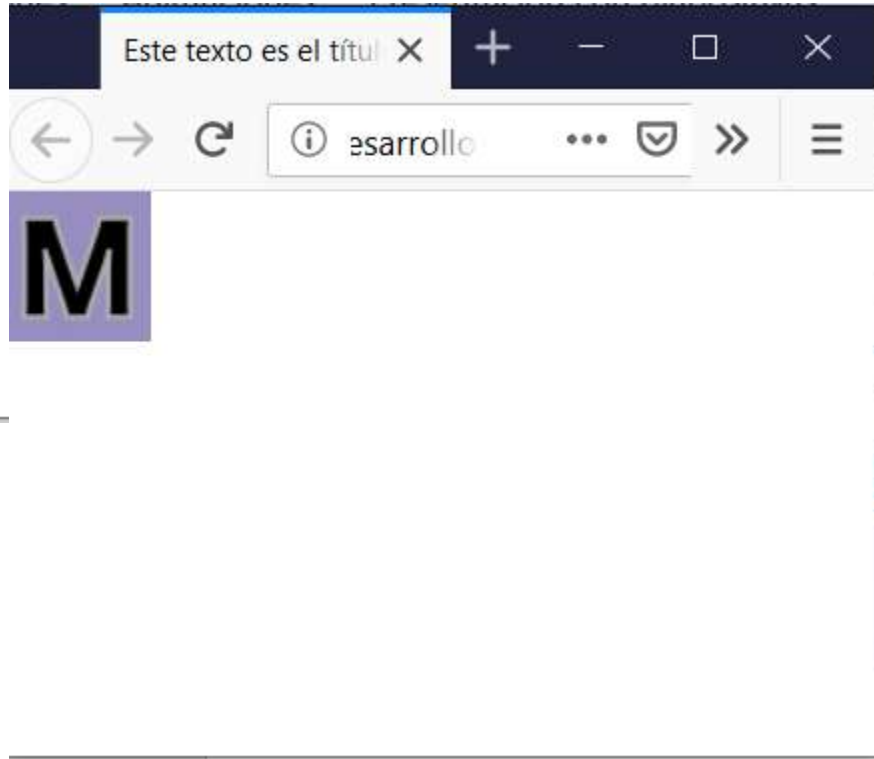
- Al igual que las imágenes introducidas con el elemento `<img>`, las imágenes de fondo también se pueden reemplazar, pero en este caso no necesitamos ningún elemento HTML porque podemos llevar a cabo todo el proceso desde CSS usando las propiedades `background` o `background-image`. Por este motivo, la imagen no se define en el documento, sino en la hoja de estilo. El documento solo tiene que incluir el elemento al cual le asignaremos la imagen de fondo.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <title>Este texto es el título del documento</title>
5      <meta charset="utf-8">
6      <meta name="description" content="Este es un documento HTML5">
7      <meta name="keywords" content="HTML, CSS, JavaScript">
8      <meta name="viewport" content="width=device-width, initial-scale=1">
9      <link rel="stylesheet" href="misestilos.css">
10 </head>
11 <body>
12     <header>
13         <div id="logo"></div>
14     </header>
15 </body>
16 </html>
```

```
1  * {
2      margin: 0px;
3      padding: 0px;
4  }
5  #logo {
6      width: 275px;
7      height: 60px;
8      background: url("logo.jpg") no-repeat;
9  }
10 @media (max-width: 480px) {
11     #logo {
12         width: 60px;
13         background: url("logoreducido.jpg") no-repeat;
14     }
15 }
```



for
Masterminds



Recurso de profundización – Semana 3

Título del recurso: Construcción de un *portfolio responsivo* con CSS Grid y Flexbox

Descripción del recurso: Este recurso es una guía práctica extendida que muestra cómo aplicar Flexbox y Grid junto con Media Queries para crear un layout de *portfolio responsivo*, integrando lo visto en los subtemas 3.1 a 3.4.

Objetivo: Profundizar en la comprensión del diseño adaptable mediante un caso práctico completo.

Paso 1. Estructura básica en HTML

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Portfolio Responsivo</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <header class="header">
    <h1>Mi Portfolio</h1>
    <nav class="nav">
      <a href="#">Inicio</a>
      <a href="#">Proyectos</a>
      <a href="#">Contacto</a>
    </nav>
  </header>

  <main class="container">
    <section class="about">
      <h2>Sobre mí</h2>
      <p>Soy estudiante de desarrollo web, apasionado por el diseño adaptable y accesible.</p>
    </section>

    <section class="projects">
      <article class="project">Proyecto 1</article>
      <article class="project">Proyecto 2</article>
      <article class="project">Proyecto 3</article>
      <article class="project">Proyecto 4</article>
    </section>
  </main>

  <footer class="footer">
    <p>© 2025 - Mi Nombre</p>
  </footer>
</body>
</html>
```

Paso 2. Estilos base con CSS (Mobile First)

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: Arial, sans-serif;
  line-height: 1.5;
  color: #222;
  background: #f9f9f9;
}
```

```

/* Header con Flexbox */
.header {
  display: flex;
  flex-direction: column;
  align-items: center;
  background: #0366d6;
  color: #fff;
  padding: 1rem;
}
.nav a {
  color: #fff;
  text-decoration: none;
  margin: 0 .5rem;
}

/* Contenido principal */
.container {
  padding: 1rem;
}

/* Grid de proyectos (1 columna por defecto) */
.projects {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
  margin-top: 1rem;
}
.project {
  background: #fff;
  border: 1px solid #ddd;
  border-radius: .5rem;
  padding: 1rem;
  text-align: center;
}

```

Paso 3. Adaptación con Media Queries

```

/* Tablet */
@media (min-width: 768px) {
  .header { flex-direction: row; justify-content: space-between; }
  .projects { grid-template-columns: repeat(2, 1fr); }
}

/* Escritorio */
@media (min-width: 1200px) {
  .projects { grid-template-columns: repeat(4, 1fr); }
  .container { max-width: 1200px; margin: auto; }
}

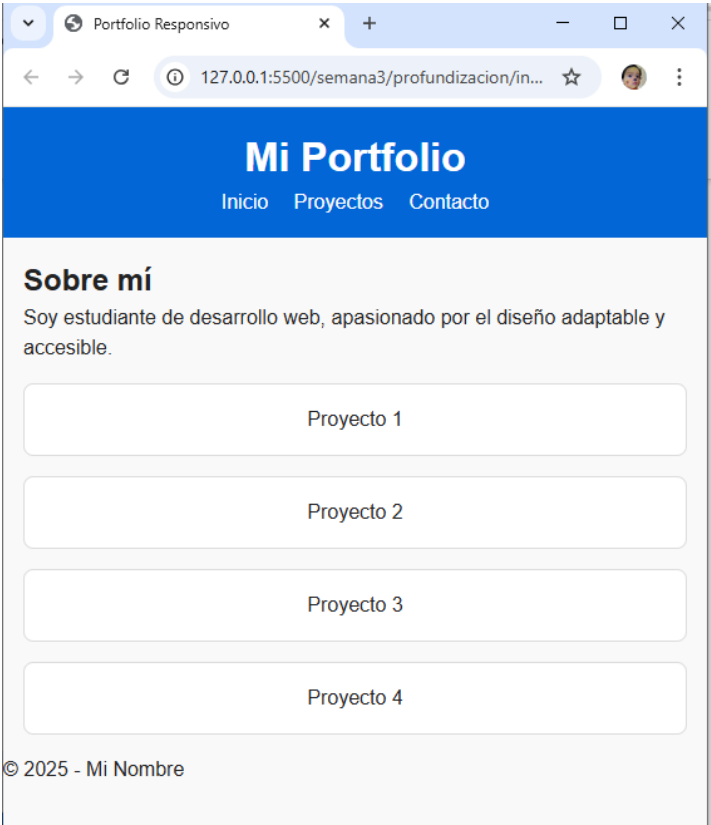
```

Paso 4. Reflexión guiada

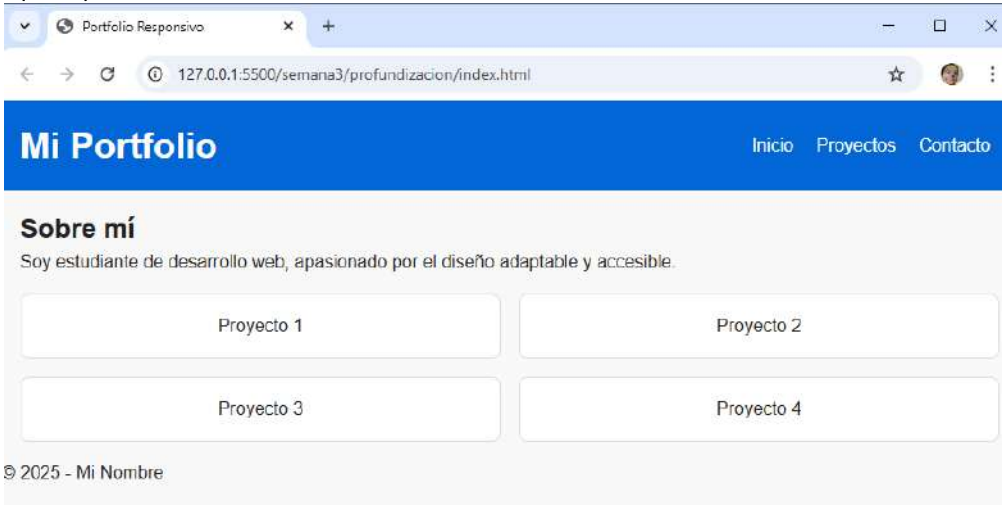
- ¿Qué papel juega Flexbox en el header?
Permite alinear logo y navegación horizontalmente en pantallas grandes.
- ¿Por qué Grid es mejor para los proyectos?
Porque distribuye automáticamente en columnas, ajustando el número según el ancho.
- ¿Qué hubiera pasado si usáramos solo float o medidas fijas en píxeles?
El diseño se rompería en móviles o pantallas grandes.

Ejemplo visual del resultado

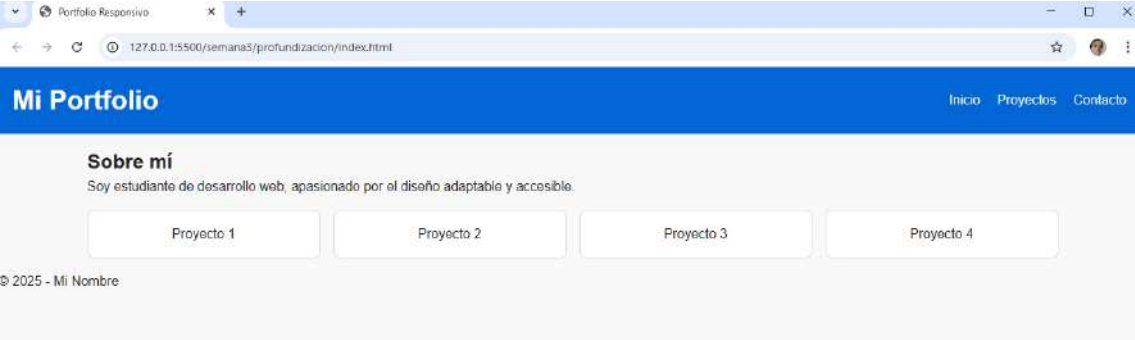
Móvil (1 col):



Tablet (2 col):



Escritorio (4 col):



Enlace externo recomendado

Título del enlace relacionado: CSS Grid — Guía completa (MDN Web Docs)

Descripción del enlace relacionado: Recurso oficial de MDN que profundiza en el uso de Grid Layout con ejemplos prácticos y casos de uso avanzados.

Enlace: https://developer.mozilla.org/es/docs/Web/CSS/CSS_grid_layout